

Neural Network

Ratchainant Thammasudjarit, Ph.D.

Prerequisite Mathematics

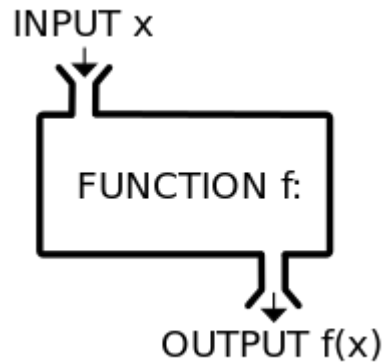
Recall your linear algebra

Linear Algebra

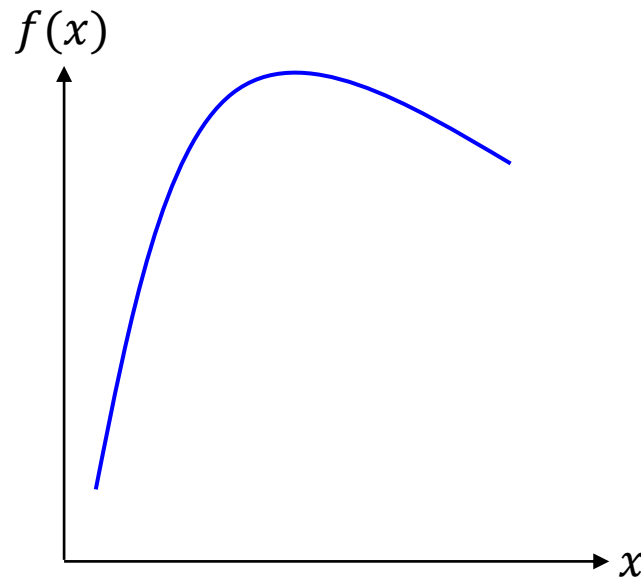
Function

- A function of univariate is defined as

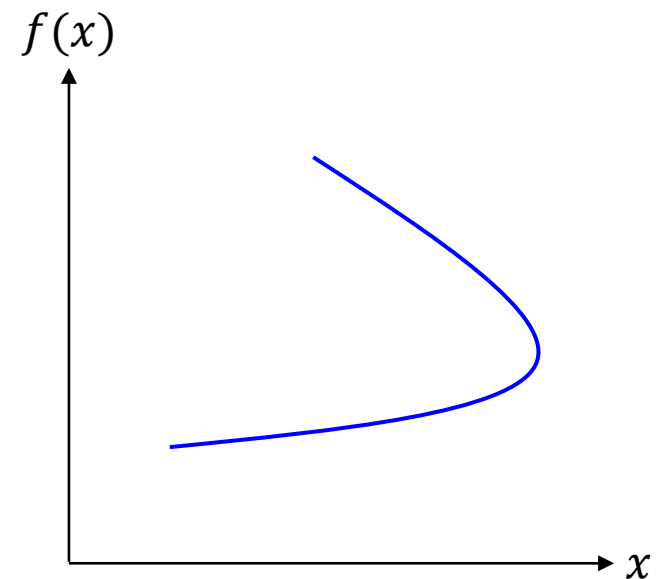
$$x \mapsto f(x)$$



Each input must have only one output



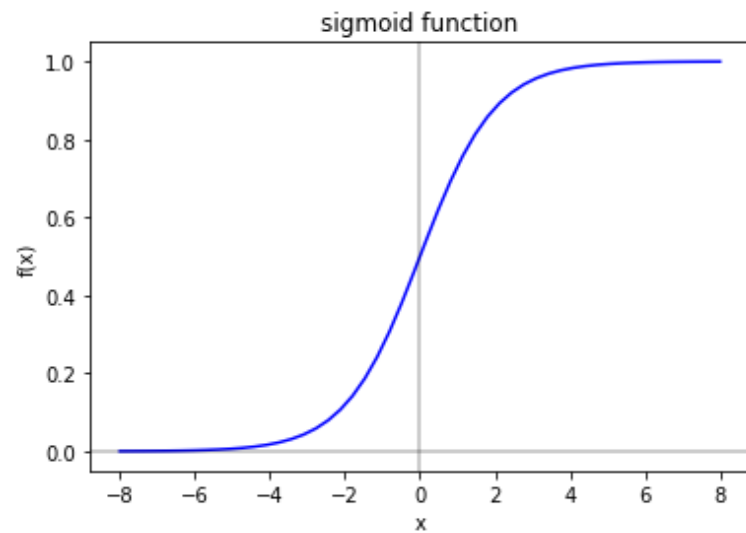
This is a function



This is NOT a function

Linear Algebra

sigmoid

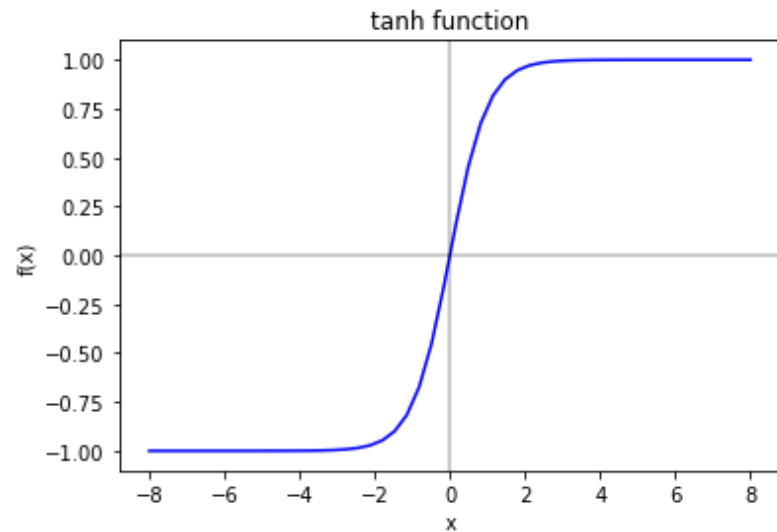


- Properties

- $f(x) \in [0, 1]$
- $x \in [-\infty, \infty]$
- $x \rightarrow 0$, $f(x)$ becomes linear
- $\text{abs}(x) > 4$, $f(x)$ changes slowly

Linear Algebra

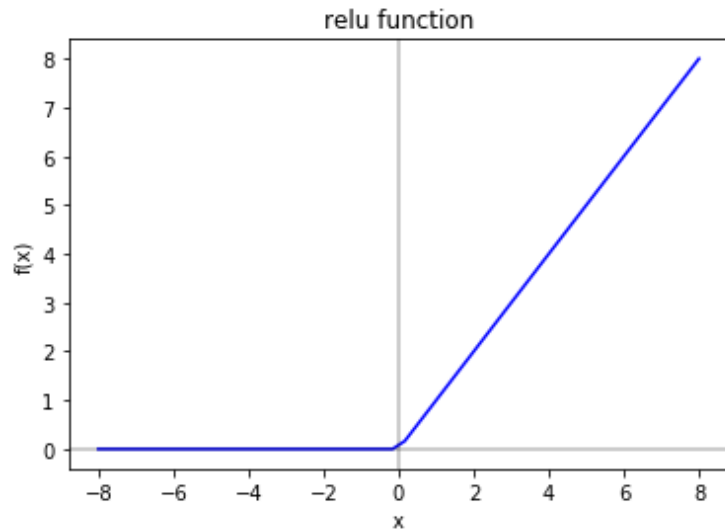
tanh



- Properties
 - $f(x) \in [-1, 1]$
 - $x \in [-\infty, \infty]$
 - $x \rightarrow 0$, $f(x)$ becomes linear
 - $\text{abs}(x) > 2$, $f(x)$ changes slowly

Linear Algebra

ReLU (Rectifier Linear Unit)



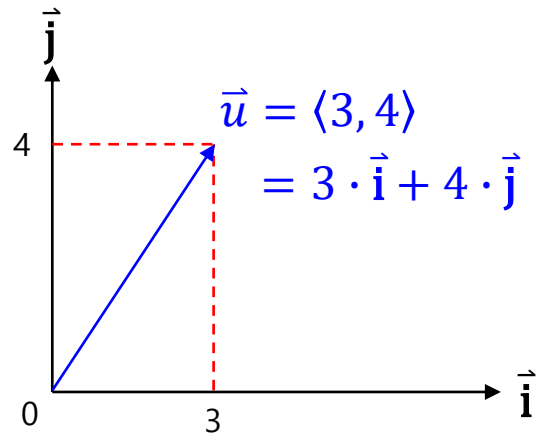
```
def relu(x):  
    y = np.maximum(0, x)  
    return y
```

- Properties

- $f(x) \in [0, \infty]$
- $x \in [-\infty, \infty]$
- $x \leq 0, f(x) = 0$
- $x > 0, f(x) = x$

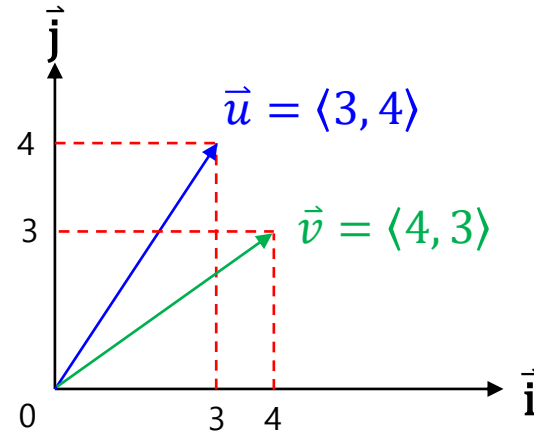
Linear Algebra

Vector



$$\|\vec{u}\| = \sqrt{3^2 + 4^2} = 5$$

- Dot product



$$\vec{u} \cdot \vec{v} = (3 \times 4) + (4 \times 3) = 24$$

- Transpose

if

$$\mathbf{u} = [3 \quad 4]$$

then

$$\mathbf{u}^T = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Linear Algebra

Matrix

A stack of vectors

- Pairwise multiplication

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix}$$

$$A \odot B = \begin{bmatrix} 1 \times 2 & 2 \times 2 & 3 \times 2 \\ 4 \times 2 & 5 \times 2 & 6 \times 2 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}$$

- Transpose

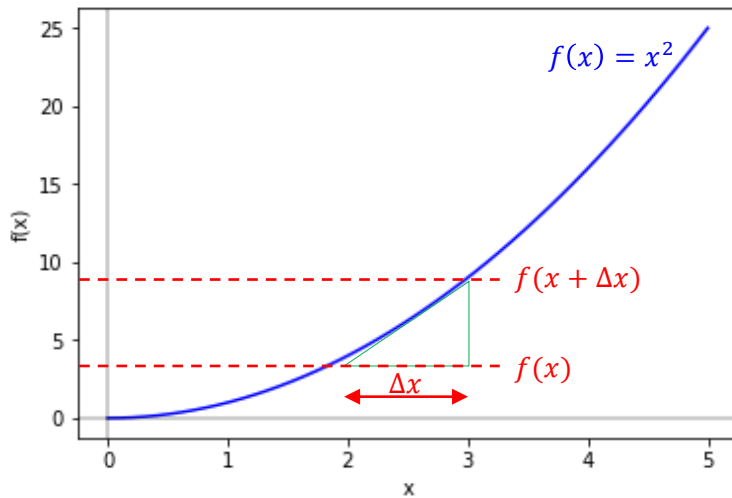
$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Linear Algebra

Calculus (Derivative)

- Derivative

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$



$\Delta x = 1$	—
if $x = 2$	then $f(x) = 4$
if $x = 3$	then $f(x) = 9$

$\Delta x = 0.001$	—
if $x = 2$	then $f(x) = 4$
if $x = 2.001$	then $f(x) \approx 4.004$

$$\text{if } x = 2 \text{ then slope} = \frac{\Delta f(x)}{\Delta x} = \frac{0.004}{0.001} = 4$$

$$\text{Mathematically, } \frac{d}{dx} f(x) = 2x = 2(2) = 4$$

Linear Algebra

Derivative of sigmoid

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$\begin{aligned}\frac{d}{dz}g(z) &= \frac{d}{dz} \left[\frac{1}{1 + e^{-z}} \right] \\ &= \frac{d}{dz} (1 + e^{-z})^{-1} \\ &= -(1 + e^{-z})^{-2} \frac{d}{dz} (1 + e^{-z}) \\ &= -(1 + e^{-z})^{-2} \frac{d}{dz} (e^{-z})\end{aligned}$$

$$\begin{aligned}&= -(1 + e^{-z})^{-2} (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\ &= \frac{1}{1 + e^{-z}} \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \\ &= \frac{1}{1 + e^{-z}} \left[\frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}} \right] \\ &= \frac{1}{1 + e^{-z}} \left[1 - \frac{1}{1 + e^{-z}} \right]\end{aligned}$$

$$\therefore \frac{d}{dz}g(z) = g(z)(1 - g(z))$$

Linear Algebra

Derivative of tanh

$$g(z) = \tanh(z)$$

$$= \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\frac{d}{dz} g(z) = \frac{(e^z + e^{-z})d(e^z - e^{-z}) - (e^z - e^{-z})d(e^z + e^{-z})}{(e^z + e^{-z})^2}$$

$$= \frac{(e^z + e^{-z})(e^z + e^{-z}) - (e^z - e^{-z})(e^z - e^{-z})}{(e^z + e^{-z})^2}$$

$$= 1 - \left(\frac{e^z - e^{-z}}{e^z + e^{-z}} \right)^2$$

$$\therefore \frac{d}{dz} g(z) = 1 - \tanh^2(z)$$

Linear Algebra

Derivative of relu

$$g(z) = \max(0, z)$$

$$\frac{d}{dz}g(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \end{cases}$$

In neural network practice, z can get close to zero but never be zero, e.g. 0.00000 ...

$$\therefore \frac{d}{dz}g(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \end{cases}$$

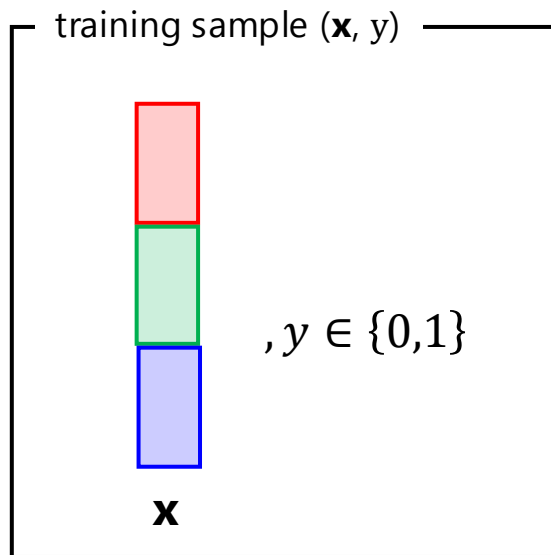
Perceptron

A smallest unit in neural network

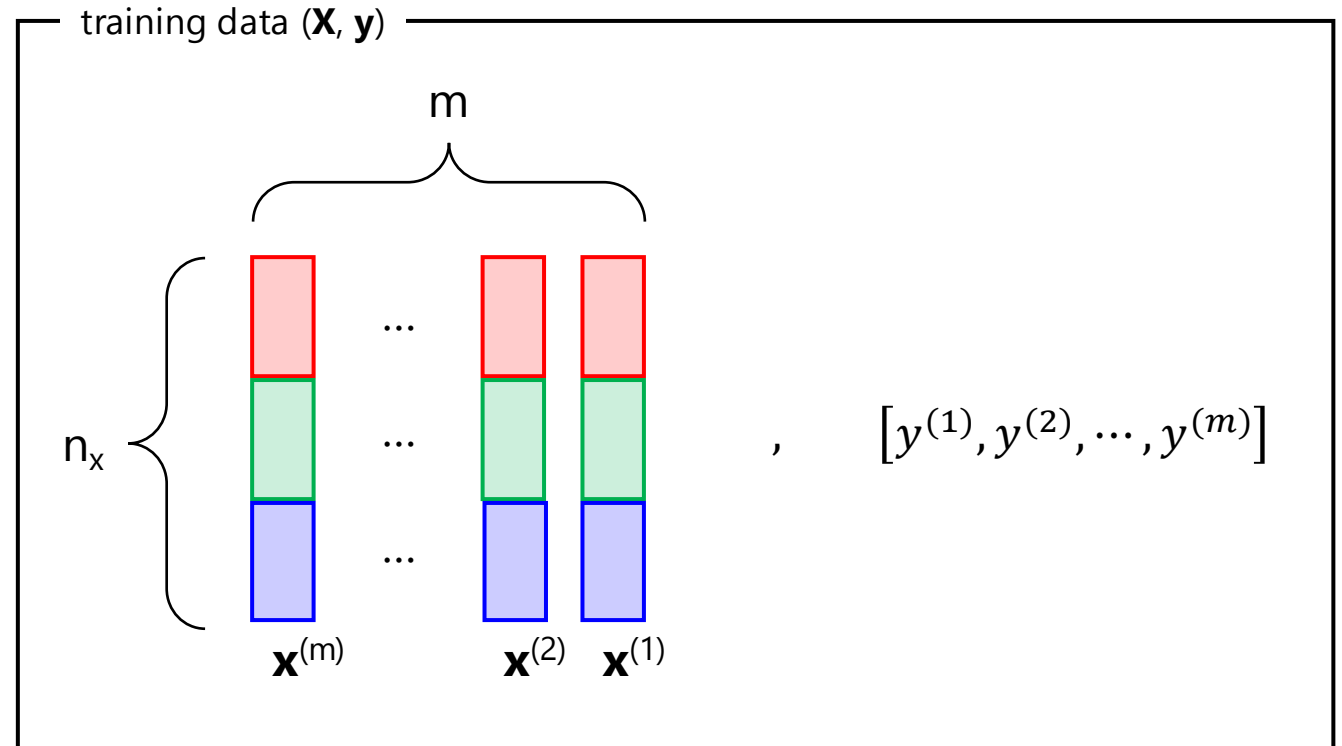
Training Sample VS Training Data

Training sample is a pair of feature vector $\mathbf{x}$ and its class label

$$(\mathbf{x}, y), \mathbf{x} \in \mathbb{R}^{n_x}, y \in \{0,1\}$$



Training data is m pair of feature vector $\mathbf{x}$ and its class label



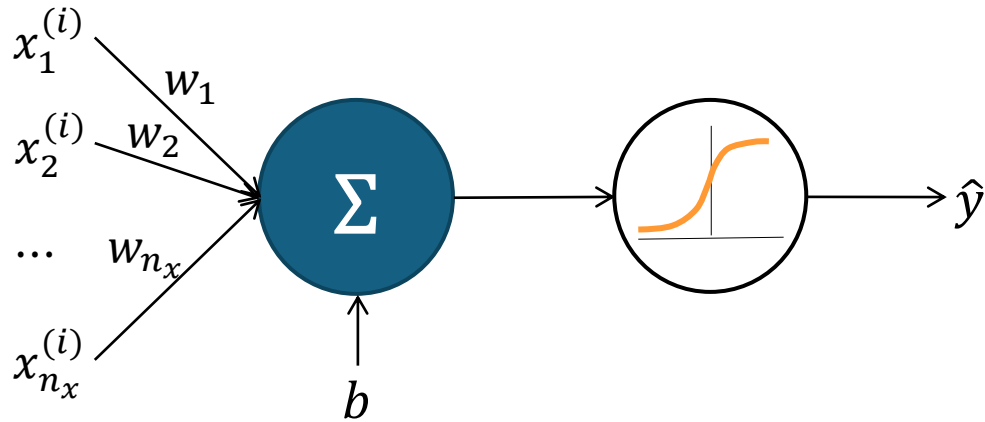
$$\mathbf{x} \in \mathbb{R}^{n_x \times 1}$$

$$\mathbf{X} \in \mathbb{R}^{n_x \times m}$$

$$\mathbf{y} \in \mathbb{R}^{1 \times m}$$

A Perceptron

The smallest unit of neural network

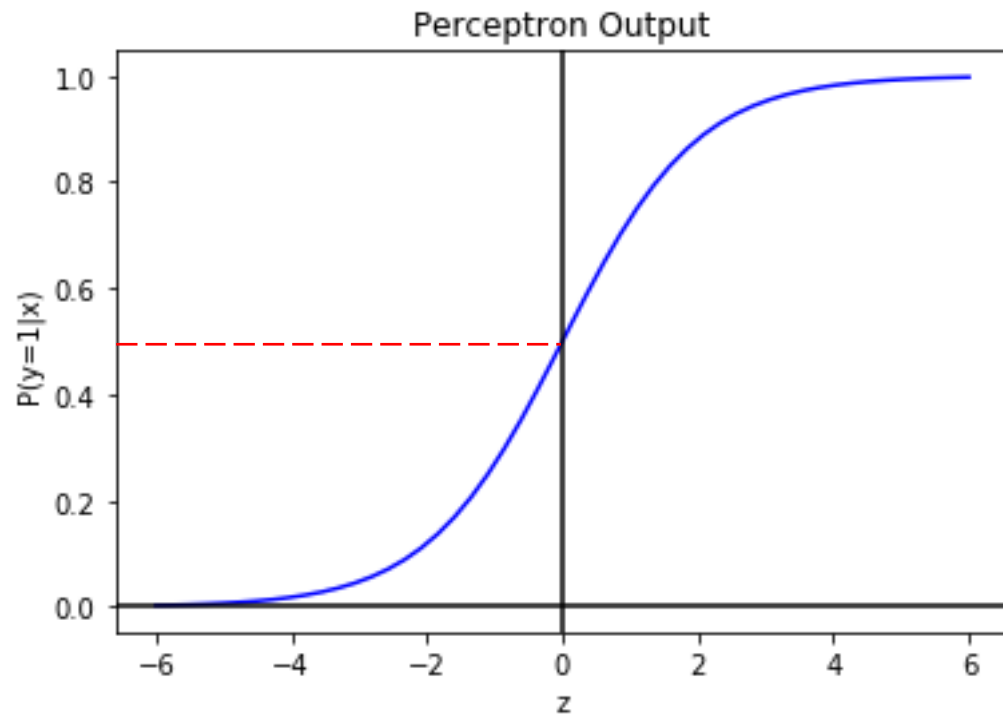


$$\hat{y} = P(y = 1 | \mathbf{x}^{(i)}) = \frac{1}{1 + e^{-(\mathbf{w}^T \cdot \mathbf{x}^{(i)} + b)}}$$

- Given $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x \times 1}$, determine
 - $\hat{y} = P(y = 1 | \mathbf{x}^{(i)})$
- Parameters
 - $\mathbf{w} \in \mathbb{R}^{n_x \times 1}$
 - $b \in \mathbb{R}$
- Output
 - $\hat{y} = g(z) = g(\mathbf{w}^T \cdot \mathbf{x}^{(i)} + b)$

A Perceptron

The smallest unit of neural network



$$g(z) = \frac{1}{1 + e^{-z}}$$

- If $z \gg 0, g(z) \rightarrow 1$
- if $z \ll 0, g(z) \rightarrow 0$

Loss Function

Error of a training sample

The loss function $\mathcal{L}(\hat{y}, y)$ determines how close of $\hat{y}$ to the ground-truth y

Error of a training sample

The loss function $\mathcal{L}(\hat{y}, y)$ determines how close of $\hat{y}$ to the ground-truth y

Binary cross entropy

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} - (1 - y) \log(1 - \hat{y}))$$

$$\text{Goal: } \mathcal{L}(\hat{y}, y) \rightarrow 0$$

if $y = 1$,

$$\therefore z = w^T \cdot x + b, \quad \rightarrow \infty$$

$$\hat{y} = \frac{1}{1 + e^{-z}}, \quad \rightarrow 1$$

$$\mathcal{L}(\hat{y}, y) = -\log \hat{y}, \quad \rightarrow 0$$

if $y = 0$,

$$\therefore z = w^T \cdot x + b, \quad \rightarrow -\infty$$

$$\hat{y} = \frac{1}{1 + e^{-z}}, \quad \rightarrow 0$$

$$\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y}), \quad \rightarrow 0$$

Cost Function

Error of training data

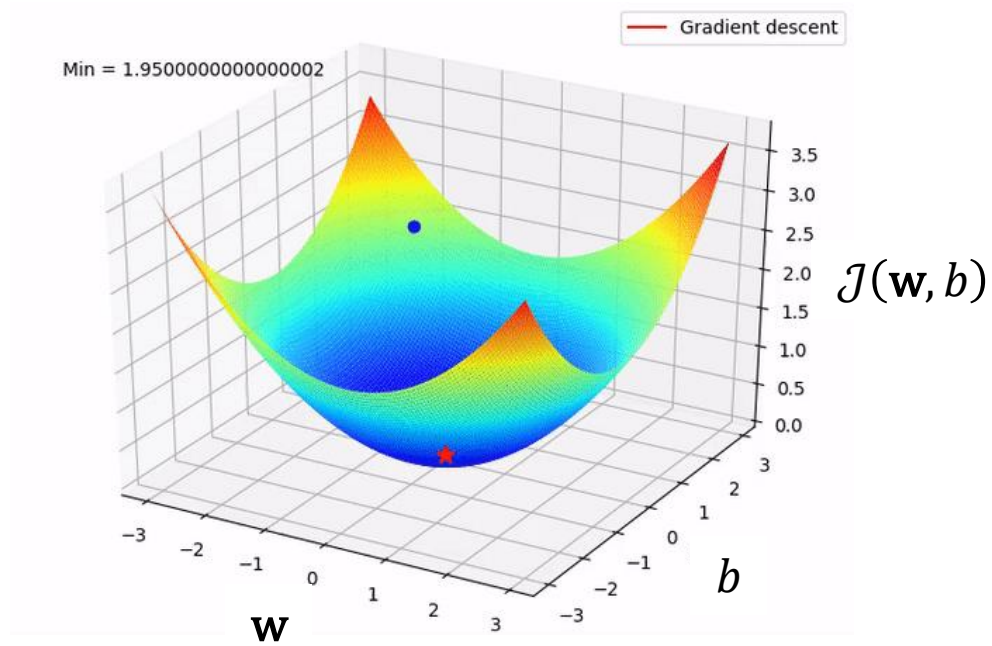
The cost function $J(\mathbf{w}, b)$ indicates how well the model does in entire training samples

$$\begin{aligned} J(\mathbf{w}, b) &= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \end{aligned}$$

- Training Goal
 - Find $\mathbf{w}, b$ that minimize $J(\mathbf{w}, b)$

Gradient Descent

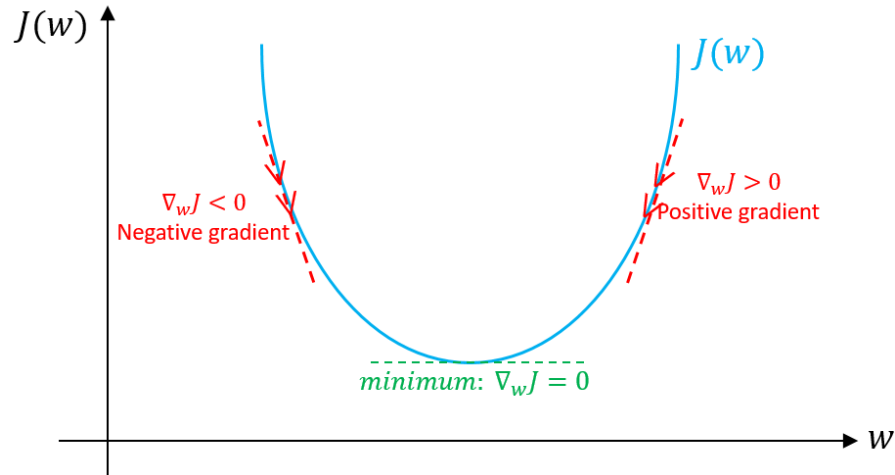
Searching for minimum point in hyperplane



- Goal
 - Determine $\mathbf{w}, b$ that minimize $J(\mathbf{w}, b)$

Gradient Descent

Initialize weight and adjust until the cost function approach to minimum



- Procedure

Repeat {

$$w := w - \alpha \frac{d}{dw} J(w)$$

Until $J(w) \rightarrow \min(J(w))$
}

where α represents the learning rate (a small positive value)

On the right

$$\frac{d}{dw} J(w) > 0, \quad \rightarrow w \text{ is adjusted by decreasing } \frac{d}{dw} J(w)$$

On the left

$$\frac{d}{dw} J(w) < 0, \quad \rightarrow w \text{ is adjusted by increasing } \frac{d}{dw} J(w)$$

Gradient Descent

Summary

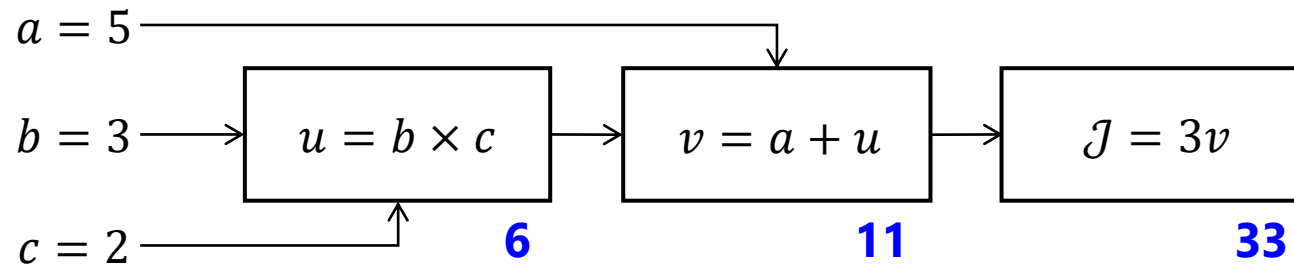
- Minimizing $J(\mathbf{w}, b)$
 - $w := w - \alpha \frac{d}{dw} J(w, b), \quad \text{where } w \in \mathbf{w}$
 - $b := b - \alpha \frac{d}{db} J(w, b)$
- In each training iteration, we need to determine

$$\frac{d}{dw} J(w, b) \quad \text{and} \quad \frac{d}{db} J(w, b)$$

Computational Graph

Forward path

Given a function $J(a, b, c) = 3(a + bc)$

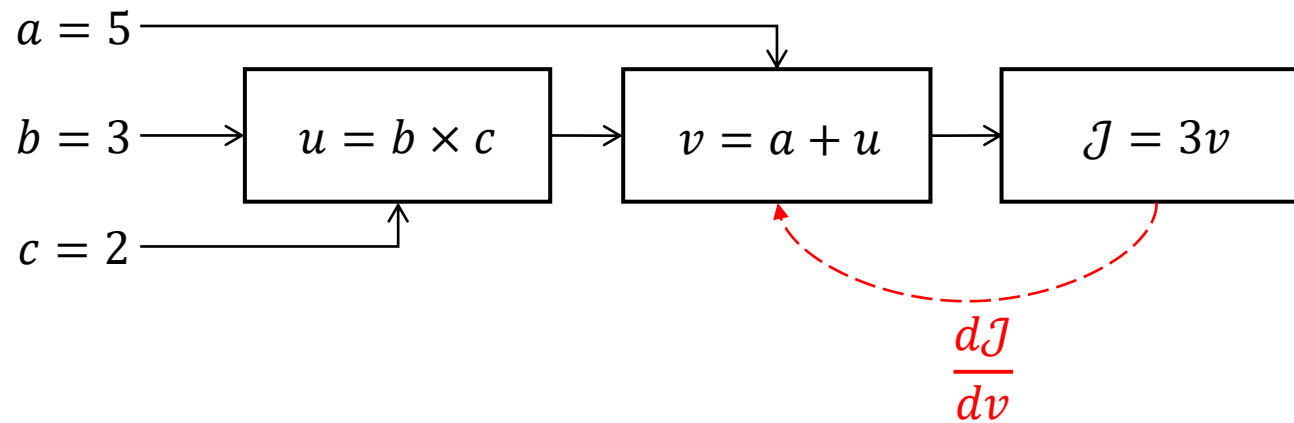


- Forward path
 - $J(a, b, c)$ can be determined
- Backward path
 - Derivative of $J(a, b, c)$ w.r.t. a or b or c can be determined

Computational Graph

Backward path

Given a function $J(a, b, c) = 3(a + bc)$



- Finding $\frac{dJ}{dv}$

- $v = 11$ $\xrightarrow{\text{Nudge its value}}$ $v = 11.001$

- $J = 33$ $\xrightarrow{\text{Changed by } v}$ $J = 33.003$

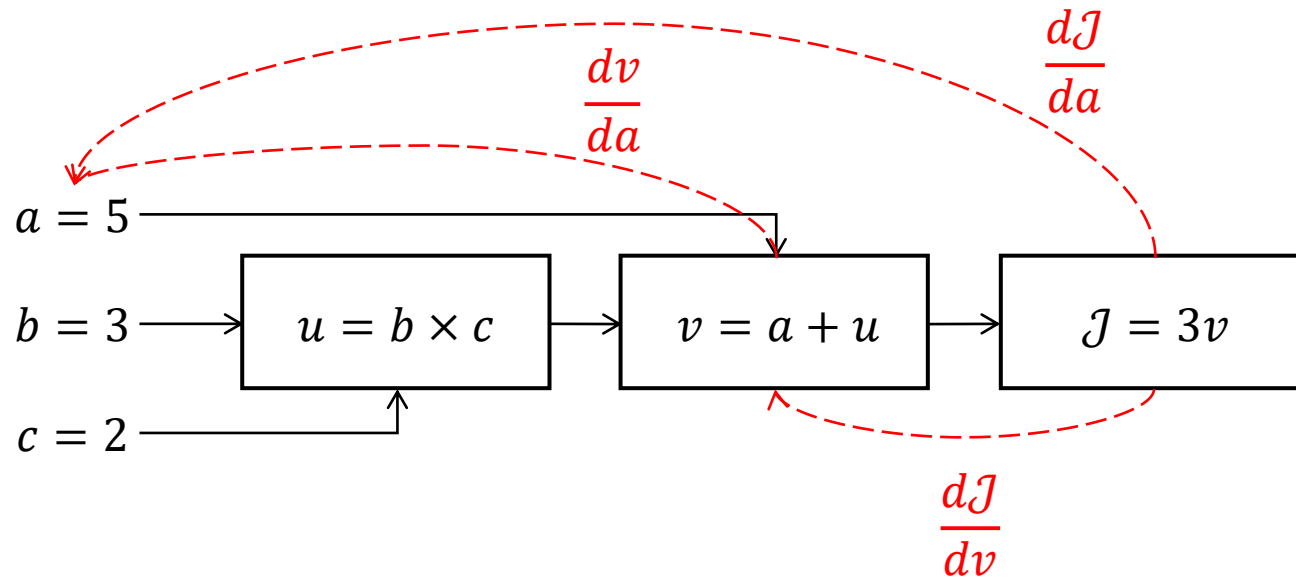
Therefore

$$\frac{dJ}{dv} = \frac{0.003}{0.001} = 3$$

Computational Graph

Backward path

Given a function $J(a, b, c) = 3(a + bc)$



• Finding $\frac{dJ}{da}$

• $a = 5$ $\xrightarrow{\text{Nudge its value}}$ $a = 5.001$

• $v = 11$ $\xrightarrow{\text{Changed by } a}$ $v = 11.001$

• $J = 33$ $\xrightarrow{\text{Changed by } a, v}$ $J = 33.003$

Therefore

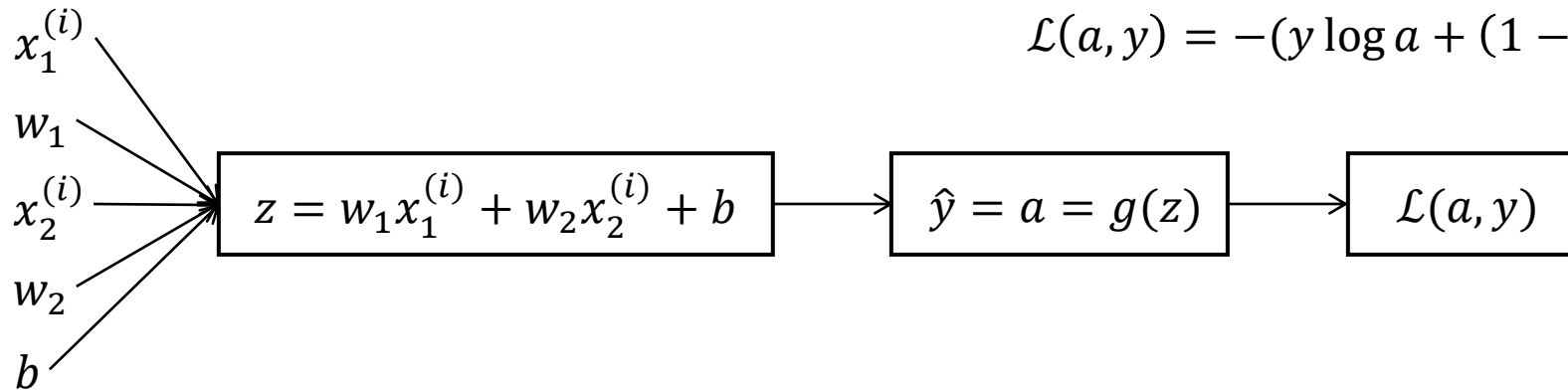
$$\frac{dJ}{da} = \frac{dJ}{dv} \cdot \frac{dv}{da} = \frac{0.003}{0.001} \cdot \frac{0.001}{0.001} = 3$$

Chain rule

Perceptron with Gradient Descent

For a training sample

Given $\mathbf{w} = [w_1 \ w_2]$ and b



- Recap

$$z = \mathbf{w}^T \cdot \mathbf{x}^{(i)} + b$$

$$\hat{y} = a = g(z)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

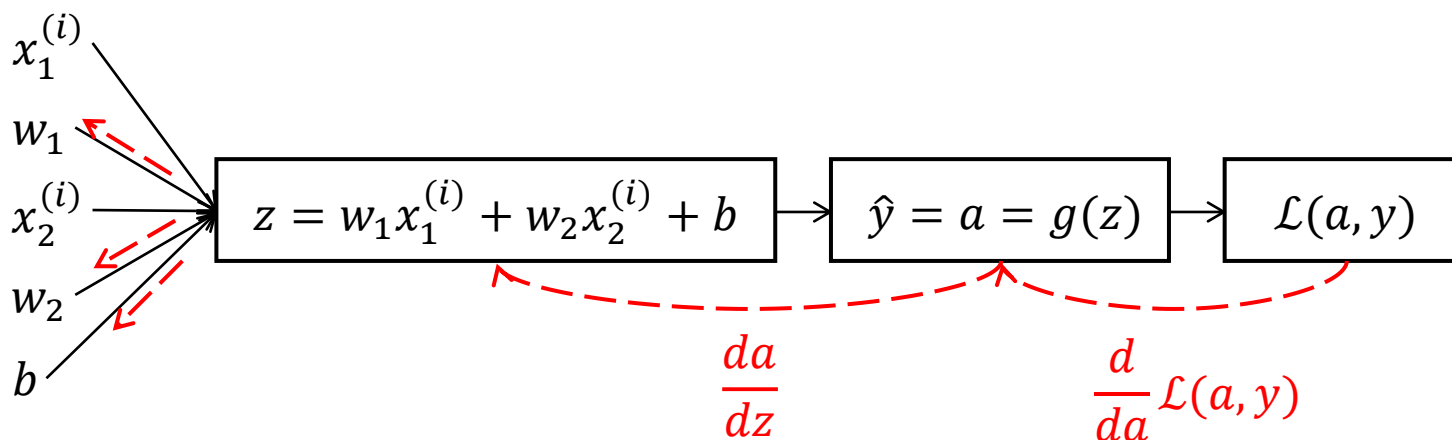
$$\mathcal{L}(a, y) = -(y \log a + (1 - y) \log(1 - a))$$

- Goal

- To adjust $\mathbf{w} = [w_1 \ w_2]$ and b to minimize $\mathcal{L}(a, y)$

Perceptron with Gradient Descent

Backward Propagation



$$\begin{aligned}\Delta z &= \frac{d}{dz} \mathcal{L}(a, y) \\ &= \frac{d}{da} \mathcal{L}(a, y) \cdot \frac{da}{dz}\end{aligned}$$

$$\text{since } \frac{d}{da} \mathcal{L}(a, y) = \frac{d}{da} [-y \log a - (1 - y) \log(1 - a)]$$

$$= -\frac{y}{a} + \frac{1 - y}{1 - a}$$

$$\text{and } \frac{da}{dz} = a(1 - a)$$

$$\begin{aligned}\therefore \Delta z &= \left(-\frac{y}{a} + \frac{1 - y}{1 - a} \right) \cdot a(1 - a) \\ &= a - y\end{aligned}$$

In other words, Δz is the difference between obtained output and desired output

Implementation

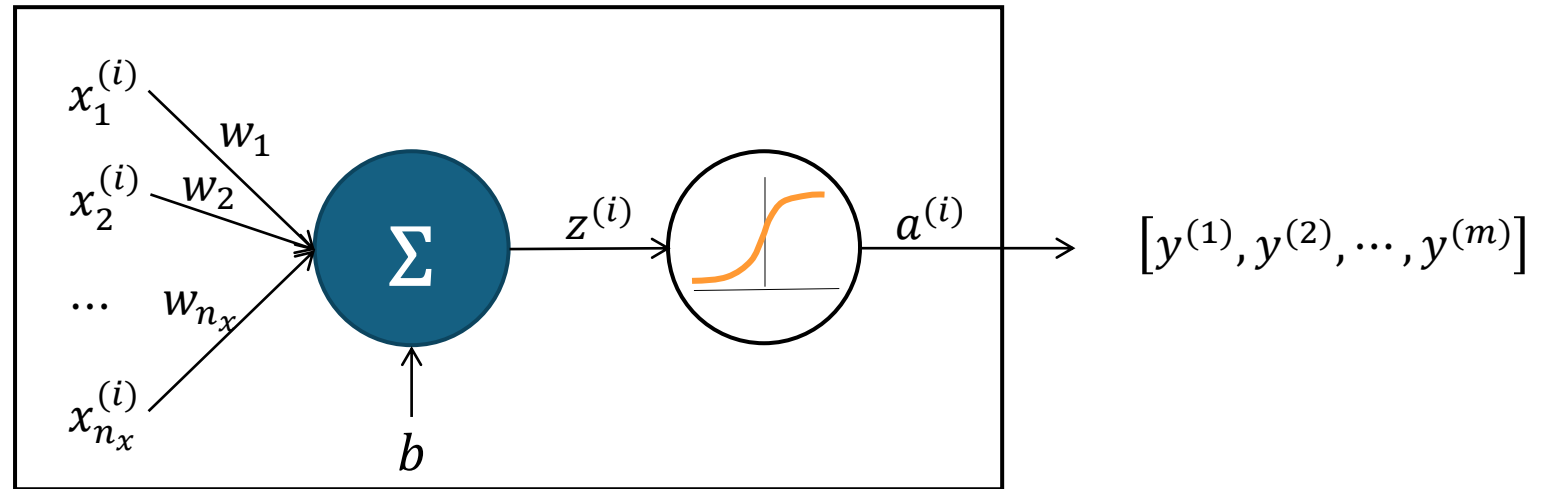
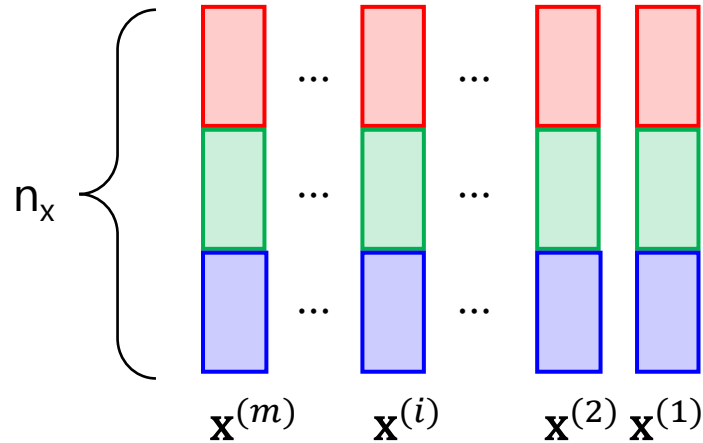


Vectorization

Applying a perceptron to m samples

Broadcasting property enable implementation without looping over training data

$$\mathbf{y} = g(\mathbf{z}) = g(\mathbf{w}^T \cdot \mathbf{X} + b)$$



Initialize

$$\mathcal{J} = 0, dw_1 = 0, dw_2 = 0, db = 0$$

Naïve Implementation

Loop and Loop and Loop

This is NOT an efficient method

Note: Python programming does not have the character `Δ`. For simplicity, let's change

$$\Delta w_1 \rightarrow dw_1$$

$$\Delta w_2 \rightarrow dw_2$$

$$\Delta b \rightarrow db$$

for epoch = 1 to max_epoch:

for i = 1 to m:

compute $z^{(i)}, a^{(i)}$

$$\mathcal{J} += -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

compute $dz^{(i)}$

update db

for j = 1 to n_x :

update dw_j

$$\mathcal{J} /= \frac{\mathcal{J}}{m}$$

Implement with Vectorization

The efficient method

Initialize

$$J = 0, dw_1 = 0, dw_2 = 0, db = 0$$

for epoch = 1 to max_epoch:

$$\mathbf{Z} = \mathbf{w}^T \cdot \mathbf{X} + b$$

$$\mathbf{A} = g(\mathbf{Z})$$

$$J = -\frac{1}{m} \sum (\mathbf{y} \log \mathbf{A} - (1 - \mathbf{y}) \log(1 - \mathbf{A}))$$

$$\mathbf{dz} = \mathbf{A} - \mathbf{Y}$$

$$\mathbf{dw} = \frac{1}{m} \mathbf{X} \cdot \mathbf{dz}^T$$

$$db = \frac{1}{m} \cdot \text{np.sum}(\mathbf{dz})$$

$$\mathbf{w} -= \alpha \cdot \mathbf{dw}$$

$$b -= \alpha \cdot db$$

Implement with Vectorization

The efficient method

Only loop over training iteration

for epoch = 1 to max_epoch:

$$\mathbf{Z}_{1 \times m} = \underbrace{(\mathbf{w}_{1 \times n_x})^T}_{1 \times m} \cdot \mathbf{X}_{n_x \times m} + b$$

$$\mathbf{A}_{1 \times m} = g(\mathbf{Z}_{1 \times m})$$

$$\mathcal{J} = -\frac{1}{m} \sum_{i=1}^m \underbrace{(\mathbf{y}_{1 \times m} \log \mathbf{A}_{1 \times m})}_{1 \times m} - \underbrace{(1 - \mathbf{y}_{1 \times m}) \log(1 - \mathbf{A}_{1 \times m})}_{1 \times m}$$

$$\mathbf{dz}_{1 \times m} = \mathbf{A}_{1 \times m} - \mathbf{y}_{1 \times m} \quad 1 \times 1$$

$$\mathbf{dw}_{1 \times n_x} = \frac{1}{m} \mathbf{X}_{n_x \times m} \cdot (\mathbf{dz}_{1 \times m})^T$$

$$db = \frac{1}{m} \sum_{i=1}^m \mathbf{dz}_{1 \times m}$$

$$\mathbf{w}_{1 \times n_x} -= \alpha \cdot \mathbf{dw}_{1 \times n_x}$$

$$b -= \alpha \cdot db$$

Logit Model

40 Minutes

Perceptron and Logit Model



Perceptron are the generalized version of logit model. When the activation function is set to be the sigmoid function, it becomes the logit model.

Original Form

Linear Combination

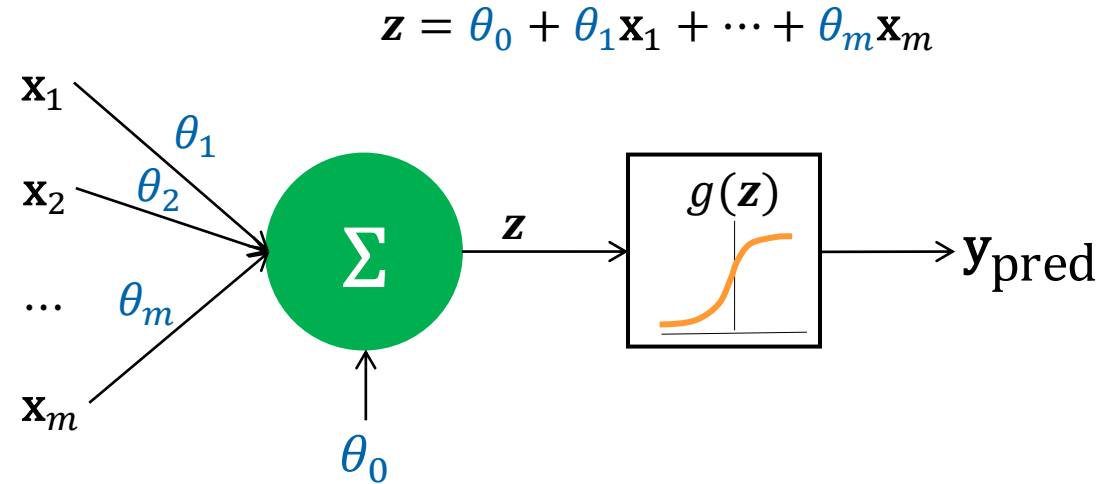
$$\theta_1 \mathbf{x}_1 + \dots + \theta_m \mathbf{x}_m$$

Interception

$$\theta_0$$

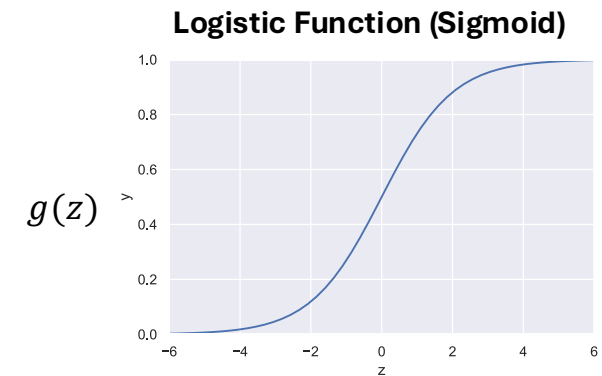
Parameters

$$\boldsymbol{\theta} = \langle \theta_0, \theta_1, \dots, \theta_m \rangle$$



$$P(y|\mathbf{X}; \boldsymbol{\theta}) = \frac{1}{1 + e^{-z}}$$

$$= \frac{1}{1 + e^{-\boldsymbol{\theta}^T \cdot \mathbf{X}}}$$



Broadcasting

Vector + Scalar

$$\mathbf{v} = [1 \quad 2 \quad 3]$$

$$\mathbf{v} + 3 = [1 \quad 2 \quad 3] + [3 \quad 3 \quad 3] \rightarrow \text{Value Replication}$$
$$= [4 \quad 5 \quad 6]$$

Broadcasting

Matrix + Scalar

$$\begin{aligned} \mathbf{v} &= \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \\ \mathbf{v} + 3 &= \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 3 & 3 & 3 \\ 3 & 3 & 3 \end{bmatrix} \xrightarrow{\text{Value Replication}} \\ &\quad \downarrow \text{Value Replication} \\ &= \begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \end{aligned}$$

Broadcasting

Matrix + Vector

$$\mathbf{v} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \mathbf{u} = [10 \quad 20 \quad 30]$$

$$\mathbf{v} + \mathbf{u} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

Value Replication

Broadcasting

Matrix + Vector

$$\mathbf{V} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \mathbf{u} = [10 \quad 20]$$

$$\mathbf{V} + \mathbf{u} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & ? \\ ? & ? & ? \end{bmatrix} \rightarrow \text{Cannot replicate values in this direction (dimension disagreed)}$$

$= \text{Error}$

Matrix Representation for Equations

$$2X_1 - 4X_2 + 6X_3 = 10$$

$$-X_1 + 2X_2 + 4X_3 = 12$$

$$5X_1 + 7X_2 - 2X_3 = 15$$



$$\begin{array}{ccc} \boldsymbol{\theta} & \boldsymbol{X} & \boldsymbol{Z} \\ \begin{bmatrix} 2 & -4 & +6 \\ -1 & 2 & 4 \\ 5 & 7 & -2 \end{bmatrix} & \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} & = \begin{bmatrix} 10 \\ 12 \\ 15 \end{bmatrix} \end{array}$$

$$\boldsymbol{\theta}^T \cdot \boldsymbol{X} = \boldsymbol{Z}$$

Demonstration

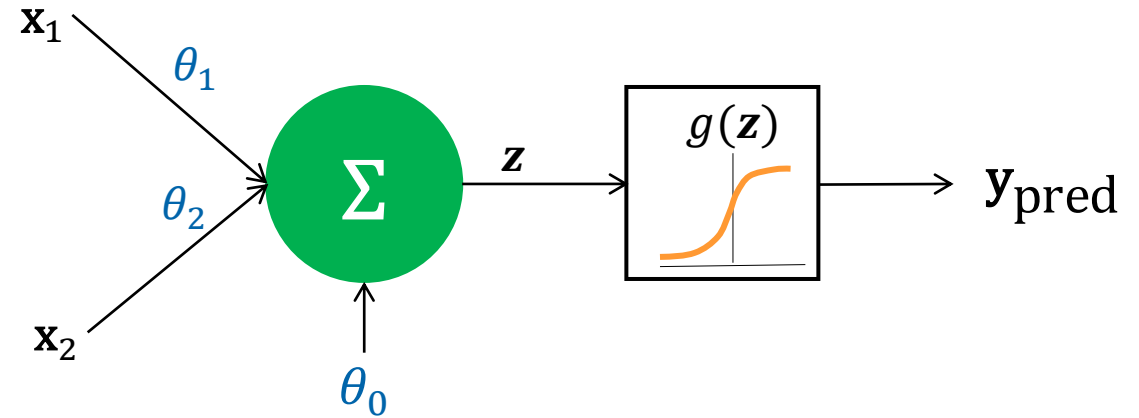
Training Data

x_1	x_2	y
3	1	0
5	0	1
...	...	...
2	1	0

This is the target vector y that represent lung cancer (1 for yes, 0 for no)

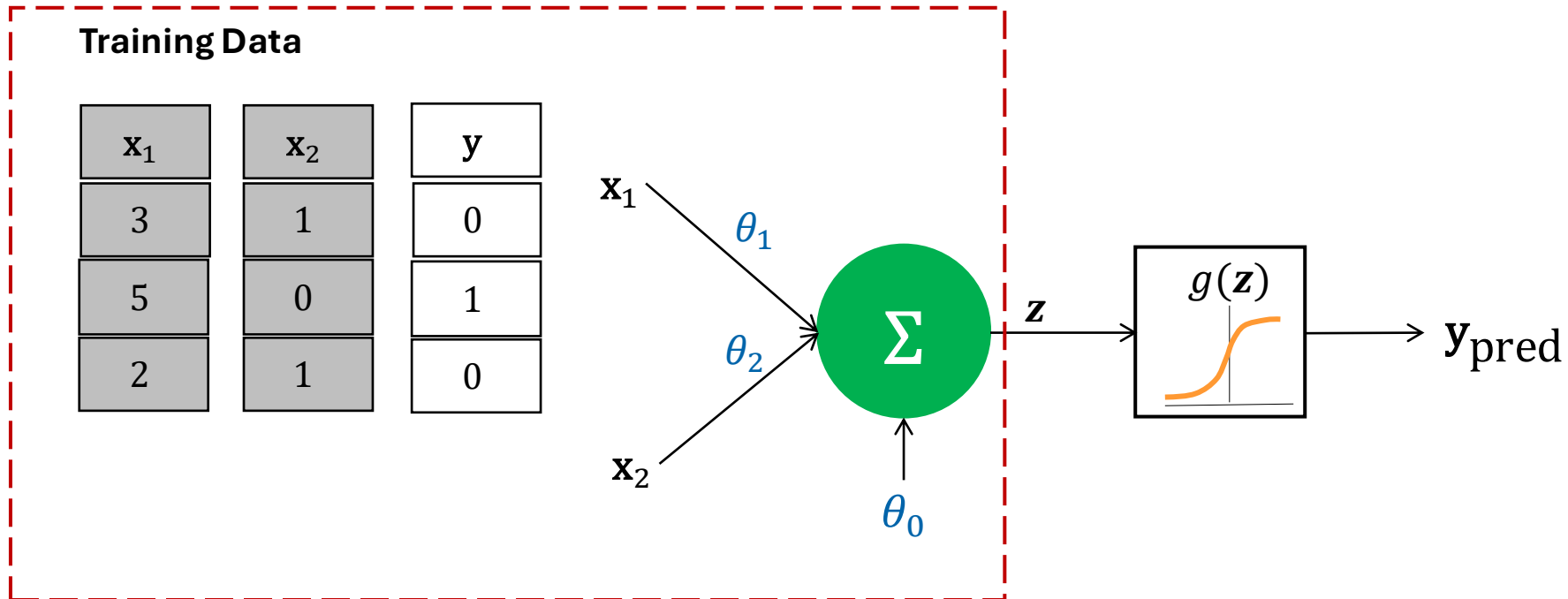
This is a vector to represent feature x_2 . In this example. It is chest pain (1 for yes, 0 for no)

This is a vector to represent feature x_1 . In this example, it is smoking frequency per week



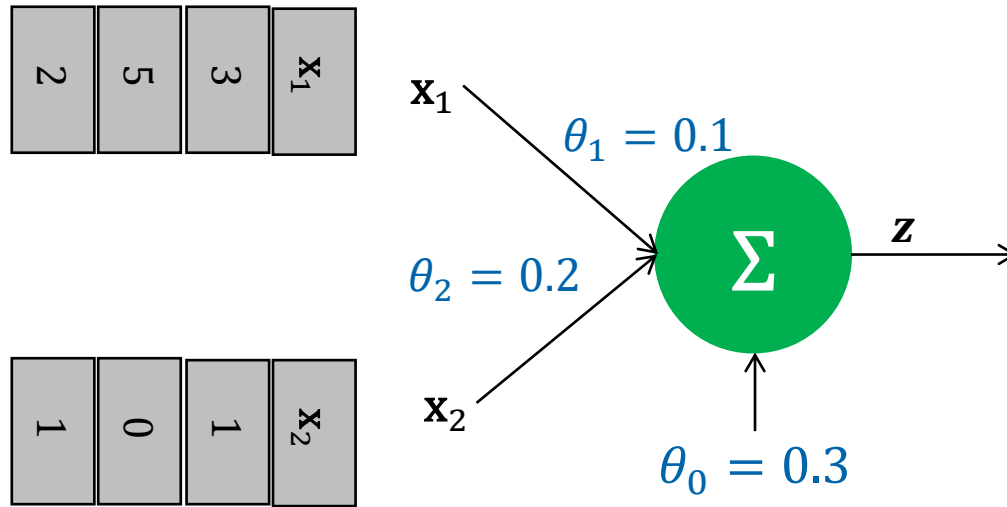
Demonstration

Assume that the number of sample n is 3 (for simplicity of explanation)

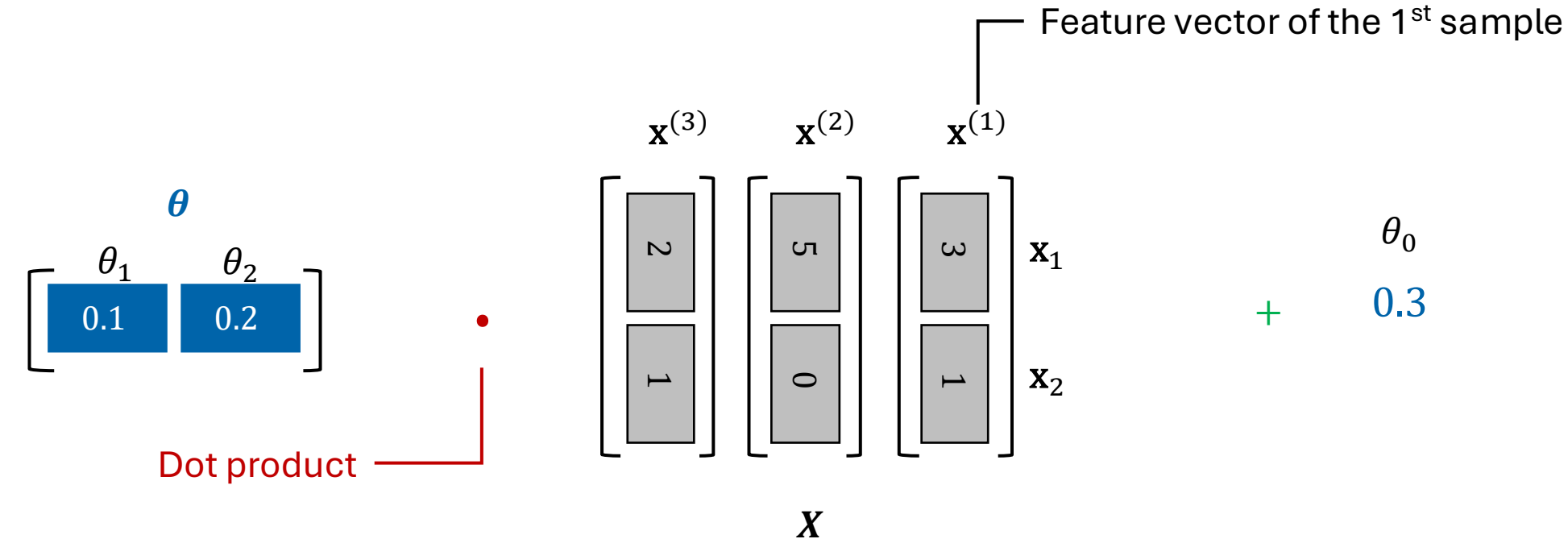
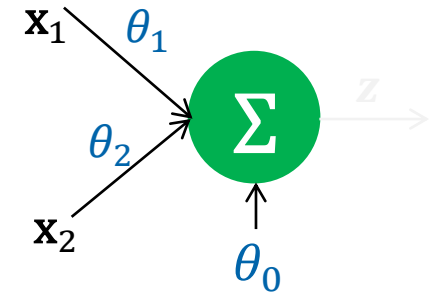


We will explain what will be happened here first

Demonstration



Demonstration



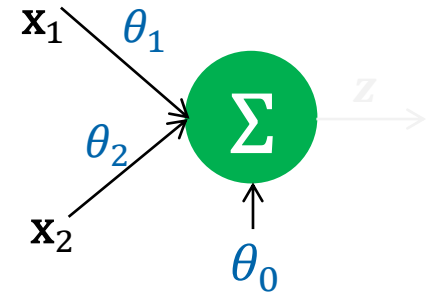
$\theta \cdot \mathbf{X}$ is invalid due to dimension disagreement for broadcasting. We need to transpose θ

Note: For convenience, from slide#11 to slide#19, θ refers to θ_1 and θ_2 only

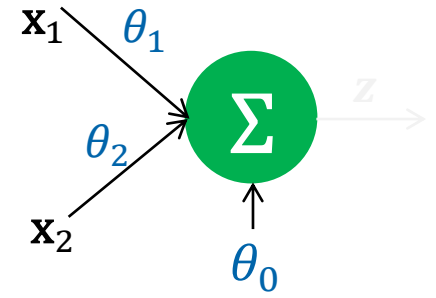
Demonstration

$$\begin{array}{c} \theta_1 \\ \theta_2 \end{array} \begin{array}{c} \theta^T \\ \left[\begin{array}{c} 0.1 \\ 0.2 \end{array} \right] \end{array} \cdot \begin{array}{ccc} \mathbf{x}^{(3)} & \mathbf{x}^{(2)} & \mathbf{x}^{(1)} \\ \left[\begin{array}{c} 2 \\ 1 \end{array} \right] & \left[\begin{array}{c} 5 \\ 0 \end{array} \right] & \left[\begin{array}{c} 3 \\ 1 \end{array} \right] \\ \mathbf{x}_1 & & \mathbf{x}_2 \end{array} \quad + \quad \begin{array}{c} \theta_0 \\ 0.3 \end{array}$$

X



Demonstration



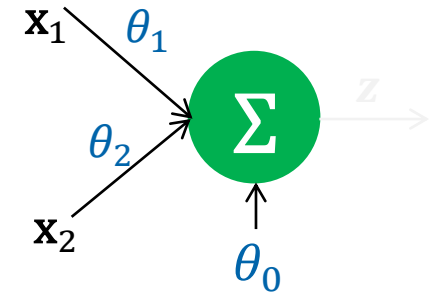
$$\theta^T = \begin{bmatrix} \theta_1 & \theta_2 \end{bmatrix} \begin{bmatrix} 0.1 & 0.1 & 0.1 \\ 0.2 & 0.2 & 0.2 \end{bmatrix}$$

Broadcasting

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(3)} & \mathbf{x}^{(2)} & \mathbf{x}^{(1)} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} & \begin{bmatrix} 5 \\ 0 \end{bmatrix} & \begin{bmatrix} 3 \\ 1 \end{bmatrix} \end{bmatrix} \begin{matrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{matrix}$$

$$+ \theta_0 = 0.3$$

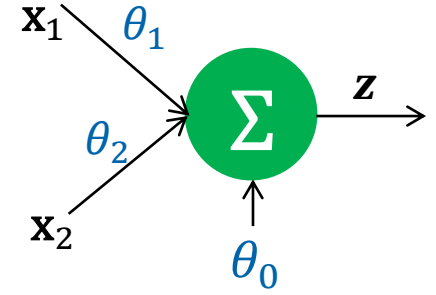
Demonstration



$$\begin{array}{c}
 \mathbf{x}^{(3)} \qquad \qquad \mathbf{x}^{(2)} \qquad \qquad \mathbf{x}^{(1)} \\
 \left[\begin{array}{c} \left(\begin{array}{c} \boxed{0.1} \times \boxed{2} \end{array} \right) \\ + \\ \left(\begin{array}{c} \boxed{0.2} \times \boxed{1} \end{array} \right) \end{array} \right] \quad \left[\begin{array}{c} \left(\begin{array}{c} \boxed{0.1} \times \boxed{5} \end{array} \right) \\ + \\ \left(\begin{array}{c} \boxed{0.2} \times \boxed{0} \end{array} \right) \end{array} \right] \quad \left[\begin{array}{c} \left(\begin{array}{c} \boxed{0.1} \times \boxed{3} \end{array} \right) \\ + \\ \left(\begin{array}{c} \boxed{0.2} \times \boxed{1} \end{array} \right) \end{array} \right] \begin{array}{l} \mathbf{x}_1 \\ \mathbf{x}_2 \end{array} \\
 0.4 \qquad \qquad 0.5 \qquad \qquad 0.5 \\
 \theta^T \cdot X
 \end{array}$$

$$\begin{array}{c}
 + \quad \theta_0 \\
 \quad 0.3
 \end{array}$$

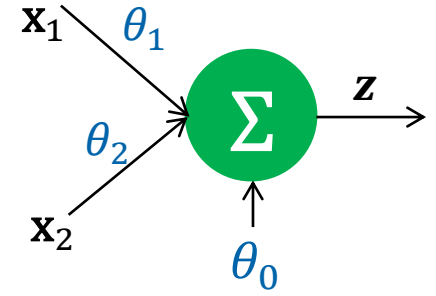
Demonstration



$$\begin{bmatrix} 0.4 & 0.5 & 0.5 \end{bmatrix} + \begin{matrix} \theta_0 \\ 0.3 \end{matrix}$$

$\theta^T \cdot X$

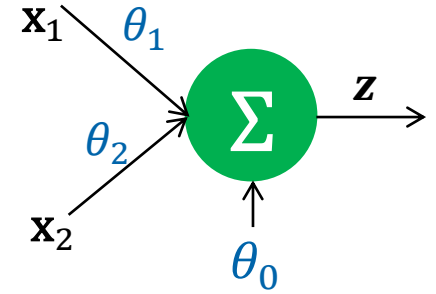
Demonstration



$$\begin{bmatrix} 0.4 & 0.5 & 0.5 \end{bmatrix} + \begin{matrix} \theta_0 \\ 0.3 & 0.3 & 0.3 \end{matrix} \rightarrow \text{Broadcasting}$$

$\theta^T \cdot X$

Demonstration



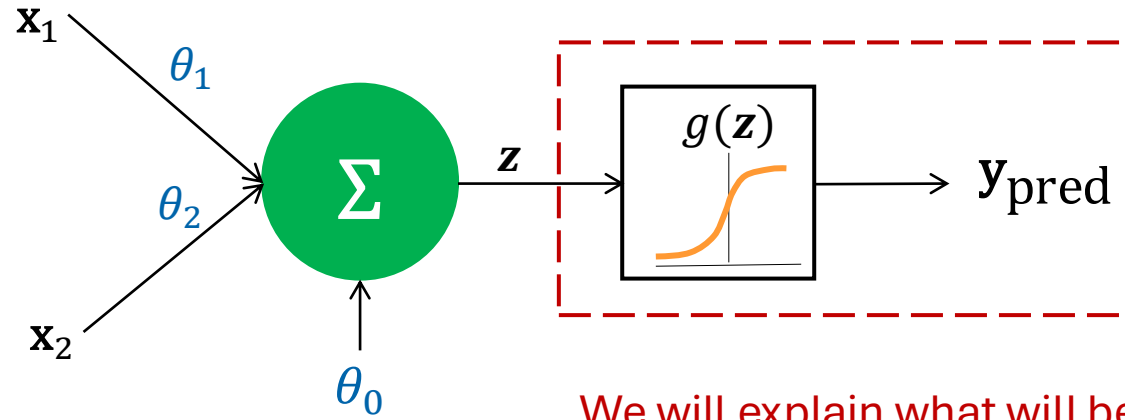
$$\theta^T \cdot X \quad \theta_0 \quad \begin{bmatrix} 0.4 & 0.5 & 0.5 \\ + & + & + \\ 0.3 & 0.3 & 0.3 \end{bmatrix} = \begin{bmatrix} 0.7 & 0.8 & 0.8 \end{bmatrix}$$

$$z = \theta_0 + \theta^T \cdot X$$

Demonstration

Training Data

x_1	x_2	y
3	1	0
5	0	1
2	1	0



We will explain what will be happened here

Demonstration

$$z_3 = \theta_0 + \boldsymbol{\theta}^T \cdot \mathbf{x}^{(3)}$$

$$\left[\begin{array}{c} 0.7 \end{array} \right]$$

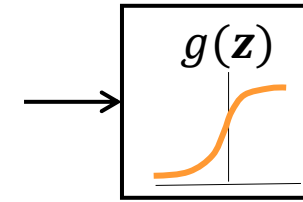
$$z_2 = \theta_0 + \boldsymbol{\theta}^T \cdot \mathbf{x}^{(2)}$$

$$0.8$$

$$z_1 = \theta_0 + \boldsymbol{\theta}^T \cdot \mathbf{x}^{(1)}$$

$$0.8$$

$$\mathbf{z} = \theta_0 + \boldsymbol{\theta}^T \cdot \mathbf{X}$$

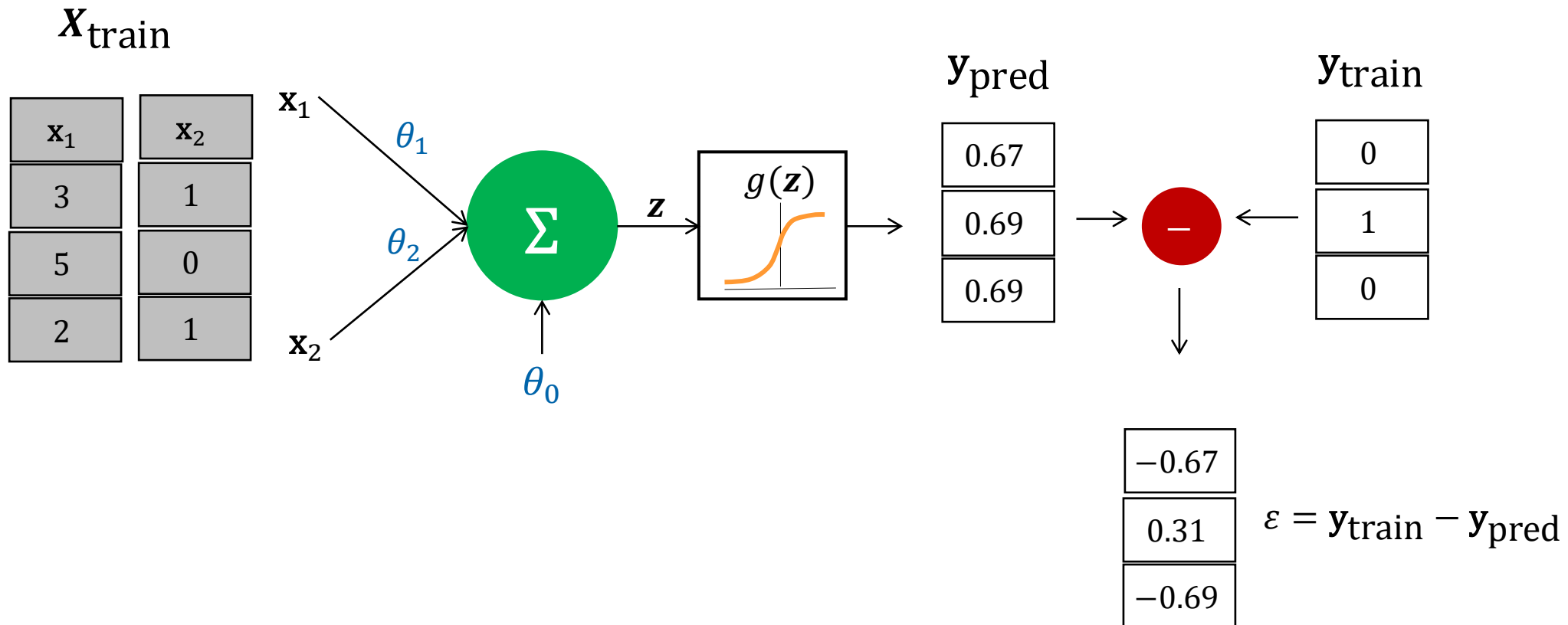


$$g(z) = \frac{1}{1 + e^{-z}}$$

$\mathbf{y}_{\text{pred}}$

$$\left[\begin{array}{c} \frac{1}{1 + e^{-0.7}} \\ \frac{1}{1 + e^{-0.8}} \\ \frac{1}{1 + e^{-0.8}} \end{array} \right]$$

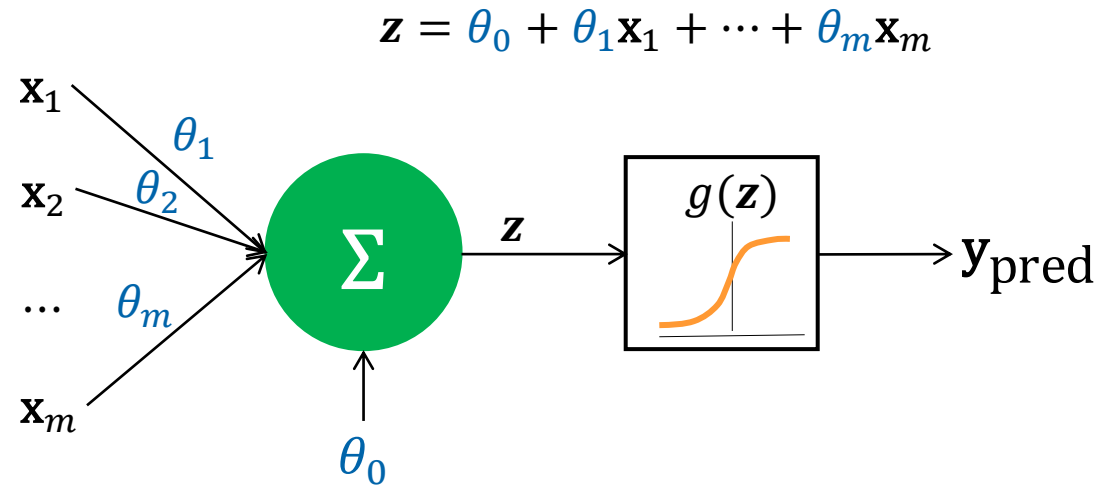
Demonstration



Original Form (Recap)

Training Data

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{y}$
3	1	0
5	0	1
2	1	0



$$P(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta}) = \frac{1}{1 + e^{-z}}$$

$$= \frac{1}{1 + e^{-\boldsymbol{\theta}^T \cdot \mathbf{X}}}$$

Logit Transformation

The original form of logistic regression

$$P(y|\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}} \quad (1)$$

$$1 - P(y|\mathbf{x}; \boldsymbol{\theta}) = 1 - \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}}$$

$$\therefore 1 - P(y|\mathbf{x}; \boldsymbol{\theta}) = \frac{e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}} \quad (2)$$

$$\begin{aligned} \therefore 1 - \frac{1}{1 + e^{-a}} &= \frac{1 + e^{-a}}{1 + e^{-a}} - \frac{1}{1 + e^{-a}} \\ &= \frac{1 + e^{-a} - 1}{1 + e^{-a}} \\ &= \frac{e^{-a}}{1 + e^{-a}} \end{aligned}$$

Logit Transformation

Divide (1) by (2)

$$\frac{P(y|\mathbf{x}; \boldsymbol{\theta})}{1 - P(y|\mathbf{x}; \boldsymbol{\theta})} = \frac{1}{\frac{e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}}}$$

$$\begin{aligned}\text{Odds}(y|\mathbf{x}; \boldsymbol{\theta}) &= \frac{1}{e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}} \\ &= e^{(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}\end{aligned}$$

$$\ln \text{Odds}(y|\mathbf{x}; \boldsymbol{\theta}) = \ln e^{(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}$$

$$\text{logit}P(y|\mathbf{x}; \boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m$$

Summary

- Keys to take away are
 - Logistic regression has 2 forms
 - Original form
 - Logit form
 - The logit form looks similar to linear regression, but the target is NOT y but log odds of y

Maximum Likelihood Estimation

30 Minutes

Background Knowledge



$$P(y; \theta) = \theta^y (1 - \theta)^{1-y}$$

where

$$y \in \{1, 0\}$$

If we want to classify 0 and 1 from a given probability, we need the cutoff value

Background Knowledge

Derivative of Sigmoid function

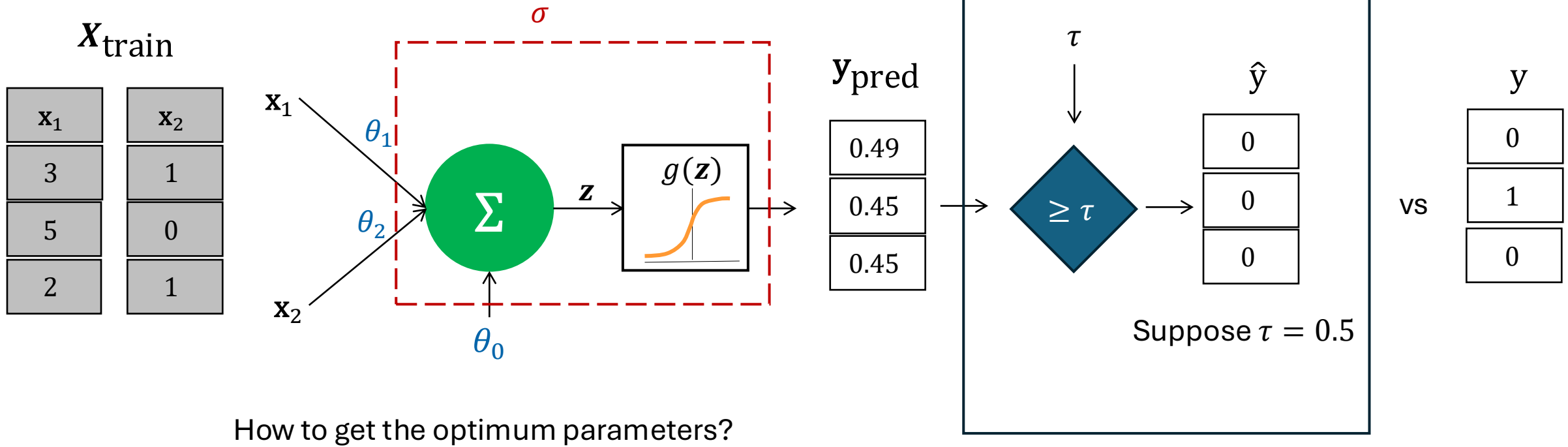
$$g(z) = \frac{1}{1 + e^{-z}}$$

$$\begin{aligned} \frac{d}{dz} g(z) &= \frac{d}{dz} \left[\frac{1}{1 + e^{-z}} \right] \\ &= \frac{d}{dz} (1 + e^{-z})^{-1} \\ &= -(1 + e^{-z})^{-2} \frac{d}{dz} (1 + e^{-z}) \\ &= -(1 + e^{-z})^{-2} \frac{d}{dz} (e^{-z}) \end{aligned}$$

$$\begin{aligned} &= -(1 + e^{-z})^{-2} (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\ &= \frac{1}{1 + e^{-z}} \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \\ &= \frac{1}{1 + e^{-z}} \left[\frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}} \right] \\ &= \frac{1}{1 + e^{-z}} \left[1 - \frac{1}{1 + e^{-z}} \right] \end{aligned}$$

$$\therefore \frac{d}{dz} g(z) = g(z)(1 - g(z))$$

Basic Idea



Maximum Likelihood Estimation

Given a dataset $\mathcal{D} = \langle X, y \rangle$

$\mathbf{x}_1$	$\mathbf{x}_2$	y
3	1	0
5	0	1
...		
2	1	0

What are the most likely model parameters obtained from this dataset?

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = P(\mathbf{y} | \mathbf{X}; \boldsymbol{\theta})$$

Assume that each sample is independent, the dataset $\mathcal{D}$ is the **joint probability of all samples** and $\mathbf{y} \sim \text{Ber}(\sigma(\mathbf{x}))$

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \prod_{i=1}^n P(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta})$$

Maximum Likelihood Estimation

From $\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \prod_{i=1}^n P(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta})$

$$= \prod_{i=1}^n \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})^{y^{(i)}} \cdot (1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))^{1-y^{(i)}} \quad \because P(y; \theta) = \theta^y (1 - \theta)^{1-y}$$

$\mathbf{x}_1$	$\mathbf{x}_2$	y
3	1	0
5	0	1
...		
2	1	0

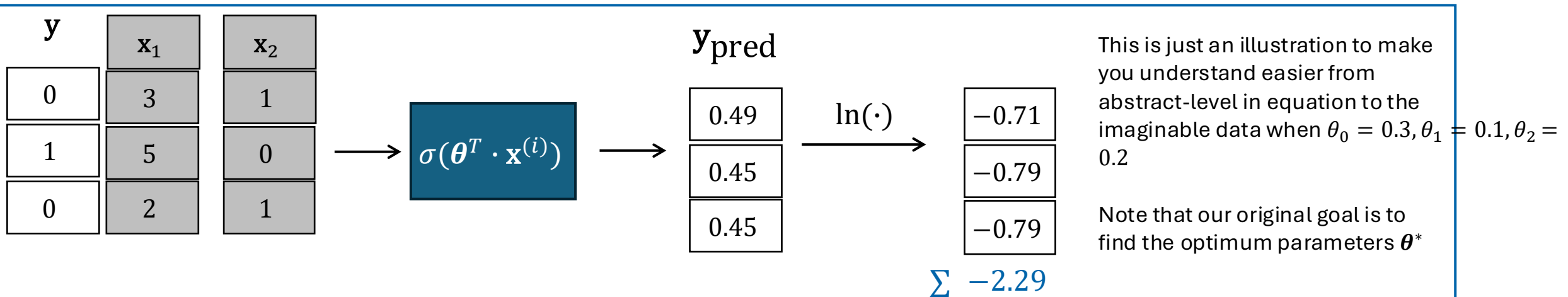
$$\begin{aligned} \therefore \ln \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) &= \ln \prod_{i=1}^n \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})^{y^{(i)}} \cdot (1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))^{1-y^{(i)}} \\ &= \sum_{i=1}^n \ln \left[\sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})^{y^{(i)}} \cdot (1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))^{1-y^{(i)}} \right] \\ &= \sum_{i=1}^n \left[\ln \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})^{y^{(i)}} + \ln (1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))^{1-y^{(i)}} \right] \\ &= \sum_{i=1}^n \left[y^{(i)} \ln \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}) + (1 - y^{(i)}) \ln (1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})) \right] \end{aligned}$$

Maximum Likelihood Estimation

From
$$\ln \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \sum_{i=1}^n [y^{(i)} \ln \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}) + (1 - y^{(i)}) \ln(1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))]$$

If $y^{(i)} = 1$, then
$$\ln \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \sum_{i=1}^n [y^{(i)} \ln \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})]$$

If $y^{(i)} = 0$, then
$$\ln \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \sum_{i=1}^n [(1 - y^{(i)}) \ln(1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))]$$



Maximum Likelihood Estimation

To determine the optimum parameter $\boldsymbol{\theta}^*$, we will take partial derivative with respect to each θ_j .

For any sample $(\mathbf{x}, y)$

$$\frac{\partial}{\partial \theta_j} \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)}) = \frac{\partial}{\partial \theta_j} [y^{(i)} \ln \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}) + (1 - y^{(i)}) \ln(1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))]$$

Applying derivative chain rule

$$\frac{\partial}{\partial \theta_j} \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)}) = \frac{\partial \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)})}{\partial \pi^{(i)}} \cdot \frac{\partial \pi^{(i)}}{\partial \theta_j} \quad \text{where } \pi^{(i)} = \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})$$

$$= \frac{\partial \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)})}{\partial \pi^{(i)}} \cdot \frac{\partial \pi^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \theta_j} \quad \text{where } z^{(i)} = \boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}$$

Maximum Likelihood Estimation

From $\ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)}) = y^{(i)} \ln \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}) + (1 - y^{(i)}) \ln(1 - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}))$

Substitute $\pi^{(i)} = \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})$ and $z^{(i)} = \boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}$

$$\ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)}) = y^{(i)} \ln \pi^{(i)} + (1 - y^{(i)}) \ln(1 - \pi^{(i)})$$

$$\frac{\partial \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)})}{\partial \pi} = \frac{y^{(i)}}{\pi^{(i)}} - \frac{1 - y^{(i)}}{1 - \pi^{(i)}} \quad \because \frac{d}{dx} \ln x = \frac{1}{x}$$

$$\frac{\partial \pi^{(i)}}{\partial z^{(i)}} = \sigma(z^{(i)})(1 - \sigma(z^{(i)})) \quad \because \pi^{(i)} = \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}) \text{ and } z^{(i)} = \boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}$$

$$\frac{\partial z^{(i)}}{\partial \theta_j} = x_j \quad \because \text{Only } x_j \text{ interacts with } \theta_j$$

Maximum Likelihood Estimation

$$\begin{aligned}
 \therefore \frac{\partial}{\partial \theta_j} \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)}) &= \frac{\partial \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)})}{\partial \pi^{(i)}} \cdot \frac{\partial \pi^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \theta_j} \\
 &= \left(\frac{y^{(i)}}{\pi^{(i)}} - \frac{1 - y^{(i)}}{1 - \pi^{(i)}} \right) \cdot \sigma(z^{(i)})(1 - \sigma(z^{(i)})) \cdot x_j \\
 &= \left(\frac{y^{(i)}}{\pi^{(i)}} - \frac{1 - y^{(i)}}{1 - \pi^{(i)}} \right) \cdot \pi^{(i)}(1 - \pi^{(i)}) \cdot x_j && \because \pi^{(i)} = \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)}) = \sigma(z^{(i)}) \\
 &= [y^{(i)}(1 - \pi^{(i)}) - \pi^{(i)}(1 - y^{(i)})] \cdot x_j \\
 &= [y^{(i)} - \pi^{(i)}] \cdot x_j \\
 &= [y^{(i)} - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})] \cdot x_j
 \end{aligned}$$

Maximum Likelihood Estimation

$$\frac{\partial}{\partial \theta_j} \ln \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}^{(i)}) = [y^{(i)} - \sigma(\boldsymbol{\theta}^T \cdot \mathbf{x}^{(i)})] \cdot x_j$$

y	$\mathbf{x}_1$	$\mathbf{x}_2$
0	3	1
1	5	0
0	2	1

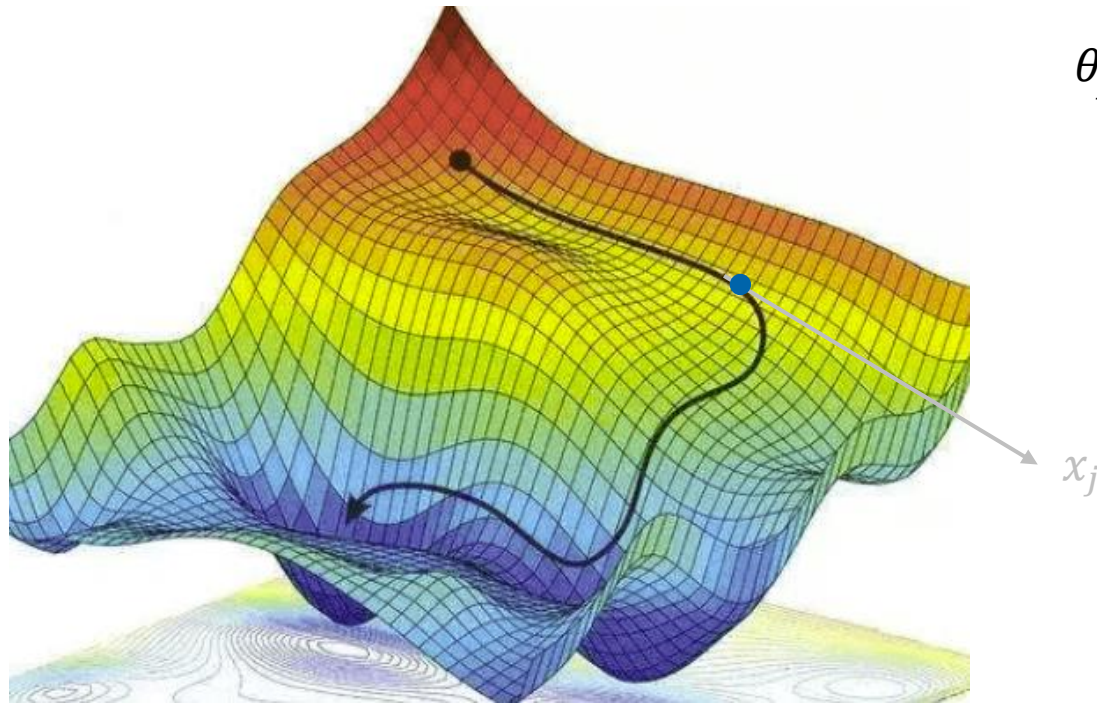
Sample log-likelihood for the feature j

Sample log-likelihood for the feature j

Sample log-likelihood for the feature j

Maximum Likelihood Estimation

Parameter adjustment over learning iteration



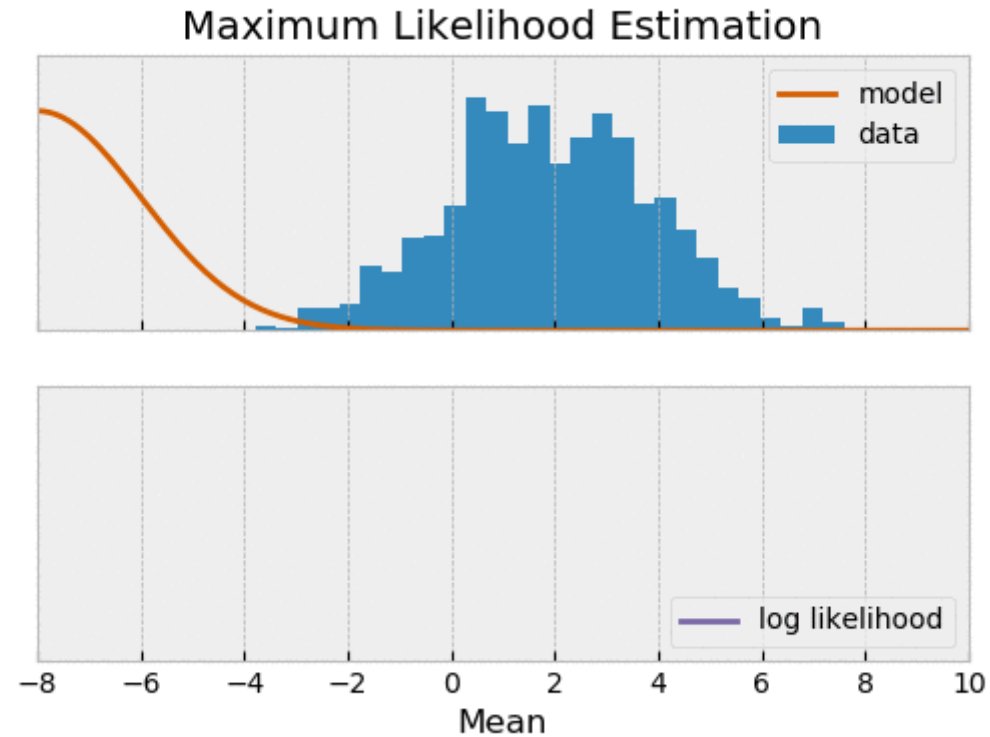
$$\theta_j^{new} = \theta_j^{old} + \alpha \cdot \frac{\partial}{\partial \theta_j^{old}} \ln \mathcal{L}(\boldsymbol{\theta}^{old}; \mathbf{x}^{(i)})$$

$$= \theta_j^{old} + \alpha \cdot \sum_{i=1}^n [y^{(i)} - \sigma(\boldsymbol{\theta}_{old}^T \cdot \mathbf{x}^{(i)})] \cdot x_j$$

where

α represents the learning rate (a small positive real number)

Maximum Likelihood Estimation



Summary

- Keys to take away
 - Maximum Likelihood Estimation (MLE) for logistic regression assumes
 - Target y to follow Bernoulli distribution
 - Parameter adjustment for the feature j is obtained by adding the multiplication of the learning rate and gradient of data log-likelihood to the old parameter
 - The learning process run iteratively
 - The initial learning process randomly initializes model parameters θ
 - The model keep learning until it is converged indicated by no improvement of log-likelihood

Categorical Variable Handling

5 Minutes

One-hot Encoding

Y : Churn	X_1 : Monthly Usage (Hours)	X_2 : Subscription Type
Yes	12	Basic
No	50	Standard
Yes	8	Premium
No	30	Standard
Yes	5	Basic

One-hot Encoding

X_2 : Subscription Type
Basic
Standard
Premium
Standard
Basic

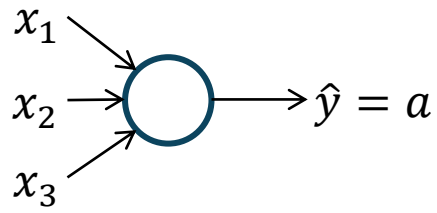
Basic	Standard	Premium
1	0	0
0	1	0
0	0	1
0	1	0
1	0	0

Neural Network

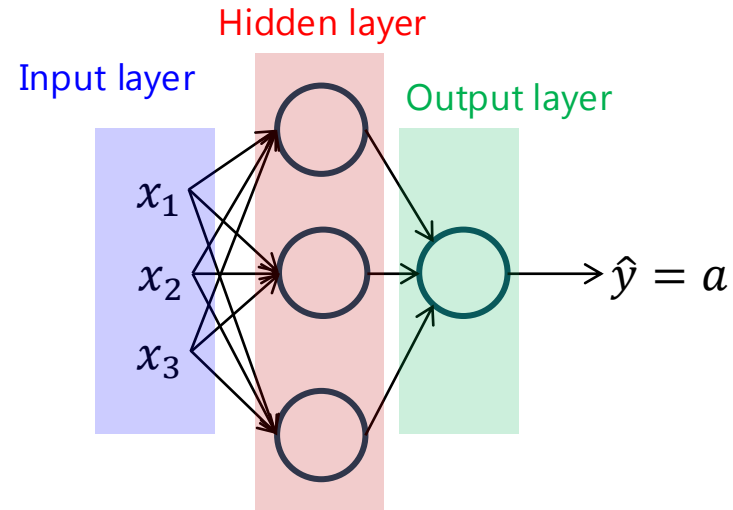
A collection of perceptrons

Shallow Neural Network

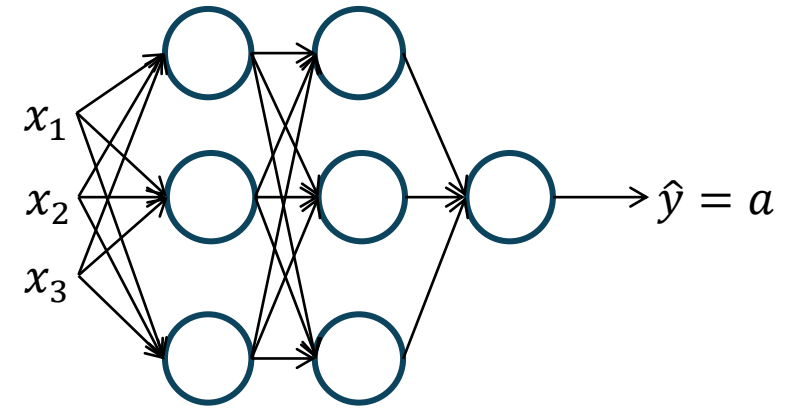
None or a few hidden layers



1-Layer Network
(0 Hidden Layer)
(perceptron or logistic regression)



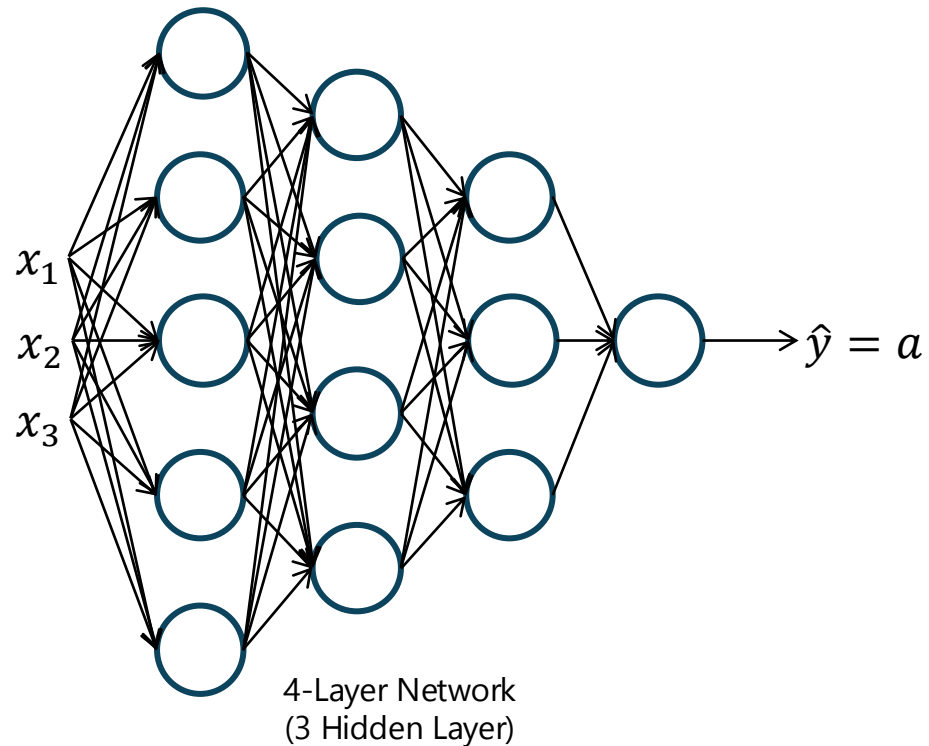
2-Layer Network
(1 Hidden Layer)



3-Layer Network
(2 Hidden Layer)

Deep Neural Network

More hidden layers



- Notations

L represents the number of layers (exclude the input layer)

$n^{[l]}$ represents the number of units in the l^{th} layer

Examples:

$$n^{[1]} = 5, \quad n^{[2]} = 4, \quad n^{[3]} = 3, \quad n^{[4]} = 1$$

$g^{[l]}(\cdot)$ represents the activation function in the l^{th} layer

$\mathbf{z}^{[l]}$ represents the linear combination in the l^{th} layer

$\mathbf{a}^{[l]}$ represents the activated values in the l^{th} layer

$\mathbf{W}^{[l]}$ represents the weights matrix in the l^{th} layer

$\mathbf{b}^{[l]}$ represents the bias in the l^{th} layer

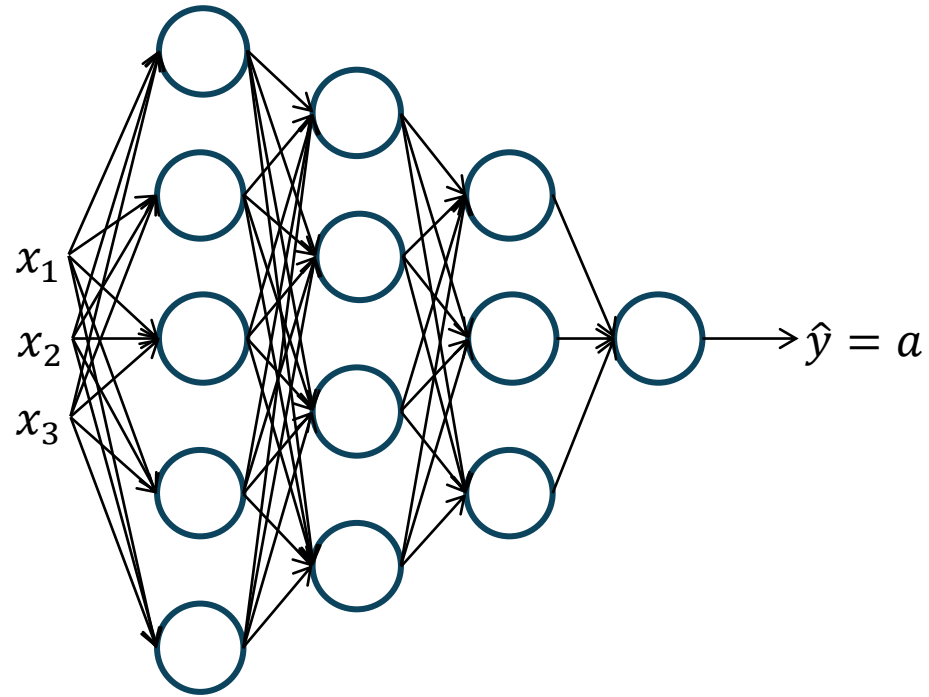
$\mathbb{W}$ represents the collection of weights matrix of all layers

$w_{i,j}$ represents the weights connecting between the i^{th} unit in the l^{th} layer to the j^{th} unit in the $(l-1)^{th}$ layer

$\mathbb{b}$ represents the collection of bias vectors of all layers

Deep Neural Network

Weights



- Weights $\mathbb{W} = \begin{bmatrix} \mathbf{W}^{[1]} \\ \mathbf{W}^{[2]} \\ \mathbf{W}^{[3]} \\ \mathbf{W}^{[4]} \end{bmatrix}$

$$\mathbf{W}^{[l]} = \begin{bmatrix} \mathbf{w}_1^{[l]} \\ \dots \\ \mathbf{w}_{n^{[l]}}^{[l]} \end{bmatrix}_{n^{[l]}, n^{[l-1]}}$$

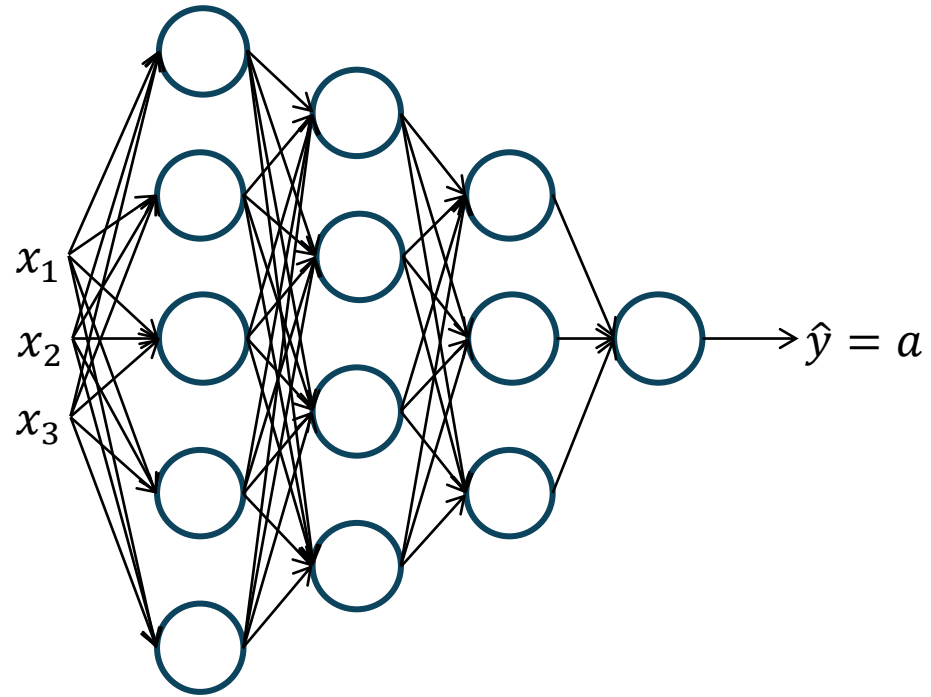
- Example

$$\mathbf{W}^{[3]} = \begin{bmatrix} w_{1,1} & \dots & w_{1,4} \\ \vdots & \ddots & \vdots \\ w_{3,1} & \dots & w_{3,4} \end{bmatrix}$$

Dimension of $\mathbf{W}^{[l]}$ is $(n^{[l]}, n^{[l-1]})$

Deep Neural Network

Biases



- Biases

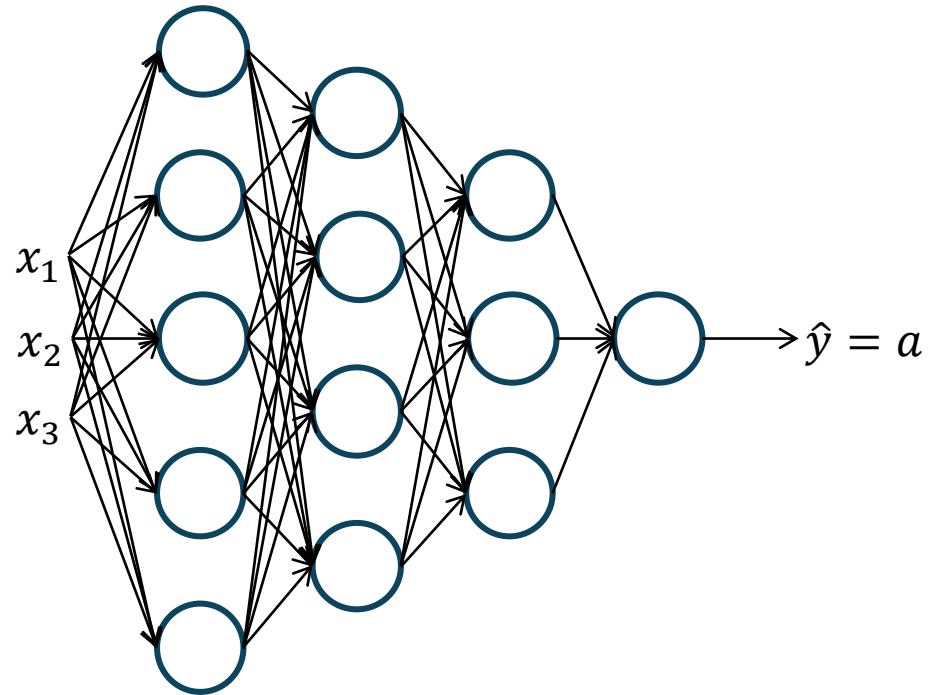
$$\mathbb{b} = [\mathbf{b}^{[1]} \quad \dots \quad \mathbf{b}^{[4]}] = \begin{bmatrix} b_1^{[1]} & \dots & b_1^{[4]} \\ \vdots & \dots & \vdots \\ b_5^{[1]} & \dots & b_3^{[4]} \end{bmatrix}$$

$$\therefore \mathbf{b}^{[l]} = \begin{bmatrix} b_1^{[l]} \\ \vdots \\ b_{n^{[l]}}^{[l]} \end{bmatrix}$$

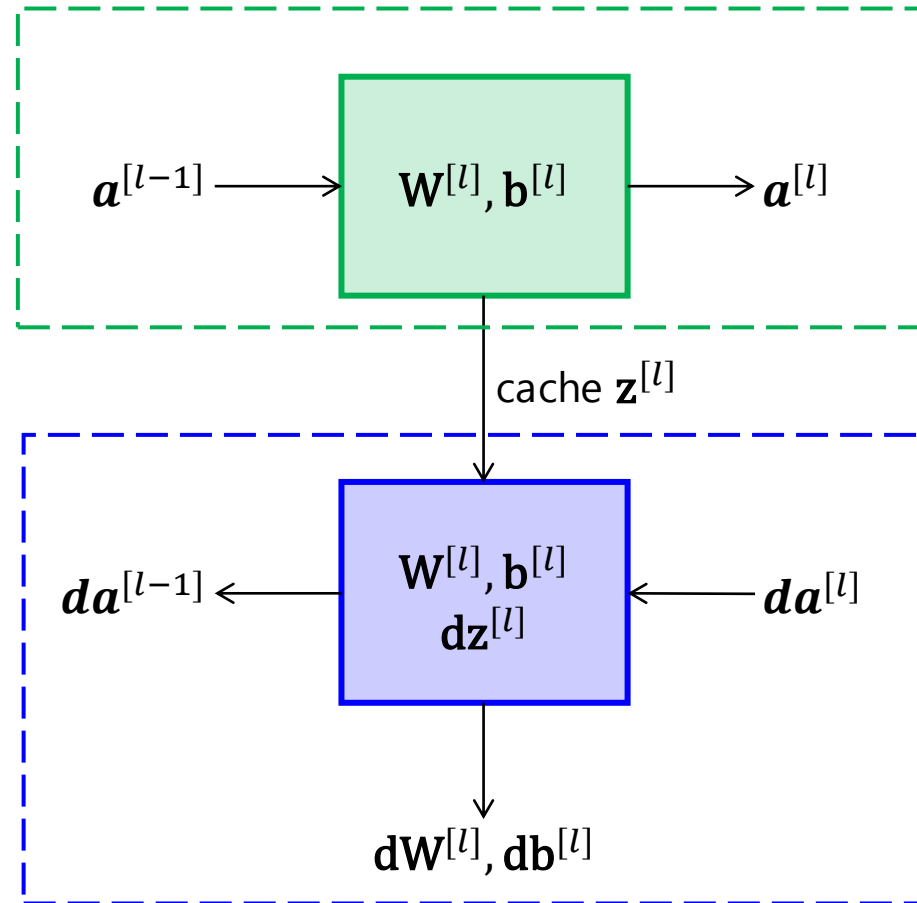
Dimension of $\mathbf{b}^{[l]}$ is $(n^{[l]}, 1)$

Implementation with Vectorization

The L layers neural network

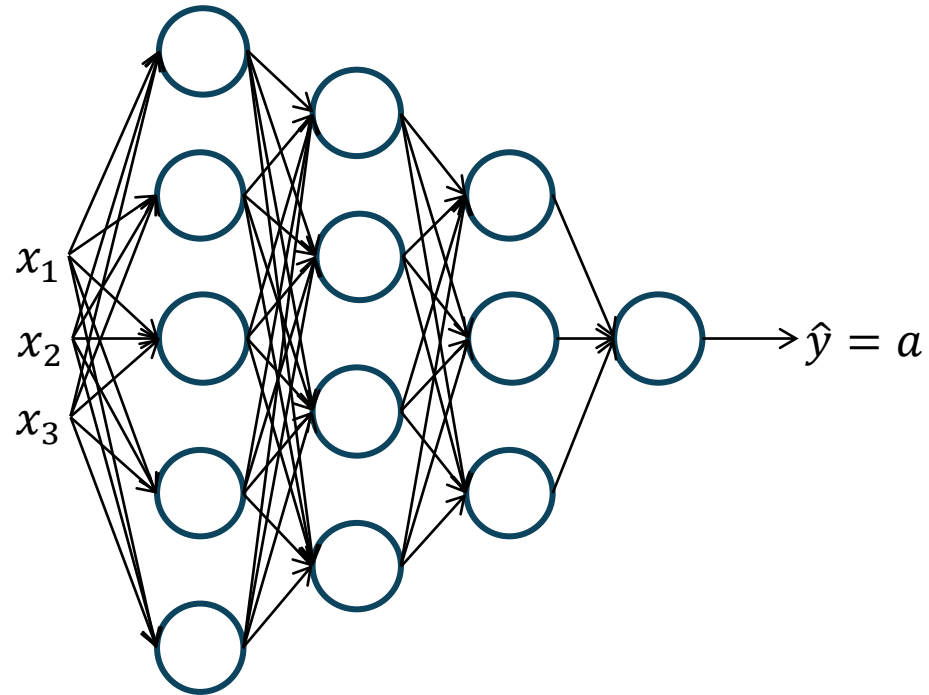


- At the layer l : $\mathbf{W}^{[l]}, \mathbf{b}^{[l]}$



Implementation with Vectorization

The L layers neural network



- At the layer l : $\mathbf{W}^{[l]}, \mathbf{b}^{[l]}$

- Forward Propagation

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \cdot \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = g^{[l]}(\mathbf{z}^{[l]})$$

- Backward Propagation

$$d\mathbf{z}^{[l]} = d\mathbf{a}^{[l]} * g'^{[l]}(\mathbf{z}^{[l]})$$

$$d\mathbf{W}^{[l]} = \frac{1}{m} d\mathbf{z}^{[l]} \cdot \mathbf{a}^{[l-1]T}$$

$$d\mathbf{b}^{[l]} = \frac{1}{m} \sum_{i=1}^m d\mathbf{z}^{[l]}$$

$np.sum(d\mathbf{z}^{[l]}, axis = 1, keepdims = True)$

$$d\mathbf{a}^{[l-1]} = \mathbf{W}^{[l]T} \cdot d\mathbf{z}^{[l]}$$

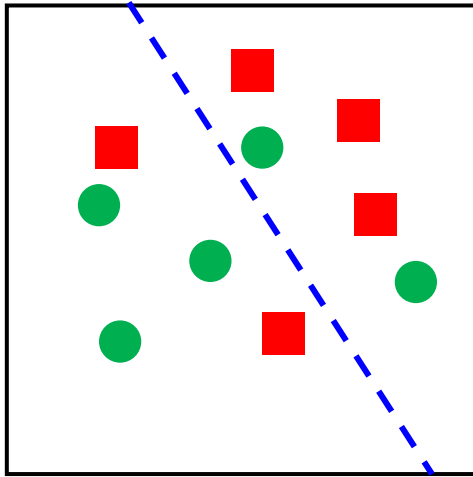
Model Tuning



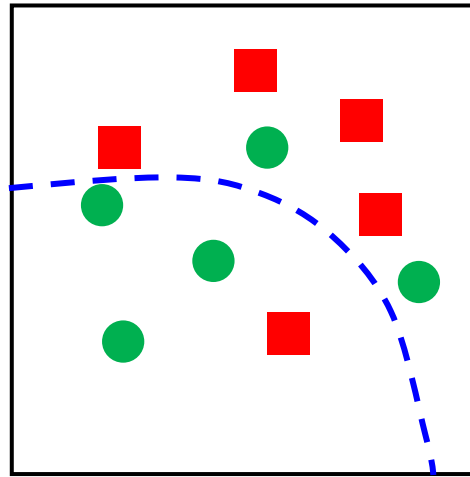
Bias and Variance Problem

Easy to learn but difficult to master

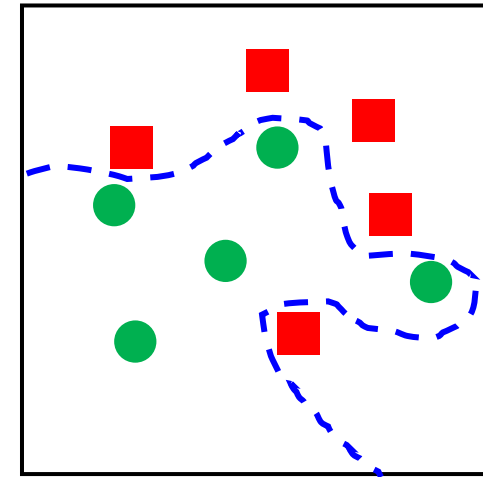
- Unable to visualize in high dimensional data



High Bias
(Underfitting)



Just right



High Variance
(Overfitting)

Regularization

L1 and L2

L1 and L2 Regularization

Recall the cost function of perceptron

Goal:

$$\min_{\mathbf{w}, b} \mathcal{J}(\mathbf{w}, b), \quad \mathbf{w} \in \mathbb{R}^{n_x}, b \in \mathbb{R}$$

- L1 regularization (Lasso)

$$\mathcal{J}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|\mathbf{w}\|_1$$

Regularization term

where $\|\mathbf{w}\|_1 = \sum_{j=1}^{n_x} |w_j|$

λ represents the regularization parameter

- L1 makes many weights become zeros
- Good for compacting the model
- L1 is not often used

L1 and L2 Regularization

Recall the cost function of perceptron

Goal:

$$\min_{\mathbf{w}, b} \mathcal{J}(\mathbf{w}, b), \quad \mathbf{w} \in \mathbb{R}^{n_x}, b \in \mathbb{R}$$

- L2 regularization (Ridge)

$$\mathcal{J}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|\mathbf{w}\|_2^2$$

Regularization term

where $\|\mathbf{w}\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = \mathbf{w}^T \cdot \mathbf{w}$

λ represents the regularization parameter

- L2 is much more often used compared to L1

L2 Regularization

L2 of Deep Neural Network

The cost function

$$\mathcal{J}(\mathbf{w}^{[1]}, \mathbf{b}^{[1]}, \dots, \mathbf{w}^{[L]}, \mathbf{b}^{[L]}) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|\mathbf{w}^{[l]}\|_F^2$$

where

Frobenious norm $\|\mathbf{w}\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2$

$$\mathbf{w}^{[l]}: (n^{[l]}, n^{[l-1]})$$

- Modifying back propagation

$$d\mathbf{w}^{[l]} = (\cdot) + \frac{\lambda}{m} \mathbf{w}^{[l]}$$

$$\mathbf{w}^{[l]} = \mathbf{w}^{[l]} - \alpha d\mathbf{w}^{[l]}$$

$$= \mathbf{w}^{[l]} - \alpha \left[(\cdot) + \frac{\lambda}{m} \mathbf{w}^{[l]} \right]$$

$$= \mathbf{w}^{[l]} - \frac{\alpha\lambda}{m} \mathbf{w}^{[l]} - \alpha(\cdot)$$

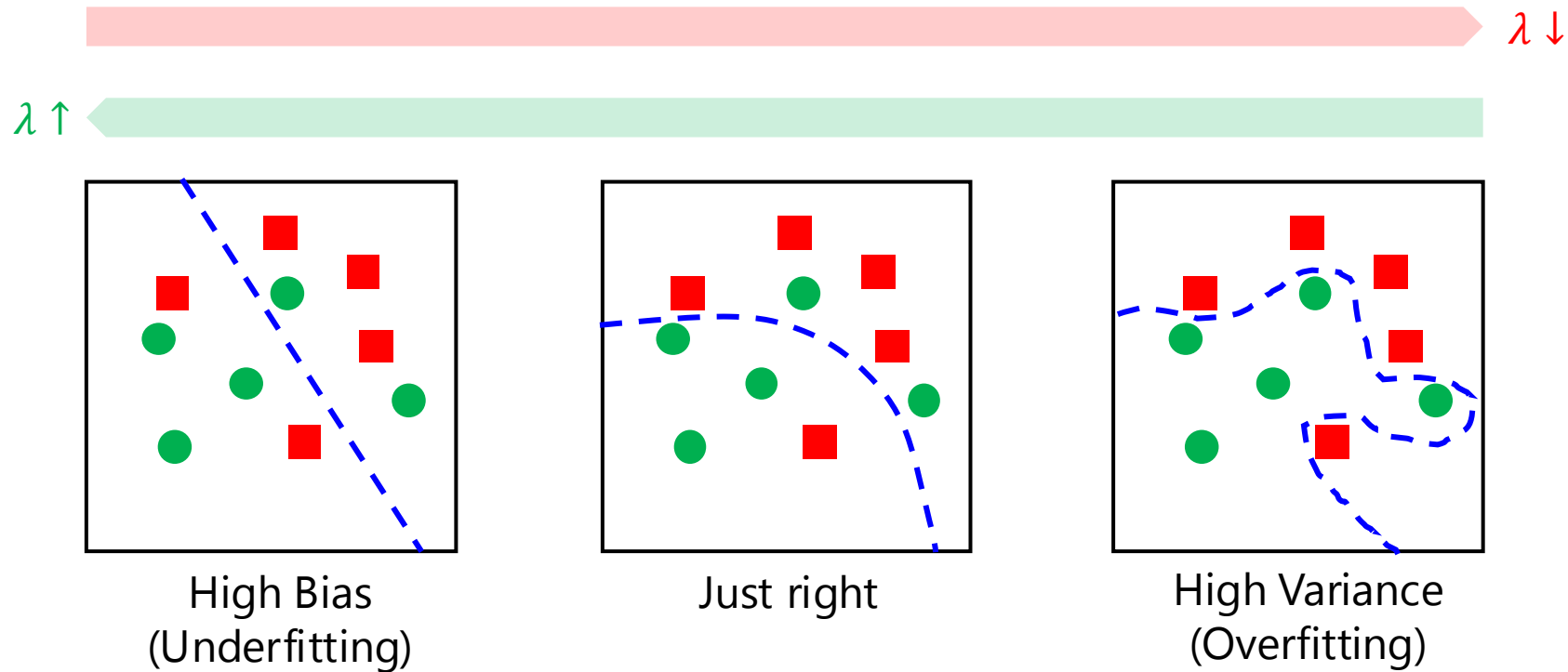
$$= \left(1 - \frac{\alpha\lambda}{m}\right) \mathbf{w}^{[l]} - \alpha(\cdot)$$

Weight Decay

where $(\cdot)$ represents the term obtained from original back propagation

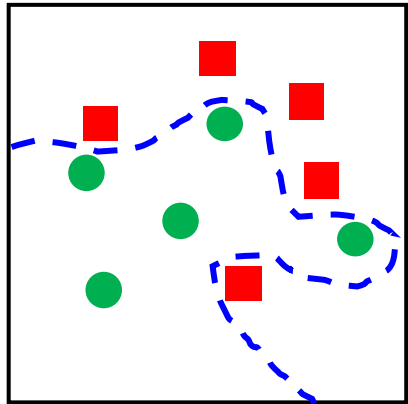
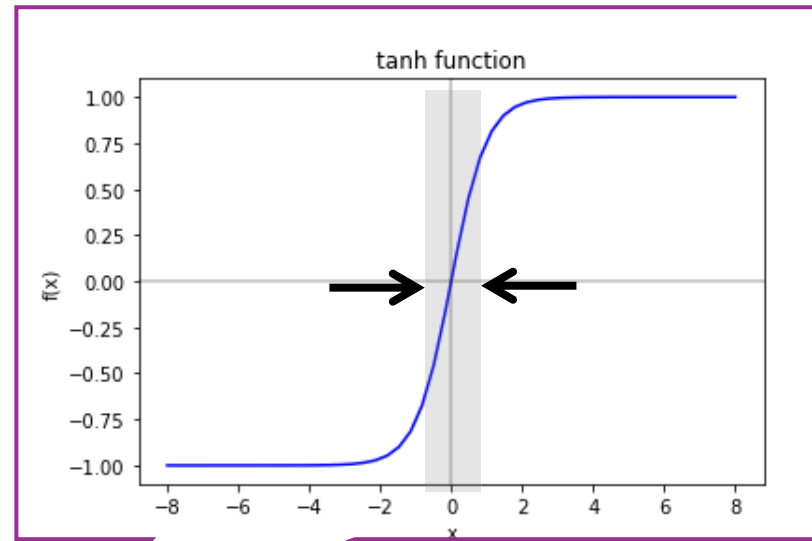
L2 Regularization

Why does L2 prevent overfitting?



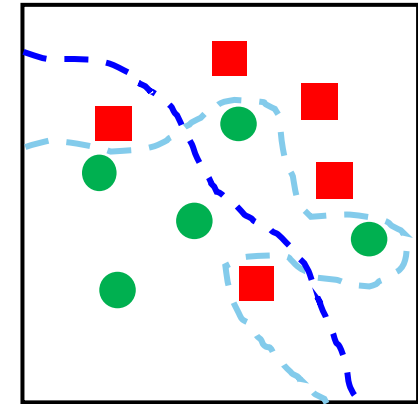
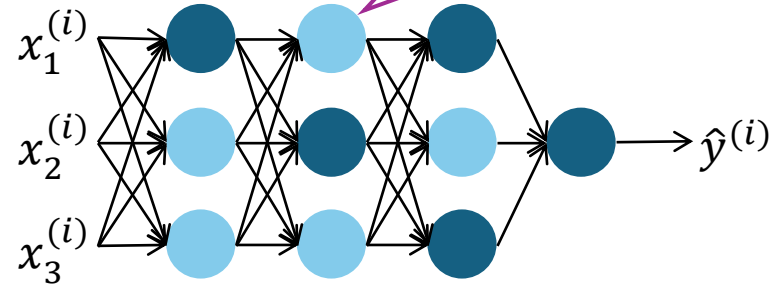
L2 Regularization

Why does L2 prevent overfitting?



$\lambda \uparrow$
 $\mathbf{w}^{[l]} \downarrow$
 $\mathbf{z}^{[l]} \downarrow$

$$\because \mathbf{z}^{[l]} = \mathbf{w}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$



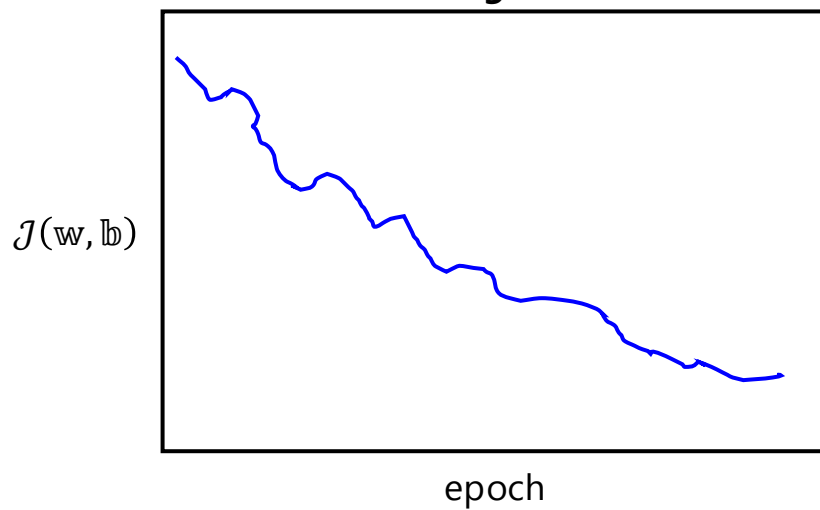
Decision boundary is stretched out

L2 Regularization

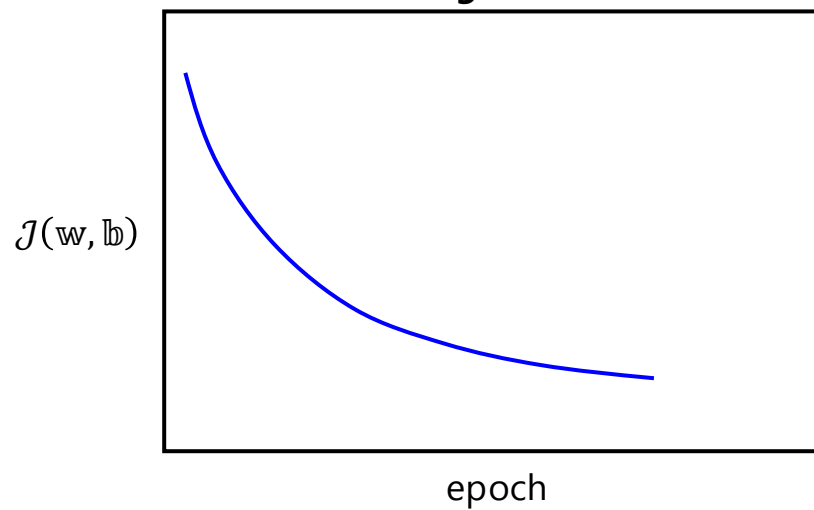
Smoother cost function

$$\mathcal{J}(\mathbf{w}^{[1]}, \mathbf{b}^{[1]}, \dots, \mathbf{w}^{[L]}, \mathbf{b}^{[L]}) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|\mathbf{w}^{[l]}\|_F^2$$

Without regularization



With regularization



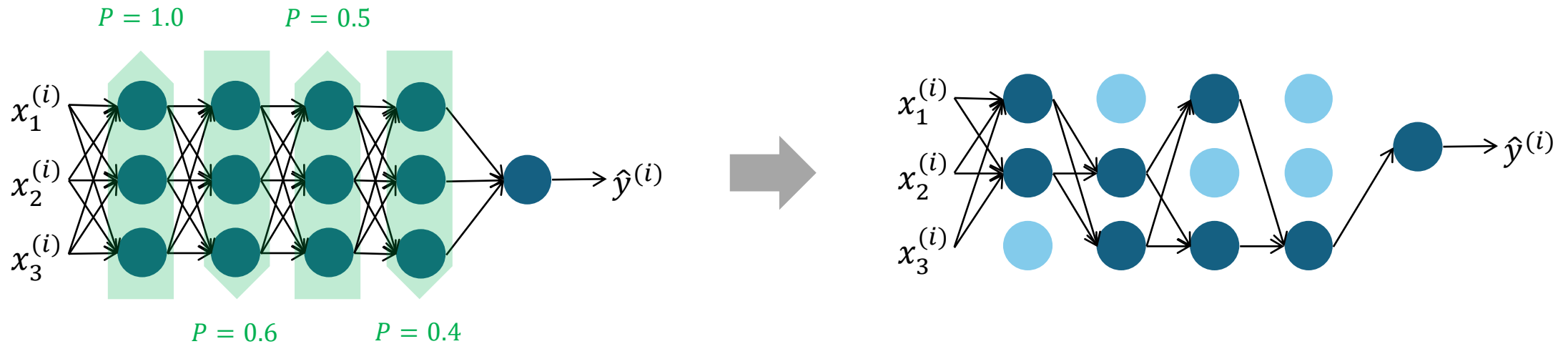
Regularization

DropOut & DropConnect

DropOut

Ensemble Neural Network

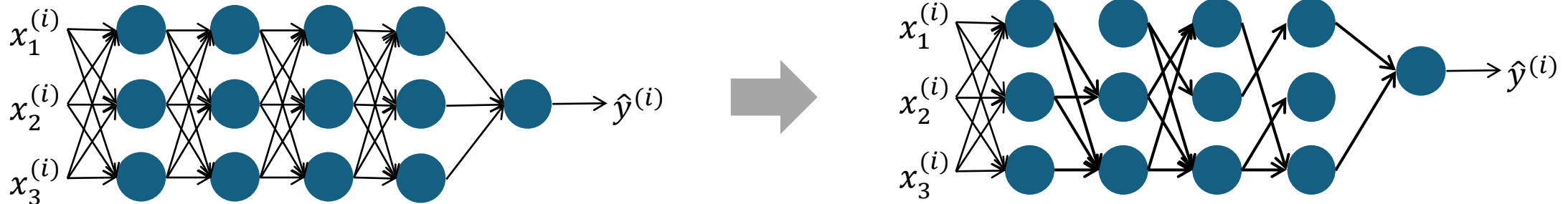
- Set the probability of enable hidden units in each hidden layer
- The edges connected to any disabled hidden unit will be removed
- DropOut is applied ONLY training but NOT testing
- DropOut produces many possible combination of neuron network
- DropOut is very popular in computer vision



DropConnect

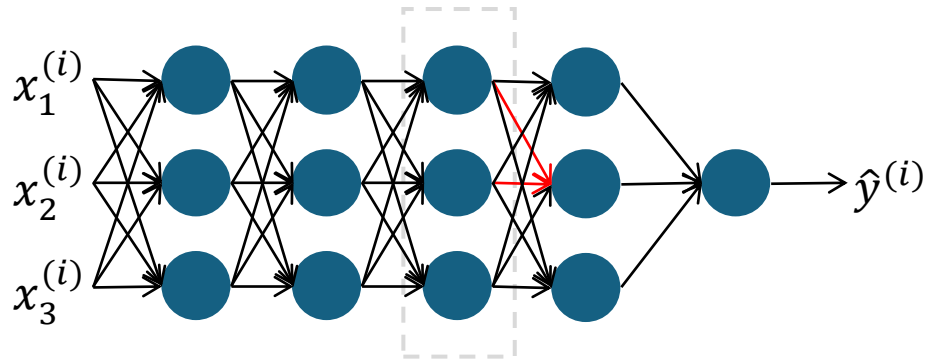
Generalization of DropOut

- Set the Boolean mask to randomly disable connections
- Rescale output on active connections
- DropConnect is applied ONLY for training but NOT testing
- DropConnect makes even more possible combination than dropout
- DropConnect is very popular in computer vision



Implementation

Illustration of DropConnect at the layer 3



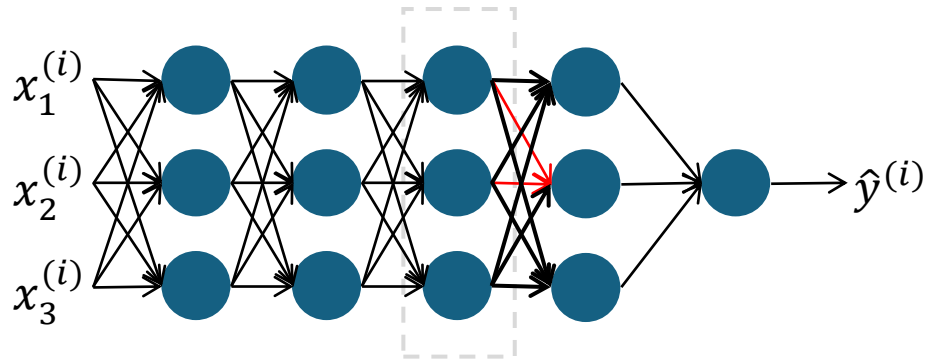
- Define theoretical variable to Python variable
 - P as keepprob
- Create a Boolean mask to randomly disable weights

```
keepprob = 0.8  
d3 = np.random.rand(a3.shape[0], a3.shape[1]) < keepprob
```

```
In [43]: d3  
Out[43]:  
array([[ True,  True,  True],  
       [False,  True,  True],  
       [ True, False,  True]])
```

Implementation

Illustration of DropConnect at the layer 3



- Rescale the $\mathbf{a}^{[3]}$

```
a3 /= keepprob
```

- Example: If $\mathbf{a}^{[3]}$ is

```
array([[1., 2., 3.],  
       [4., 5., 6.],  
       [7., 8., 9.]])
```

- Rescale $\mathbf{a}^{[3]}$ after drop connect will be

```
array([[ 1.25,  2.5 ,  3.75],  
       [ 0.   ,  6.25,  7.5 ],  
       [ 8.75,  0.   , 11.25]])
```

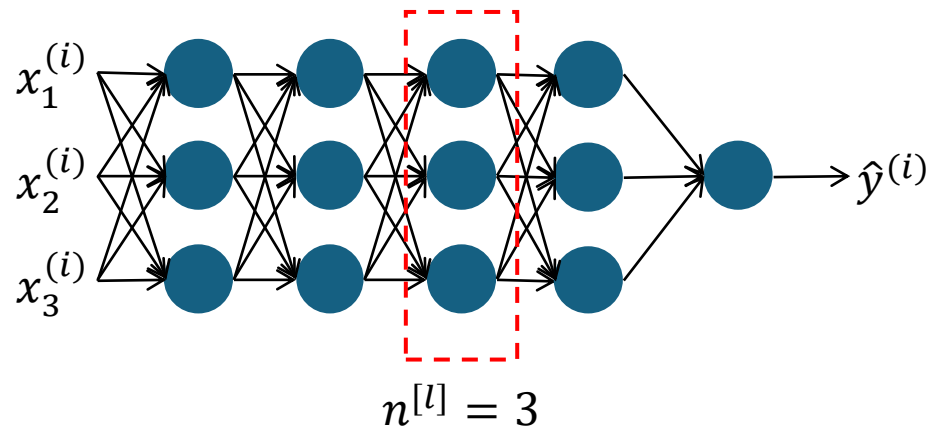
- In practice $\mathbf{a}^{[3]}$ is obtained from

$$\mathbf{a}^{[3]} = g^{[3]}(\mathbf{z}^{[3]}) = g^{[3]}(\mathbf{W}^{[3]} \cdot \mathbf{a}^{[2]} + \mathbf{b}^{[3]})$$

Recommended Settings

Some idea to set your keepprob

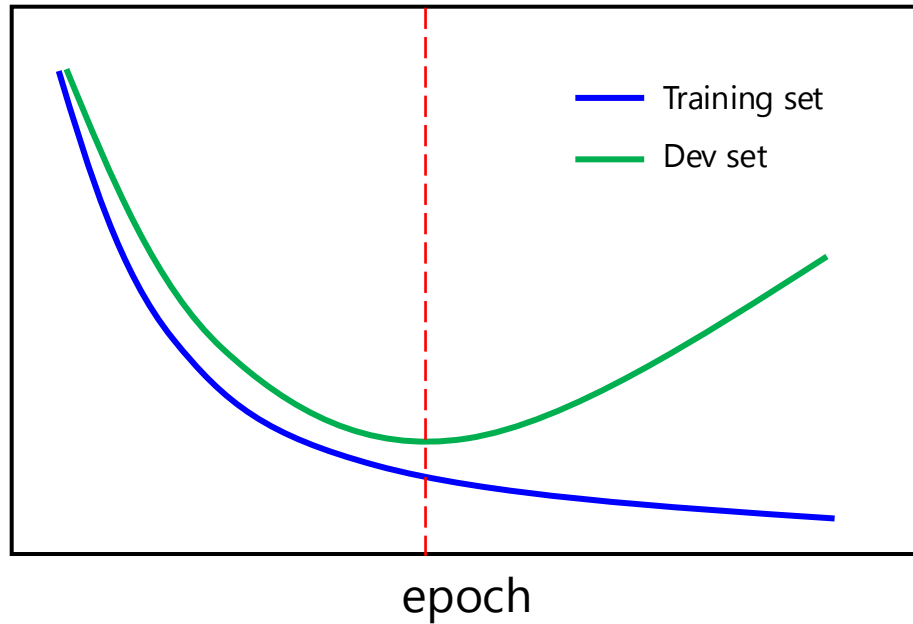
- The keepprob can be different value in different layer
- The lower $n^{[l]}$ (number of hidden units in the layer) the higher keepprob
- Input layer has keepprob equal to 1



Other Regularizations

Early stopping

$\mathcal{J}(\mathbf{W}, \mathbf{b})$



- Stop training based on the cost convergence

Gradient



Normalized Input

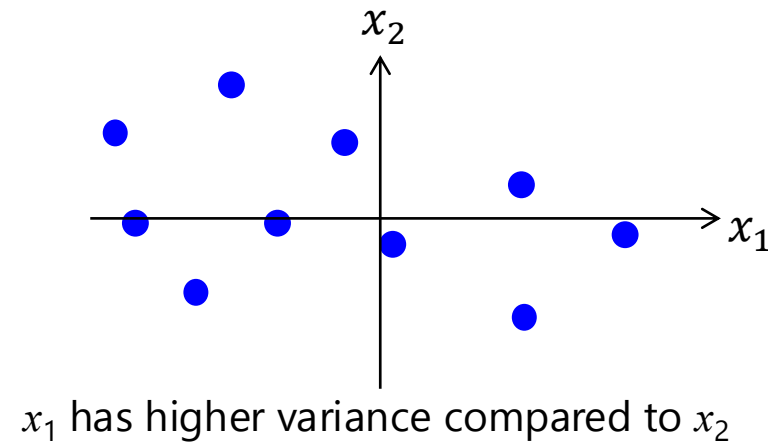
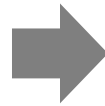
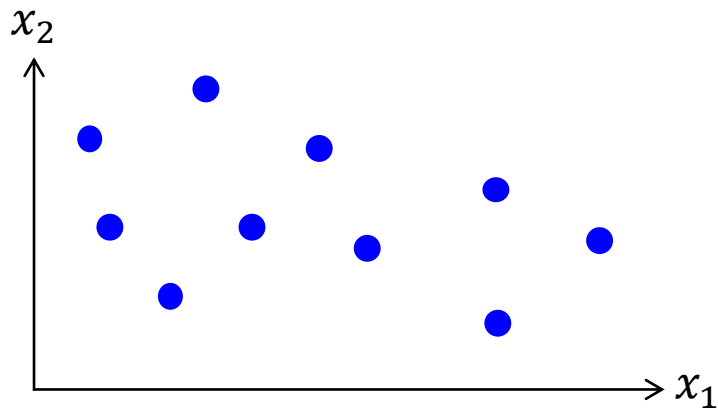
Speed up your training

- Subtract mean

$$\mu = \frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)}$$

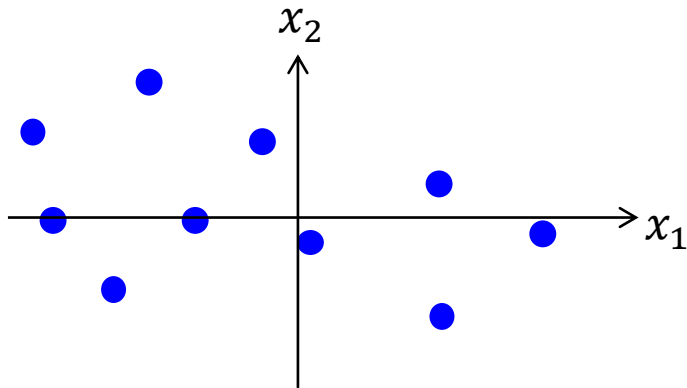
$$\mathbf{X} := \mathbf{X} - \mu$$

- Data will have zero mean



Normalized Input

Speed up your training

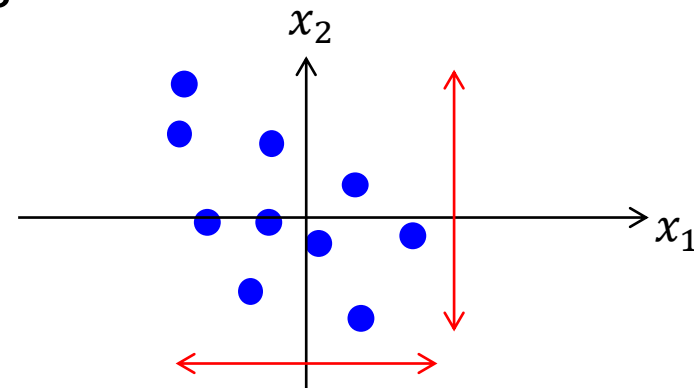


- Normalized variance

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}^{(i)})^2$$

$$\mathbf{X} = \frac{\mathbf{X}}{\sigma^2}$$

- Data will have zero mean and normalized variance

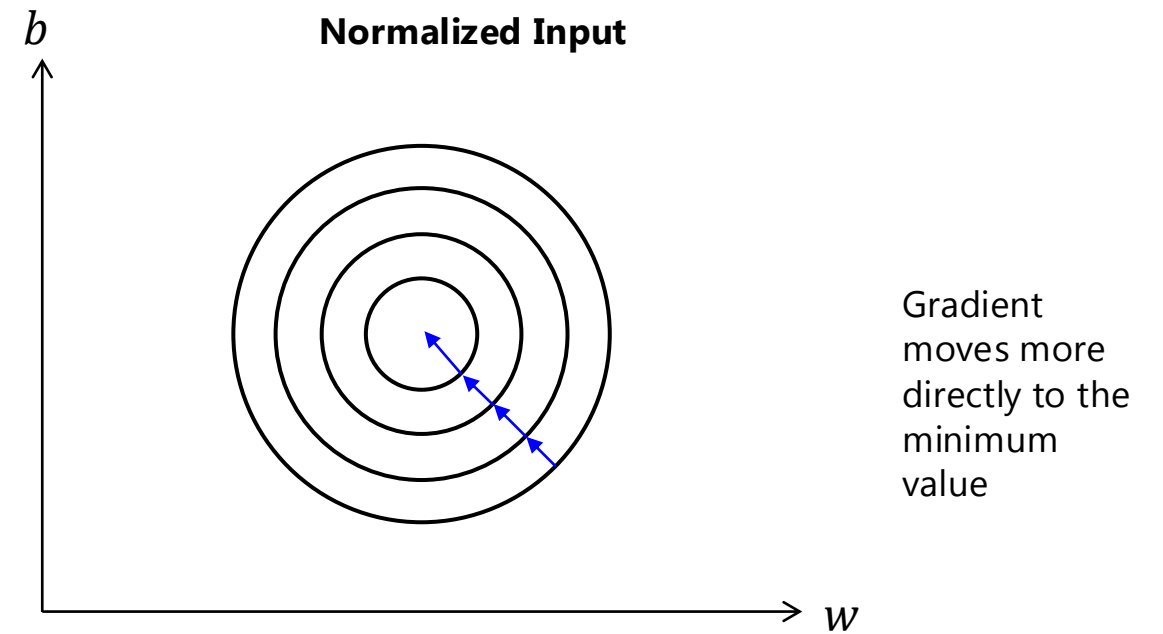
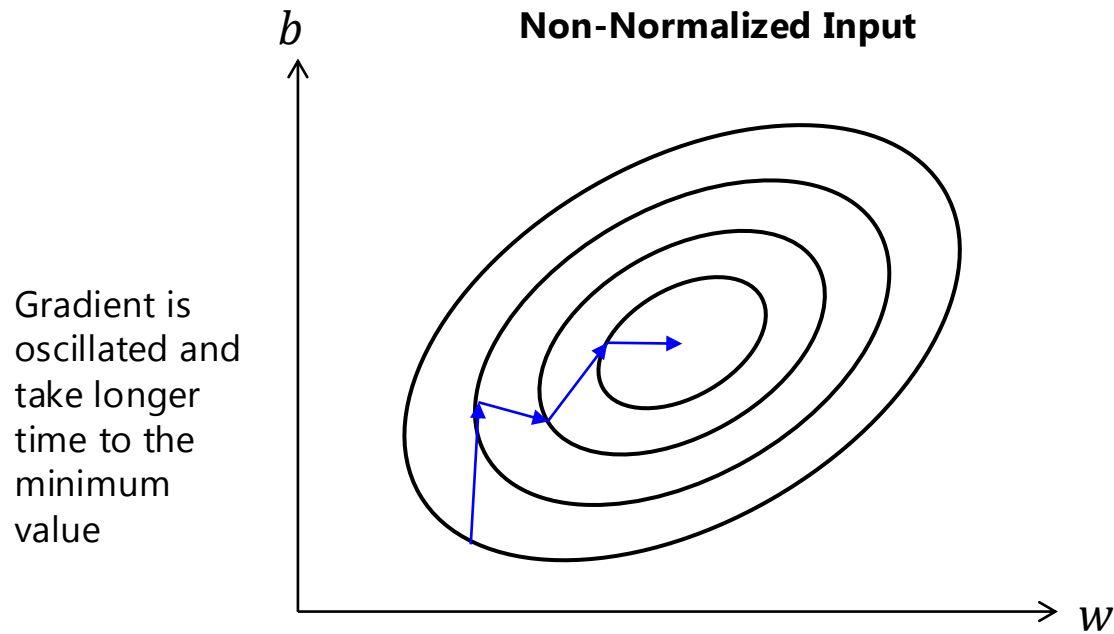


- μ and σ^2 from training set will be used in test set

- Given a contour of the cost function

Normalized Input

Speed up your training



Minibatch Gradient Descent



Batch VS Mini-batch

Dividing training data into minibatch

Notations

$\mathbf{X}^{\{t\}}$ and $\mathbf{y}^{\{t\}}$ for training samples of mini-batch t

- Vectorization enables effective computation over m samples

$$\mathbf{X}_{n_x \times m} = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}]$$

$$\mathbf{y}_{1 \times m} = [y^{(1)}, y^{(2)}, \dots, y^{(m)}]$$

- What if m is extremely large, e.g. 5,000,000
 - Use mini-batch

$$\begin{aligned} \mathbf{X}_{n_x \times m} &= [\underbrace{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(1000)}}_{\mathbf{X}^{\{1\}}} | \underbrace{\mathbf{x}^{(1001)}, \mathbf{x}^{(1002)}, \dots, \mathbf{x}^{(2000)}}_{\mathbf{X}^{\{2\}}} | \dots \dots | \dots \dots \underbrace{\mathbf{x}^{(5,000,000)}}_{\mathbf{X}^{\{5000\}}}] \\ \mathbf{y}_{1 \times m} &= [\underbrace{y^{(1)}, y^{(2)}, \dots, y^{(1000)}}_{\mathbf{y}^{\{1\}}} | \underbrace{y^{(1001)}, y^{(1002)}, \dots, y^{(2000)}}_{\mathbf{y}^{\{2\}}} | \dots \dots | \dots \dots \underbrace{y^{(5,000,000)}}_{\mathbf{y}^{\{5000\}}}] \end{aligned}$$

Mini-Batch Implementation

From the previous example

```
while {  
  for t in range(0, 5000):  
    # Forward propagation on  $X^{\{t\}}$ 
```

$$\mathbf{z}^{[1]} = \mathbf{w}^{[1]} \cdot \mathbf{X}^{\{t\}} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = g(\mathbf{z}^{[1]})$$

$\vdots$

$$\mathbf{z}^{[L]} = \mathbf{w}^{[L]} \cdot \mathbf{a}^{[L-1]} + \mathbf{b}^{[L]}$$

$$\mathbf{a}^{[L]} = g(\mathbf{z}^{[L]})$$

Use vectorization implementation

from $\mathbf{X}^{\{t\}}, \mathbf{y}^{\{t\}}$

$$\mathcal{J}^{\{t\}} = \frac{1}{1000} \sum_{i=1}^{1000} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \times 1000} \sum_l \|\mathbf{w}^{[l]}\|_F^2$$

Backward propagation w.r.t. $\mathcal{J}^{\{t\}}$ using $\mathbf{X}^{\{t\}}$ and $\mathbf{y}^{\{t\}}$

$$\mathbf{w}^{[l]} = \mathbf{w}^{[l]} - \alpha \mathbf{dw}^{[l]}$$

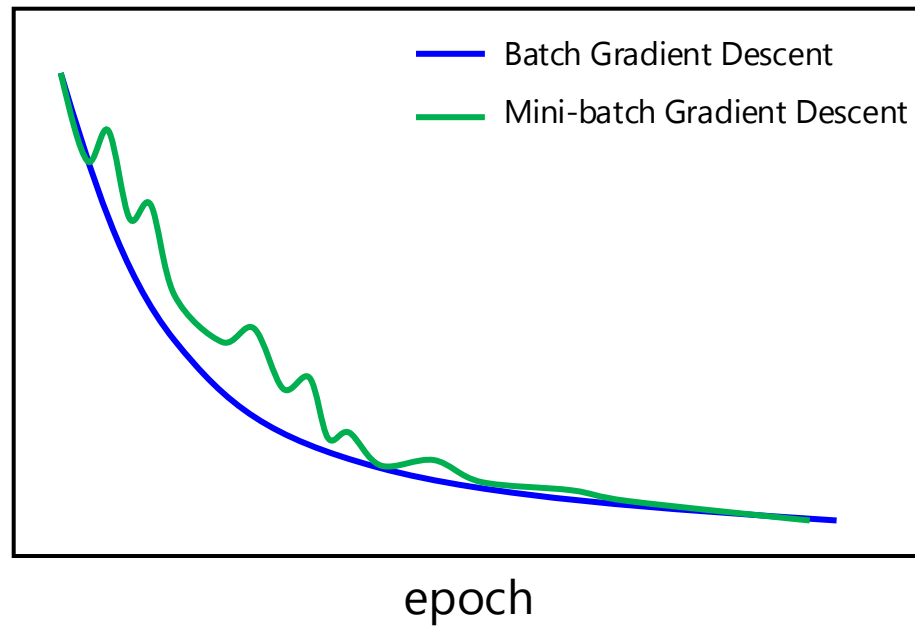
$$\mathbf{b}^{[l]} = \mathbf{b}^{[l]} - \alpha \mathbf{db}^{[l]}$$

```
} until converge
```

Mini-batch gradient descent

Cost function comparison

$\mathcal{J}(\mathbf{W}, \mathbf{b})$

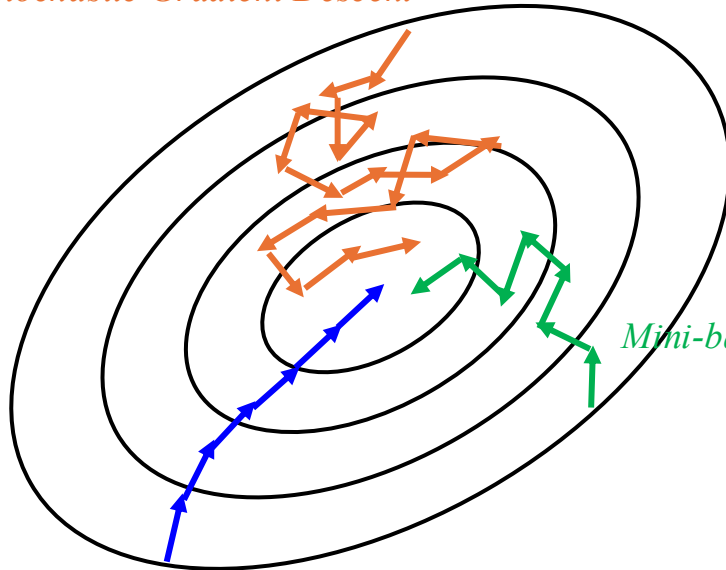


- The cost function of mini-batch gradient descent could be oscillated
 - $\mathbf{X}^{\{1\}}, \mathbf{y}^{\{1\}}$ might be an easy gradient
 - $\mathbf{X}^{\{2\}}, \mathbf{y}^{\{2\}}$ might be a harder gradient
 - Overall should be downward

Choosing Mini-batch size

The extreme cases

Stochastic Gradient Descent



Batch Gradient Descent

- Mini-batch size = m
 - Batch gradient descent

$$(\mathbf{X}^{\{1\}}, \mathbf{y}^{\{1\}}) = (\mathbf{X}, \mathbf{y})$$

- Mini-batch size = 1
 - Stochastic gradient descent

$$(\mathbf{X}^{\{1\}}, \mathbf{y}^{\{1\}}) = (\mathbf{x}^{(1)}, \mathbf{y}^{(1)})$$

Characteristics	Batch	Mini-batch	SGD
Speed	Very Slow	Fast	Slow
Convergence to minimum	Guarantee	Not Guarantee	Not Guarantee

Does not leverage vectorization

Choosing Mini-batch size

Conclusion

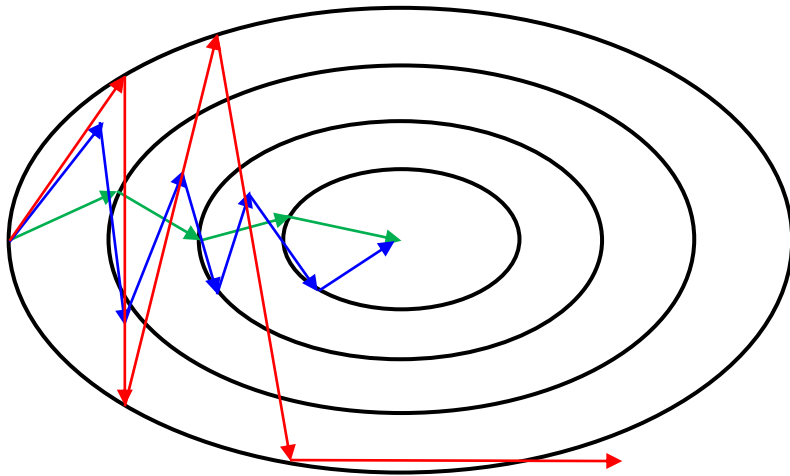
- Small Training Set ($m \leq 2000$)
 - Use Batch gradient descent
- Otherwise
 - Use Mini-batch
- Suggested mini-batch size
 - 2^k to perfectly fit in CPU/GPU memory, e.g. 64, 128, 256, 512, etc

Gradient Optimization

Momentum Gradient Descent, RMSprop, and
ADAM

Momentum Gradient Descent

Idea



- Gradient Descent
- Momentum Gradient Descent
- Overshoot (too much learning rate)

- Desired movement directions
 - Fast to the minimum point

From our example, movement on this direction (→) should be fast

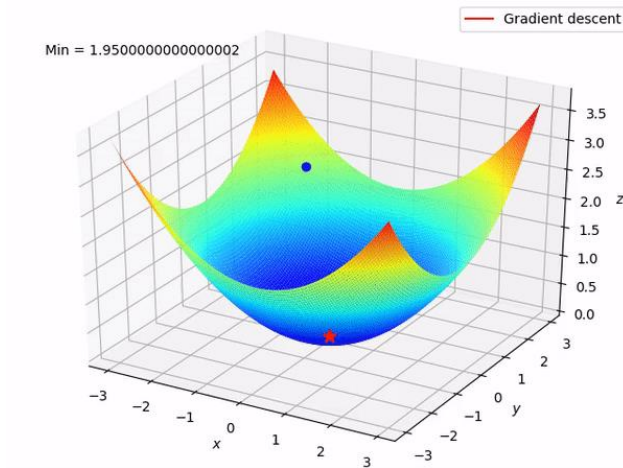
- Slow to others

From our example, movement on this direction (↕) should be slow

Momentum Gradient Descent

Velocity and Acceleration

Similar to a ball rolling in a bowl



- On iteration t:
Compute $\mathbf{dW}$, $\mathbf{db}$ on a set of mini-batch

$$\begin{aligned} \mathbf{v}_{\mathbf{dW}} &= \beta \mathbf{v}_{\mathbf{dW}} + (1 - \beta) \mathbf{dW} \\ \mathbf{v}_{\mathbf{db}} &= \beta \mathbf{v}_{\mathbf{db}} + (1 - \beta) \mathbf{db} \end{aligned}$$

Diagram illustrating the momentum update equations for $\mathbf{v}_{\mathbf{dW}}$ and $\mathbf{v}_{\mathbf{db}}$. The equations show the current velocity (green box) being updated with a fraction β of the previous velocity and a fraction $(1 - \beta)$ of the current acceleration (blue box). The diagram includes arrows indicating the flow of information: a red arrow labeled 'friction' points from the previous velocity to the current velocity, a green arrow labeled 'velocity' points from the current velocity to the current velocity, and a blue arrow labeled 'acceleration' points from the current acceleration to the current velocity.

$$\mathbf{W} = \mathbf{W} - \alpha \mathbf{v}_{\mathbf{dW}}$$

$$\mathbf{b} = \mathbf{b} - \alpha \mathbf{v}_{\mathbf{db}}$$

Momentum Gradient Descent

Implementation

Initialize $\mathbf{v}_{\mathbf{dW}} = 0, \mathbf{v}_{\mathbf{db}} = 0$

Note: dimension of $\mathbf{v}_{\mathbf{dW}} = \mathbf{dW}$

dimension of $\mathbf{v}_{\mathbf{db}} = \mathbf{db}$

Hyper-parameters are α, β

- On iteration t :
Compute $\mathbf{dW}, \mathbf{db}$ on a set of mini-batch

$$\mathbf{v}_{\mathbf{dW}} = \beta \mathbf{v}_{\mathbf{dW}} + (1 - \beta) \mathbf{dW}$$

$$\mathbf{v}_{\mathbf{db}} = \beta \mathbf{v}_{\mathbf{db}} + (1 - \beta) \mathbf{db}$$

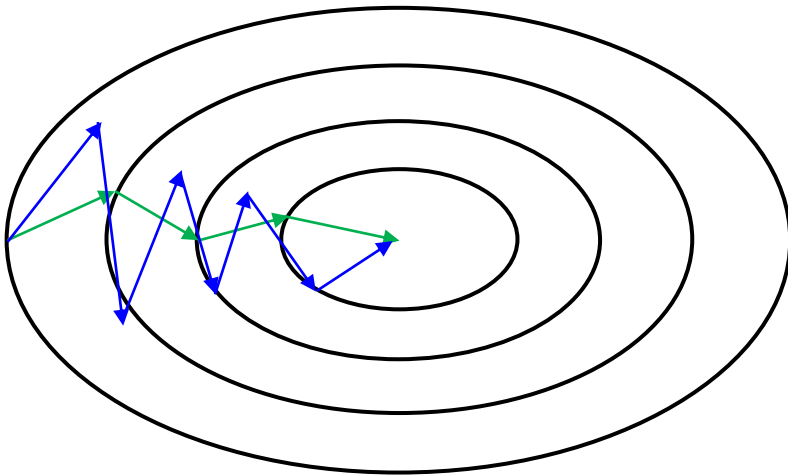
$$\mathbf{W} = \mathbf{W} - \alpha \mathbf{v}_{\mathbf{dW}}$$

$$\mathbf{b} = \mathbf{b} - \alpha \mathbf{v}_{\mathbf{db}}$$

- In practice, $\beta = 0.9$
- Some literature omitted $1 - \beta$

RMSprop

Speed up gradient descent



- Gradient Descent
- RMSprop

- On iteration t :
Compute $\mathbf{dW}$, $\mathbf{db}$ on a set of mini-batch

$$s_{\mathbf{dW}} = \beta s_{\mathbf{dW}} + (1 - \beta) \mathbf{dW}^2$$

$$s_{\mathbf{db}} = \beta s_{\mathbf{db}} + (1 - \beta) \mathbf{db}^2$$

$$\mathbf{W} = \mathbf{W} - \frac{\alpha \mathbf{dW}}{\sqrt{s_{\mathbf{dW}} + \epsilon}}$$

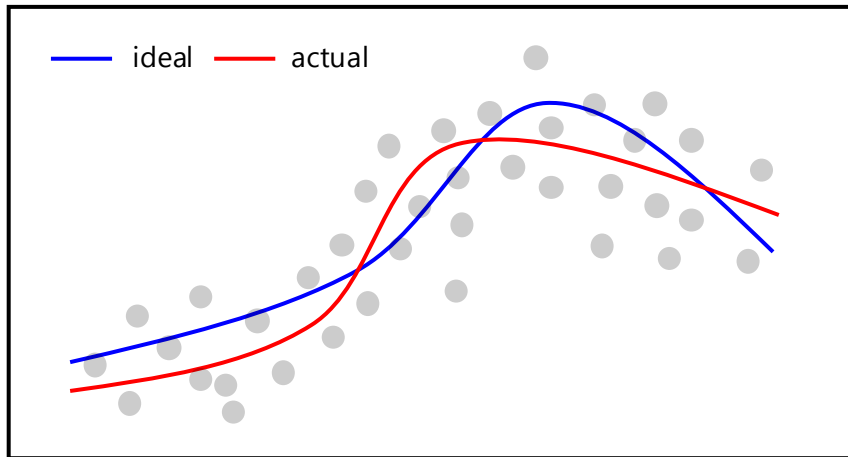
$$\mathbf{b} = \mathbf{b} - \frac{\alpha \mathbf{db}}{\sqrt{s_{\mathbf{db}} + \epsilon}}$$

*a small positive number for numerical stabilization
(preventing divided by zero)*

- RMSprop enables to use the larger learning rate without overshoot movement

Bias Correction

Idea



- Bias correction of data point t is defined as

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t \xrightarrow{\text{data point } t}$$

$$\text{set } \beta = 0.98, \quad v_0 = 0$$

$$v_1 = 0.98v_0 + 0.02\theta_1$$

$$v_2 = 0.98v_1 + 0.02\theta_2 = 0.0196\theta_1 + 0.02\theta_2$$

- Use $\frac{v_t}{1 - \beta^t}$ to estimate the ideal data point t
- Not accurate at the beginning of t but later will be much more accurate

Adaptive Moment Estimation (Adam)

Combining momentum and RMSprop

Initialize $\mathbf{v}_{dW} = 0, s_{dW} = 0, \mathbf{v}_{db} = 0, s_{db} = 0$

Note: Adam works well in wide range across deep learning applications

- On iteration t:

Compute $\mathbf{dW}, \mathbf{db}$ on a set of mini-batch

$$\mathbf{v}_{dW} = \beta_1 \mathbf{v}_{dW} + (1 - \beta_1) \mathbf{dW}$$

$$\mathbf{v}_{db} = \beta_1 \mathbf{v}_{db} + (1 - \beta_1) \mathbf{db}$$

momentum

$$s_{dW} = \beta_2 s_{dW} + (1 - \beta_2) \mathbf{dW}^2$$

$$s_{db} = \beta_2 s_{db} + (1 - \beta_2) \mathbf{db}^2$$

RMSprop

$$\mathbf{v}_{dW}^{\text{correct}} = \frac{\mathbf{v}_{dW}}{1 - \beta_1^t}, \quad \mathbf{v}_{db}^{\text{correct}} = \frac{\mathbf{v}_{db}}{1 - \beta_1^t}$$

$$s_{dW}^{\text{correct}} = \frac{s_{dW}}{1 - \beta_2^t}, \quad s_{db}^{\text{correct}} = \frac{s_{db}}{1 - \beta_2^t}$$

bias correction

$$\mathbf{W}^{[l]} = \mathbf{W}^{[l]} - \alpha \frac{\mathbf{v}_{dW}^{\text{correct}}}{\sqrt{s_{dW}^{\text{correct}} + \epsilon}}$$

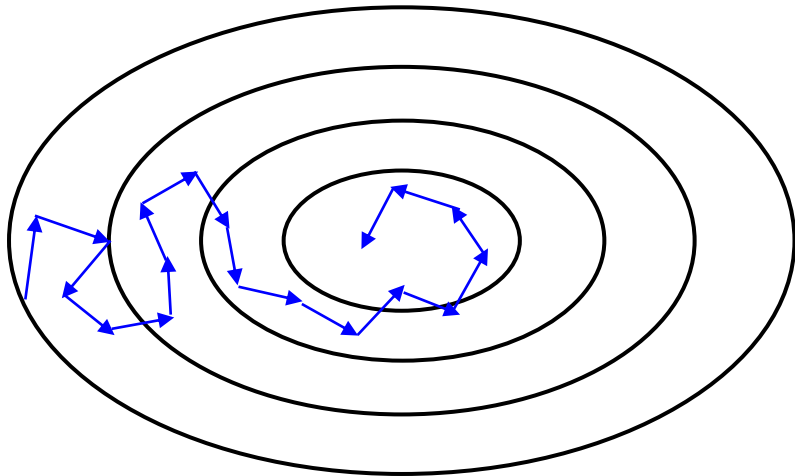
$$\mathbf{b}^{[l]} = \mathbf{b}^{[l]} - \alpha \frac{\mathbf{v}_{db}^{\text{correct}}}{\sqrt{s_{db}^{\text{correct}} + \epsilon}}$$

updating

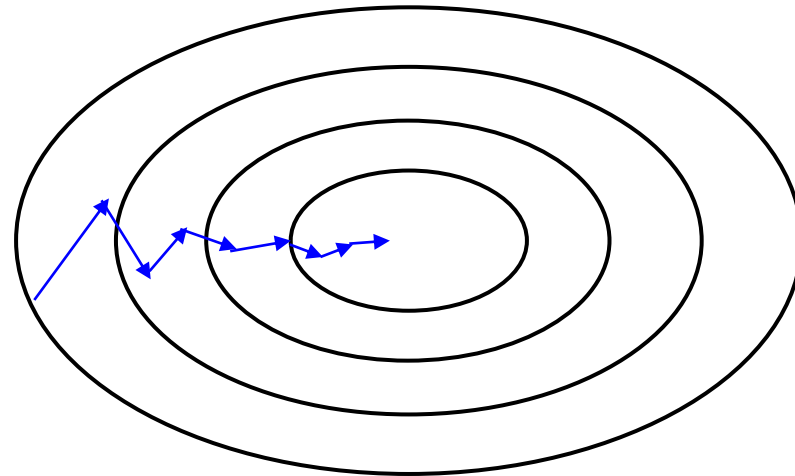
Suggested Hyper-parameters

Fixed learning rate VS Decayed Learning rate

- α : Need to be tuned
- β_1 : 0.9
- β_2 : 0.999
- ε : 10^{-8}



Fixed learning rate



Decayed Learning rate

Hyperparameter Tuning

Strategy to choose hyper-parameters

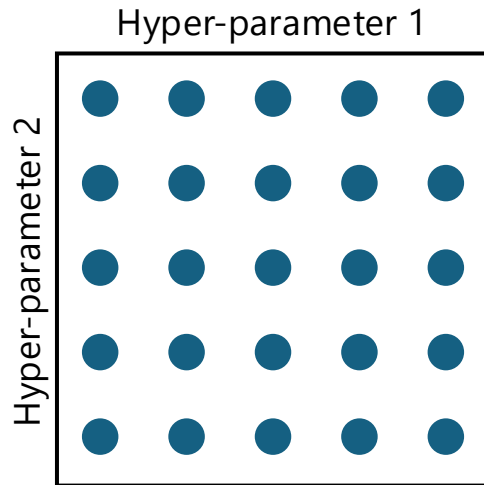
Deep learning hyperparameters

Ranked by importance

- 1st priority
 - α : Learning rate
- 2nd priority
 - β : Adam's parameters only ($\beta_1, \beta_2, \varepsilon$) (Use default values)
 - Mini-batch size
 - $n^{[l]}$: Number of hidden units in the layer l
- 3rd priority
 - L : Number of layers
 - β : Momentum and RMSprop

Tuning Strategies

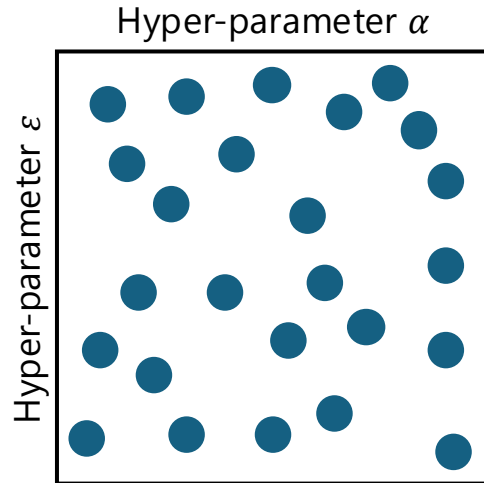
Grid approach



- Grid approach is acceptable for traditional ML
- Limitations
 - Important of some hyperparameters might not be fully address
 - Ex. Grid allows trial on 5 values of α and ε from 25 experiments

Tuning Strategies

For deep learning random grid is better

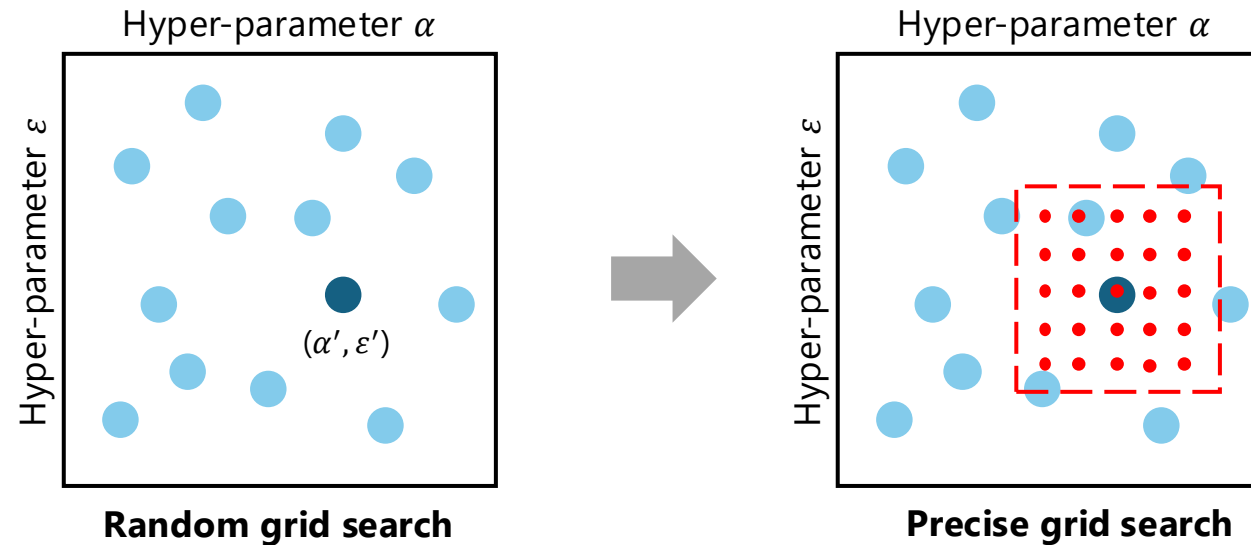


- Random grid allows trial on 25 values of α from 25 experiments
- Random grid gives us more richly to explore sets of possible hyperparameters

Tuning Strategies

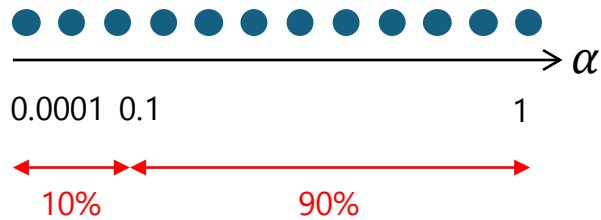
Coarse to fine

- Start from randomly coarse
- Randomly fine later



Tuning Strategies

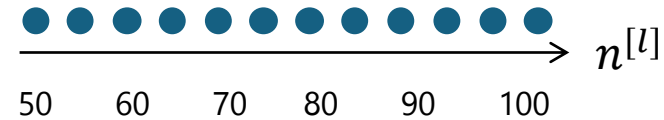
Use the appropriate scale to pitch hyperparameters



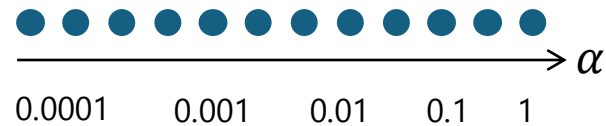
Spending 10% of resource to search between 0.0001 to 0.1 whereas 90% of resources to search between 0.1 to 1

- Normal scale works for some hyperparameters

- $n^{[l]}$
- L



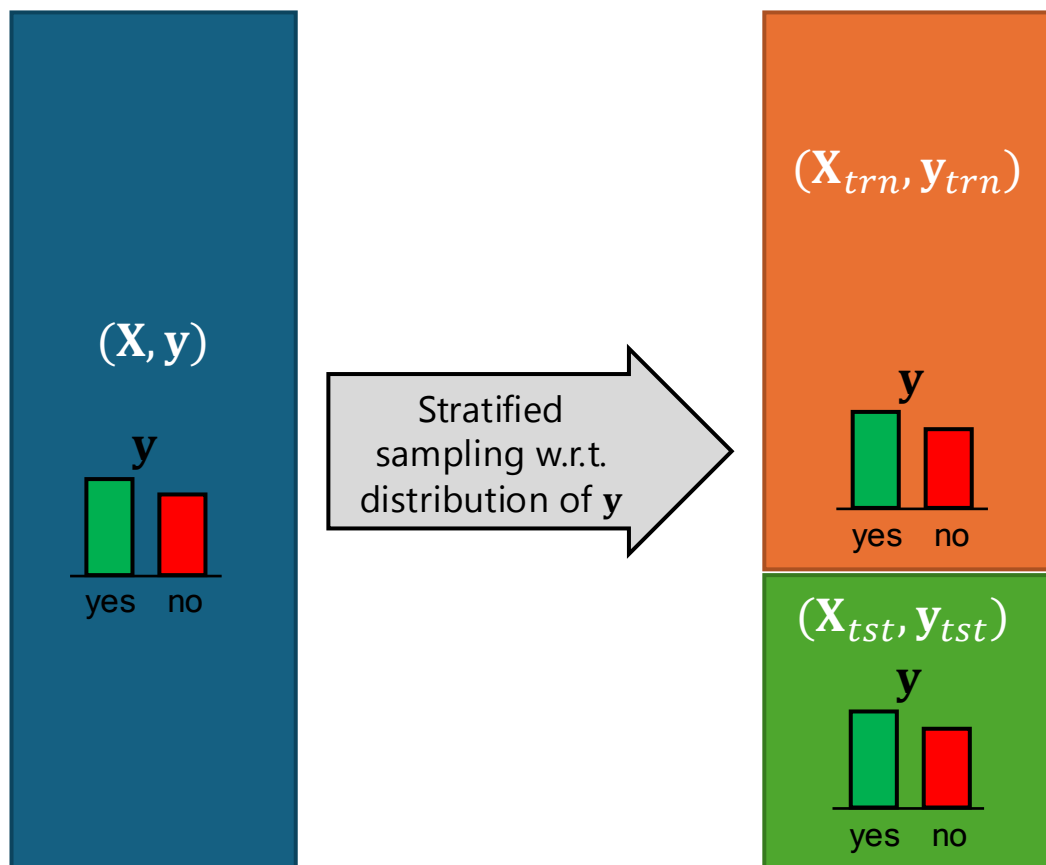
- Log-scale is good for learning rate



Machine Learning Experiment

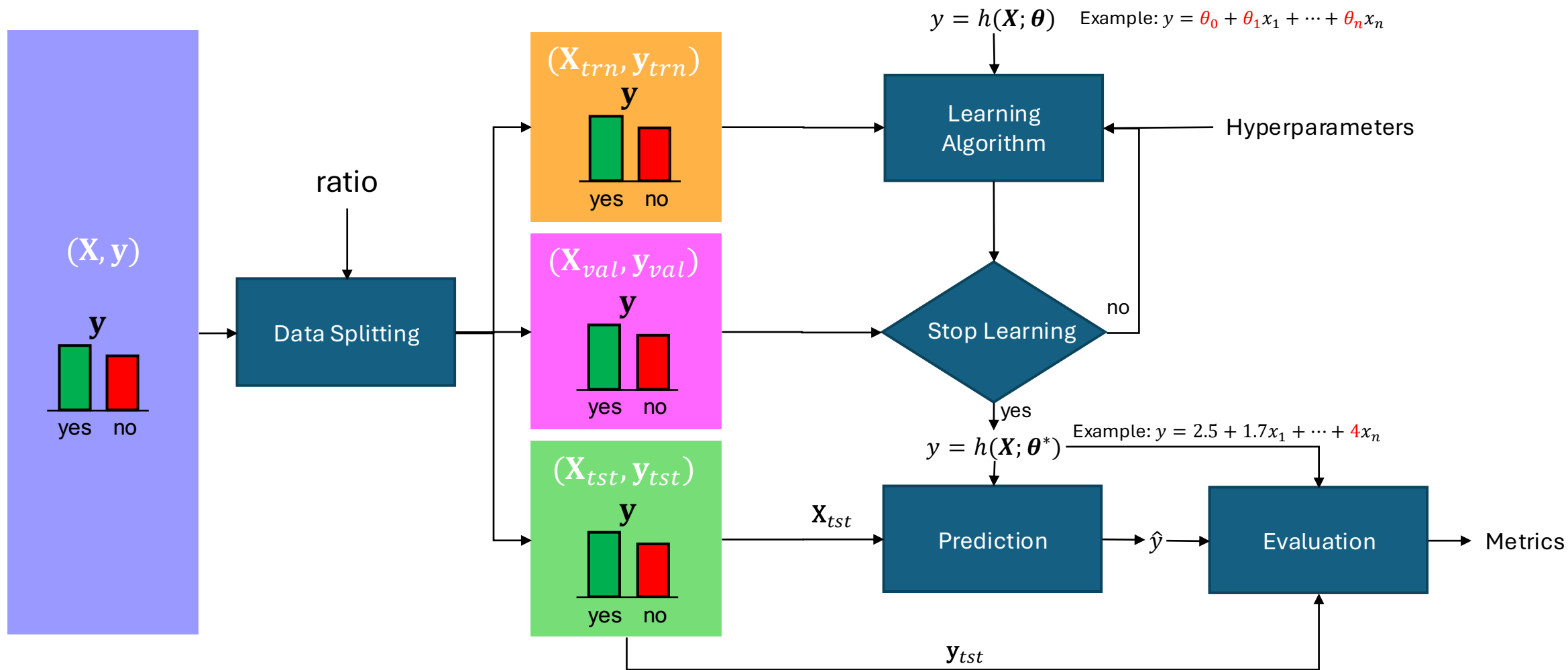


ML Experiment

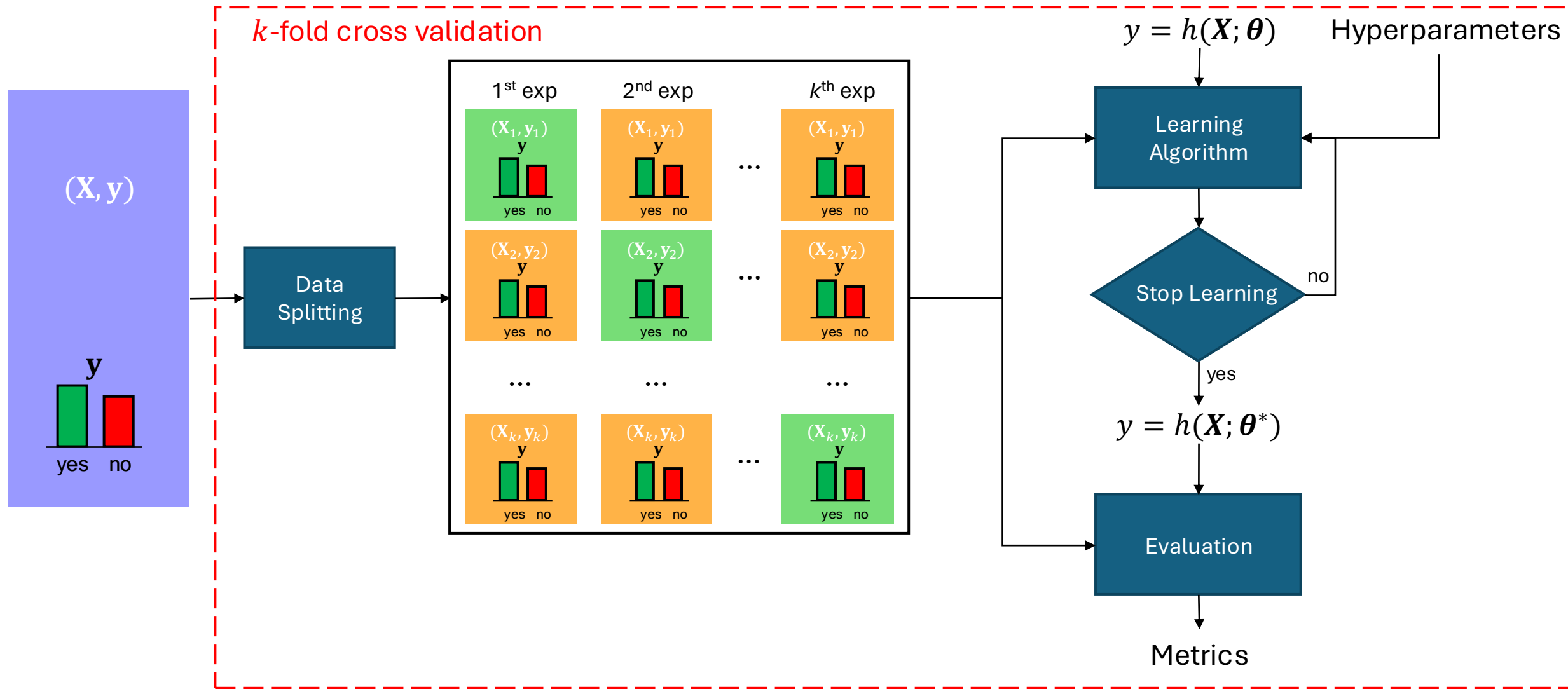


- Classical Machine Learning
 - Number of samples is not very large
 - Training samples: Test samples
 - 80:20
 - 70:30

ML Experiment

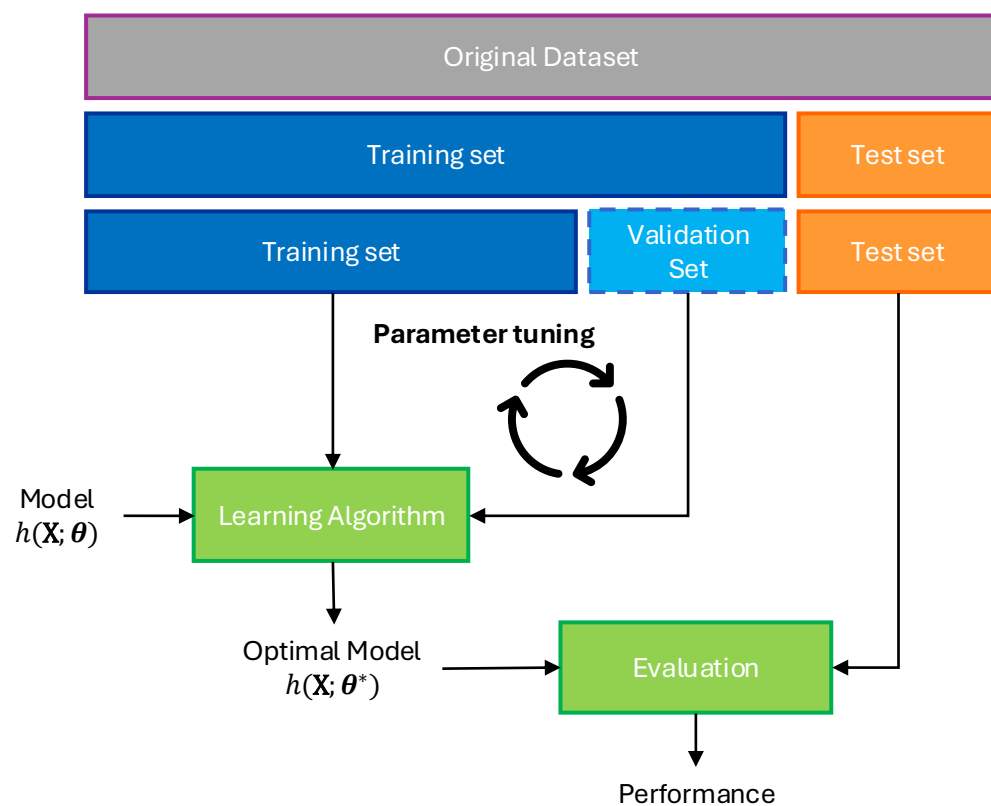


ML Experiment

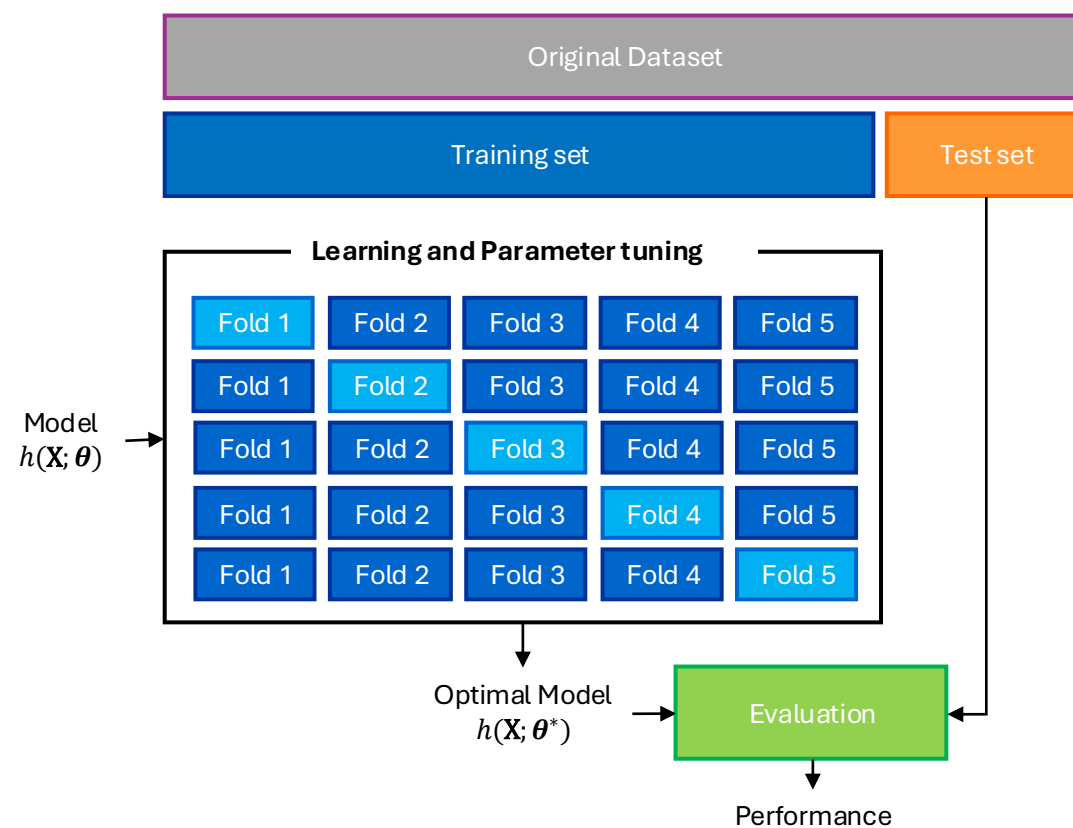


ML Experiment

Train Test Validate



K-Fold Cross Validation (Ex. 5-folds)



Evaluation

30 Minutes

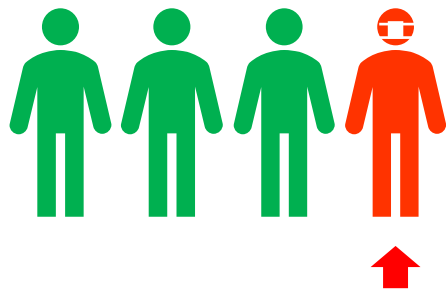
Types of Error

- Given a binary classification problem
 - **True-Positive (TP)** refers to any **positive sample** that was **correctly classified as positive**



Churn Prediction

TP: Correctly identify customer's decision as leaving our business for any churn customer

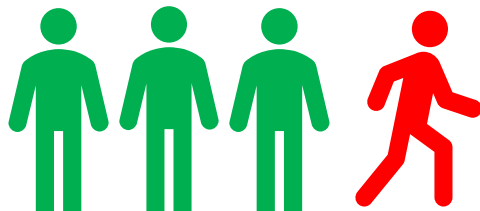


Disease Detection

TP: Correctly identify anyone with disease as a patient with disease

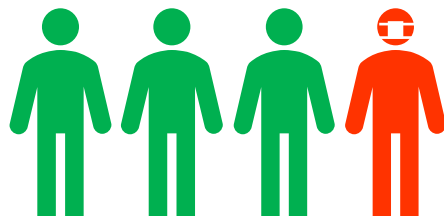
Types of Error

- Given a binary classification problem
 - **False-Positive (FP)** (a.k.a. Type-I Error) refers to any **negative sample** that was **incorrectly classified as positive**



Churn Prediction

FP: Incorrectly identify customer's decision to stay with our business for any churn customer



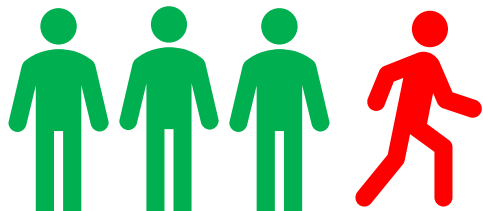
Disease Detection

FP: Incorrectly identify any healthy one as a patient with disease



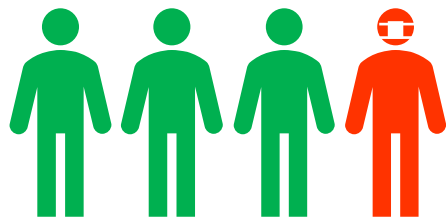
Types of Error

- Given a binary classification problem
 - **True-Negative (TN)** refers to any **negative sample** that was **correctly classified as negative**



Churn Prediction

TN: Correctly identify customer's decision to stay with our business for any non-churn customer



Disease Detection

TN: Correctly identify any healthy one as a healthy one



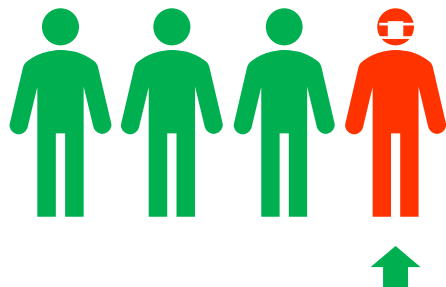
Types of Error

- Given a binary classification problem
 - **False-Negative (FN)** (a.k.a. Type-II Error) refers to any **positive sample** that was **incorrectly classified as negative**



Churn Prediction

FN: Incorrectly identify customer's decision to stay with our business for any churn customer



Disease Detection

FN: Incorrectly identify anyone with disease as a healthy one

Confusion Matrix

- For binary classification, a confusion matrix is a 2-by-2 matrix

Example: Given a prediction results ($\hat{y}$) and true labels (y) for churn prediction problem

		Actual	
		y	$\sim y$
Predict	y	TP	FP
	$\sim y$	FN	TN

y	$\hat{y}$
1	1
1	0
1	1
0	0
0	1
0	0
1	1
0	1
0	0
1	1



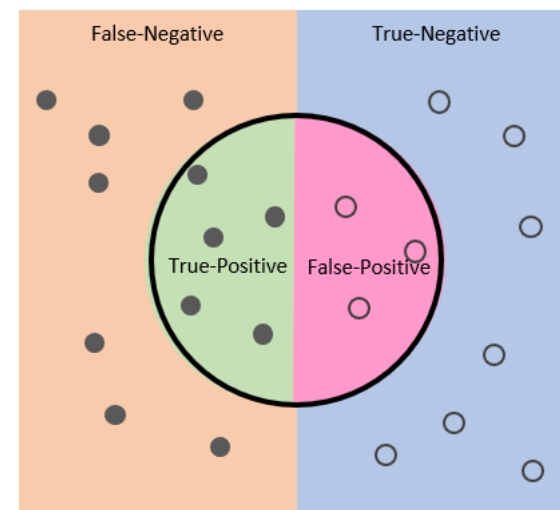
		Actual	
		y	$\sim y$
Predict	y	4	2
	$\sim y$	1	3

Evaluation Metrics

- Accuracy

- How many samples are correctly labeled out of all samples?
- Accuracy is a good measure when impact of *FP* and *FN* are similar and balanced class distribution
 - Distinguishing male and female chicks

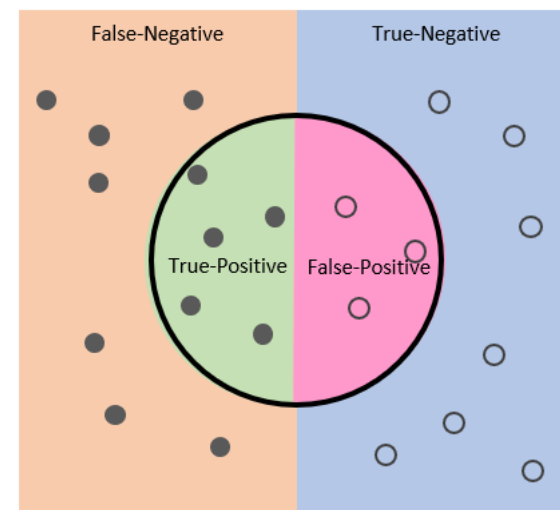
$$\text{accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$



Evaluation Metrics

- Precision [a.k.a. Positive Predictive Value (PPV)]
 - How many samples labeled as positive are actually positive samples?
 - Precision is a good measure when impact of *FP* must be minimized
 - Spam email detection

$$\text{precision} = \frac{TP}{TP + FP}$$

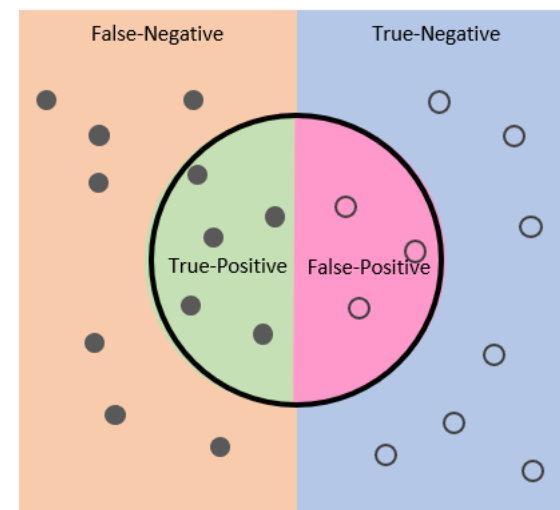


Evaluation Metrics

- Recall (a.k.a. Sensitivity)

- How many samples from all positive samples are correctly predicted?
- Recall is a good measure when impact of *FN* must be minimized
 - Churn Prediction, Disease Detection, Fraud Detection

$$\text{recall} = \frac{TP}{TP + FN}$$

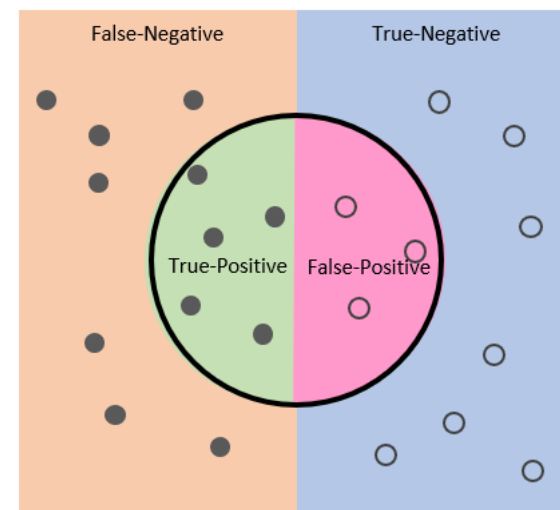


Evaluation Metrics

- Specificity

- How many samples from all negative samples are correctly predicted?
- Specificity is a good measure when we want to cover all TN and do not want FP
 - Drug Testing

$$\text{specificity} = \frac{TN}{TN + FP}$$

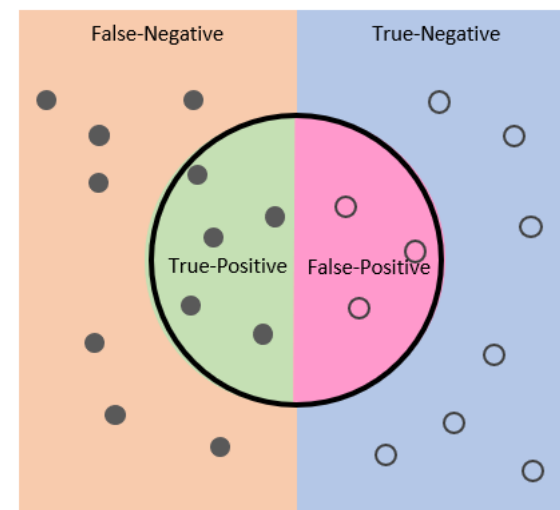


Evaluation Metrics

- F1

- F1 balances between precision and recall using harmonic mean
- F1 is a good measure when impact of *FP* and *FN* are different and imbalanced class distribution
 - Search Engine

$$\begin{aligned} F1 &= \frac{\frac{1}{P} + \frac{1}{R}}{2} \\ &= \frac{2PR}{P + R} \end{aligned}$$



Practical Evaluation

- FAQ



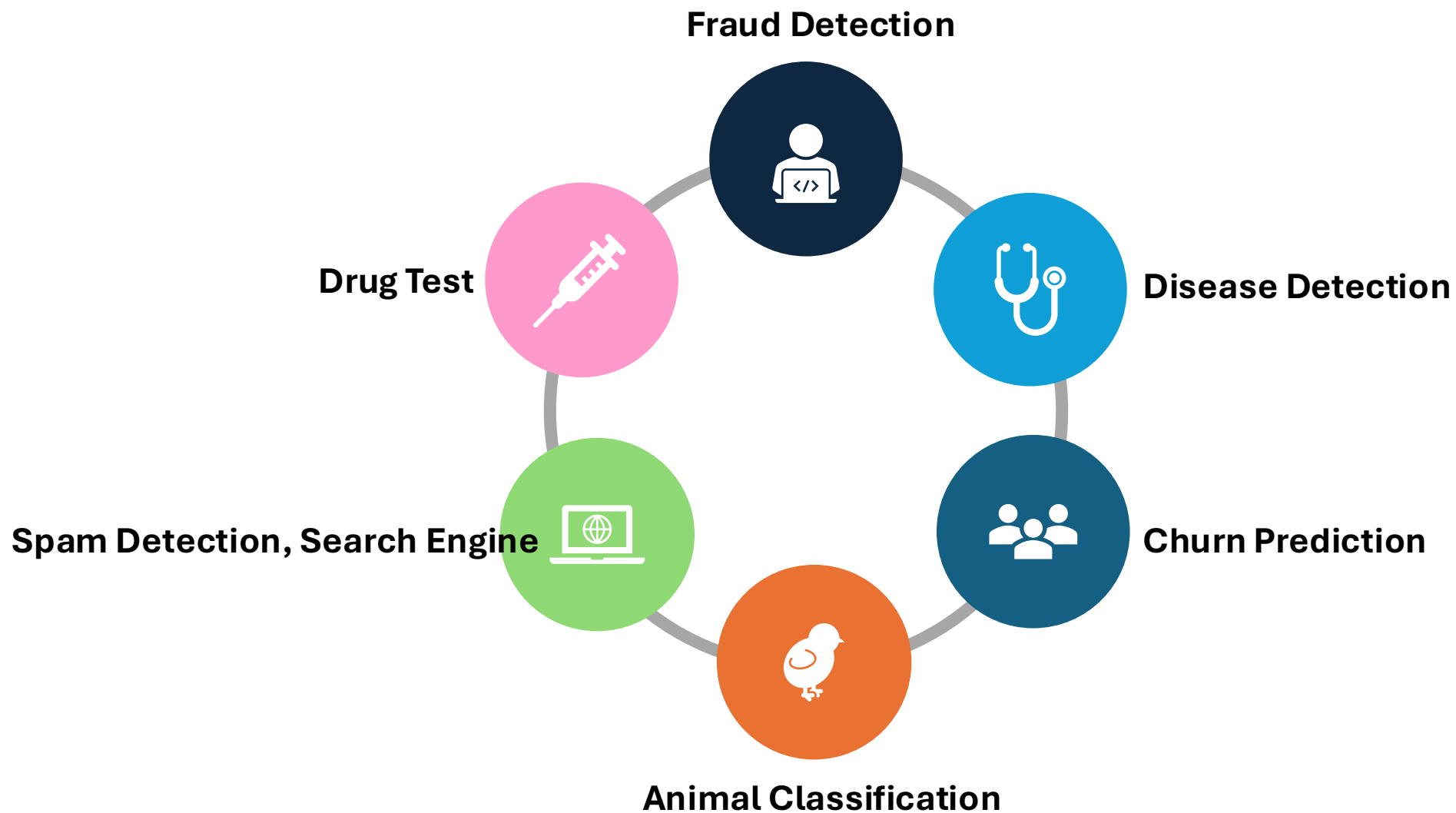
Accuracy: 0.90

Q: Is my model good enough?

A: Depends on application, impact, and expectations

Maximize objective, subjected to constraints

Practical Evaluation



Model Selection

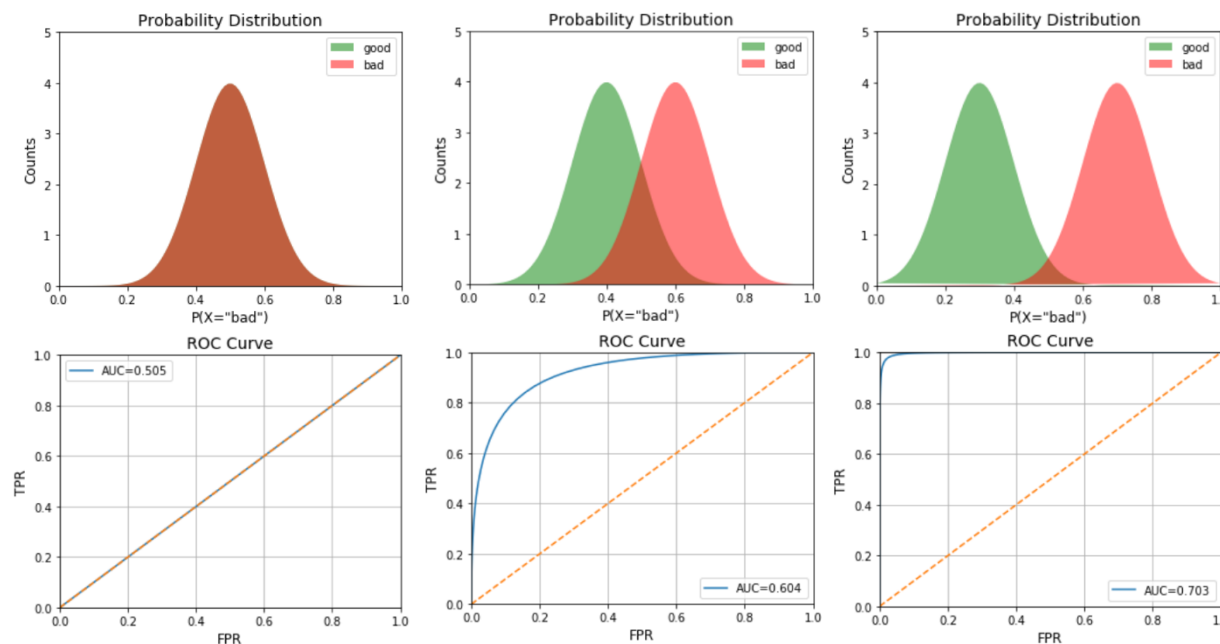
- Practical model selection is to **maximize objectives subjected to constraints**
 - Implement Covid-19 screening from X-ray machine at the airport
 - Which classifiers is the best for application if the average passenger per minute to each station is 3 minutes?

Classifiers	Sensitivity	Runtime (minutes)
A	90%	2
B	92%	2.5
C	95%	5

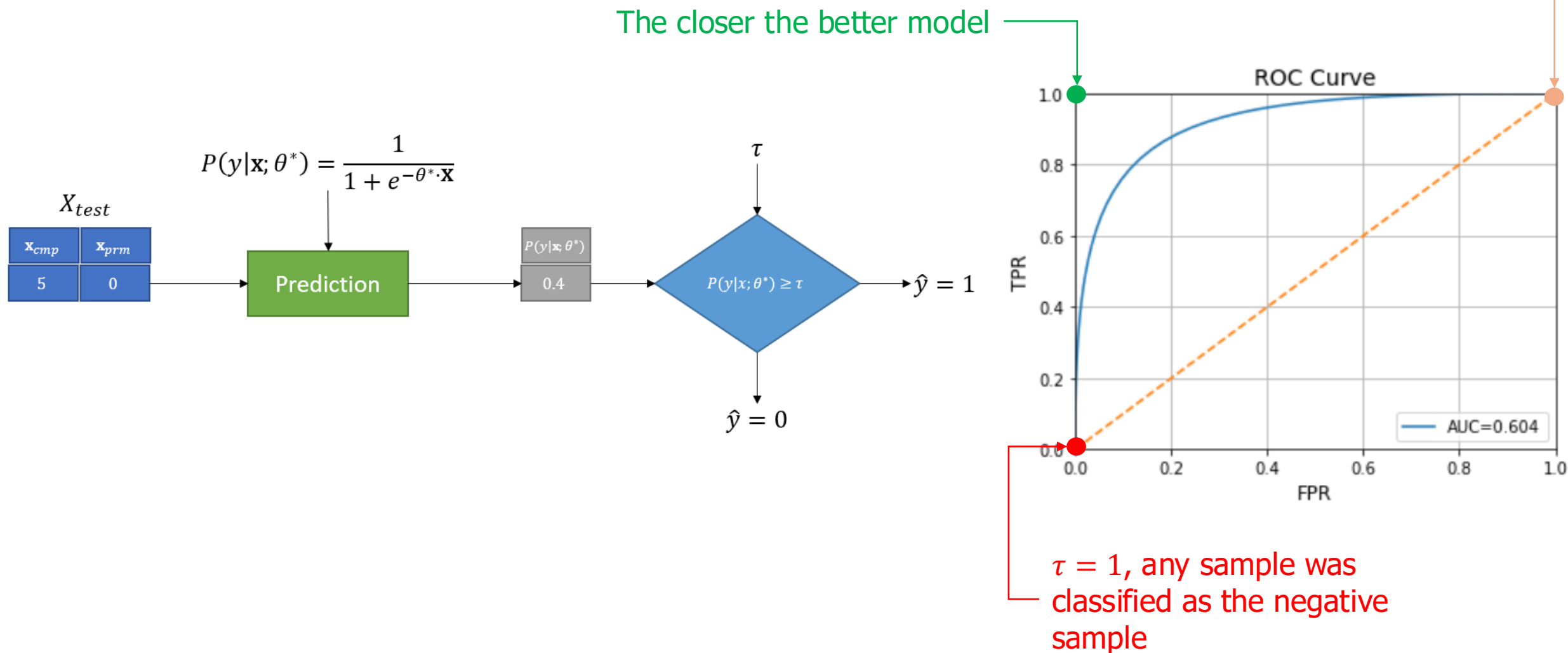
- Choose B because it maximizes sensitivity subjected to runtime

Receiver Operating Curve

- The ROC describes model performance with respect to decision threshold (τ)
 - Line plot between True-Positive Rate (TPR) on y-axis and False-Positive Rate (FPR)



Receiver Operating Curve



Evaluating Multiclass Classification

- Consider a microbial classification problem with 3 possible classes
 - Virus: The virus (V) was detected on specimen
 - Bacteria: The bacteria (B) was detected on specimen
 - Fungus: The fungus (F) was detected on specimen

		Actual (y)		
		V	B	F
Predict ($\hat{y}$)	V	2	1	2
	B	1	1	1
	F	2	1	3

y	$\hat{y}$
V	B
B	V
F	V
V	F
V	F
B	F
F	V
F	B
V	V
V	V
B	B
F	F
F	F
F	F

Evaluating Multiclass Classification

- Calculating evaluation metrics is based on
 - Micro Average
 - Macro Average

Evaluating Multiclass Classification

- Micro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 5 = 28$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: Fungus

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

$$precision = \frac{TP_V + TP_B + TP_F}{TP_V + TP_B + TP_F + FP_V + FP_B + FP_F} = \frac{16 + 14 + 6}{16 + 14 + 6 + 1 + 1 + 5} = \frac{36}{43} \approx 0.837$$

Evaluating Multiclass Classification

- Micro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: yes

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

$$recall = \frac{TP_V + TP_B + TP_F}{TP_V + TP_B + TP_F + FN_V + FN_B + FN_F} = \frac{16 + 14 + 6}{16 + 14 + 6 + 5 + 0 + 2} = \frac{36}{43} \approx 0.837$$

Evaluating Multiclass Classification

- Micro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: yes

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

$$f1 = \frac{2PR}{P+R} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

$$= \frac{TP_V + TP_B + TP_F}{TP_V + TP_B + TP_F + \frac{1}{2}(FP_V + FP_B + FP_F + FN_V + FN_B + FN_F)}$$

$$= \frac{16 + 14 + 6}{16 + 14 + 6 + \frac{1}{2}(1 + 1 + 5 + 5 + 0 + 2)} = \frac{36}{43} \approx 0.837$$

Evaluating Multiclass Classification

- Micro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: yes

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

There is no micro average for accuracy

$$\text{accuracy} = \frac{\text{\#correctly classified samples}}{\text{\#samples}} = \frac{16 + 14 + 6}{16 + 14 + 6 + 0 + 5 + 0 + 0 + 1 + 1} = \frac{36}{43} \approx 0.837$$

Evaluating Multiclass Classification

- Macro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

$$\text{precision}_V = \frac{TP_V}{TP_V + FP_V} = \frac{16}{16 + 1} = 0.941$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

$$\text{precision}_B = \frac{TP_B}{TP_B + FP_B} = \frac{14}{14 + 1} = 0.933$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: yes

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

$$\text{precision}_F = \frac{TP_F}{TP_F + FP_F} = \frac{6}{6 + 5} = 0.545$$

$$\therefore \text{precision}_{macro} = \frac{0.941 + 0.933 + 0.545}{3} = 0.807$$

Evaluating Multiclass Classification

- Macro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

$$\text{recall}_V = \frac{TP_V}{TP_V + FN_V} = \frac{16}{16 + 5} = 0.762$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

$$\text{recall}_B = \frac{TP_B}{TP_B + FN_B} = \frac{14}{14 + 0} = 1.000$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: Fungus

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

$$\text{recall}_F = \frac{TP_F}{TP_F + FN_F} = \frac{6}{6 + 2} = 0.750$$

$$\therefore \text{recall}_{macro} = \frac{0.762 + 1.000 + 0.750}{3} = 0.837$$

Evaluating Multiclass Classification

- Macro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

$$F1_V = \frac{TP_V}{TP_V + \frac{1}{2}(FP_V + FN_V)} = \frac{16}{16 + \frac{1}{2}(1 + 5)} = 0.842$$

$$F1 = \frac{2PR}{P + R} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

$$F1_B = \frac{TP_B}{TP_B + \frac{1}{2}(FP_B + FN_B)} = \frac{14}{14 + \frac{1}{2}(1 + 0)} = 0.966$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: yes

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

$$F1_F = \frac{TP_F}{TP_F + \frac{1}{2}(FP_F + FN_F)} = \frac{6}{6 + \frac{1}{2}(5 + 2)} = 0.632$$

$$\therefore F1_{macro} = \frac{0.842 + 0.966 + 0.632}{3} = 0.813$$

Evaluating Multiclass Classification

- Macro Average

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: V

$$TP_V = 16$$

$$TN_V = 14 + 0 + 1 + 6 = 21$$

$$FP_V = 0 + 1 = 1$$

$$FN_V = 0 + 5 = 5$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	V	16	0	5
	B	0	14	0
	F	1	1	6

Class: B

$$TP_B = 14$$

$$TN_B = 16 + 6 + 1 + 6 = 29$$

$$FP_B = 0 + 1 = 1$$

$$FN_B = 0 + 0 = 0$$

		Predict ($\hat{y}$)		
		V	B	F
Actual (y)	no	16	0	5
	wait	0	14	0
	yes	1	1	6

Class: yes

$$TP_F = 6$$

$$TN_F = 16 + 0 + 0 + 14 = 30$$

$$FP_F = 5 + 0 = 5$$

$$FN_F = 1 + 1 = 2$$

There is no macro average for accuracy

Summary

- The key concepts to take away
 - Positive class is the class of interest
 - Evaluation metrics can be derived from a confusion matrix
 - Selecting the evaluation metrics depends on problem
 - Selecting the best model is based on
 - Maximizing objectives subjected to constraints

Convolutional Neural Network

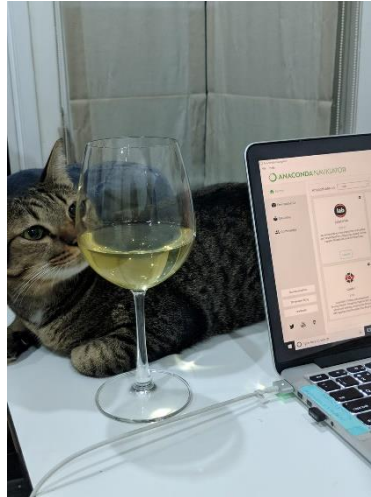
Neural Network with Self-Feature Extraction

Computer Vision Problem

Image Classification



A



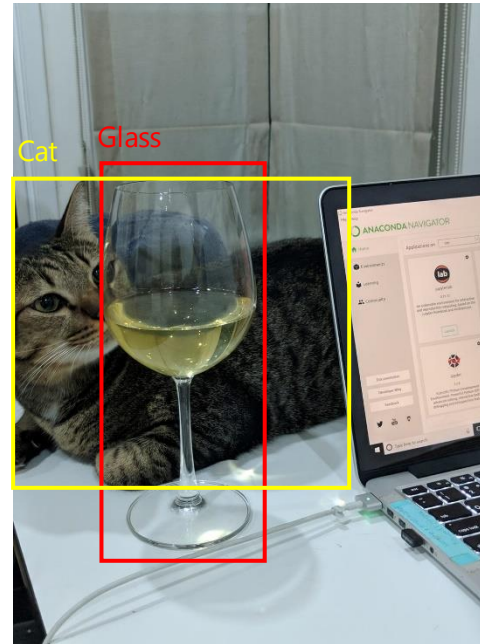
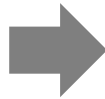
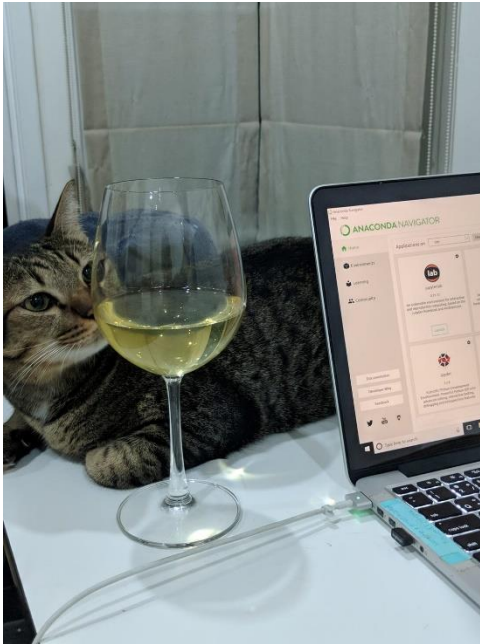
B

- Classify the type of an entity in a given image
 - A is an image of a cat
- What if there is more than one entity in a given image
 - B has a cat, a glass of wine, etc

Computer Vision Problem

Object Detection

- Detecting and recognize what are those objects in a given image



Computer Vision Problem

Neural Style Transfer

- Art generating by A.I.



Content Image



Style Image



Generated Image

Computer Vision Problem

Facial Recognition

- Given an image, recognize who is he/she or unknown?
- One shot learning problem

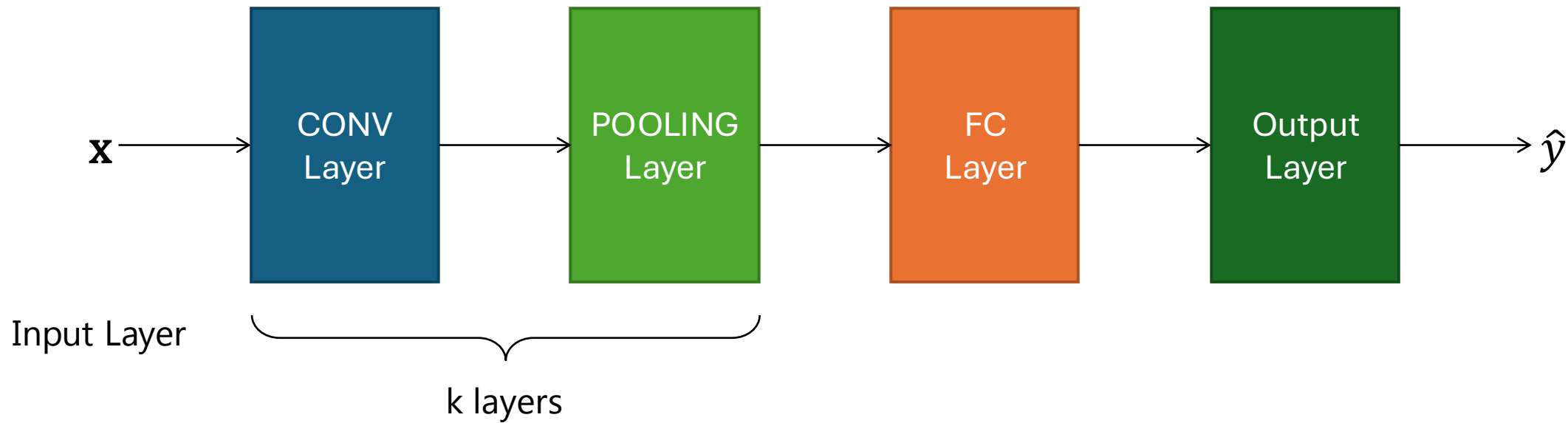


Ratchainant Thammasudjarit

Convolutional Neural Network

Model Architecture

- What are these layers?
- How does each layer work?

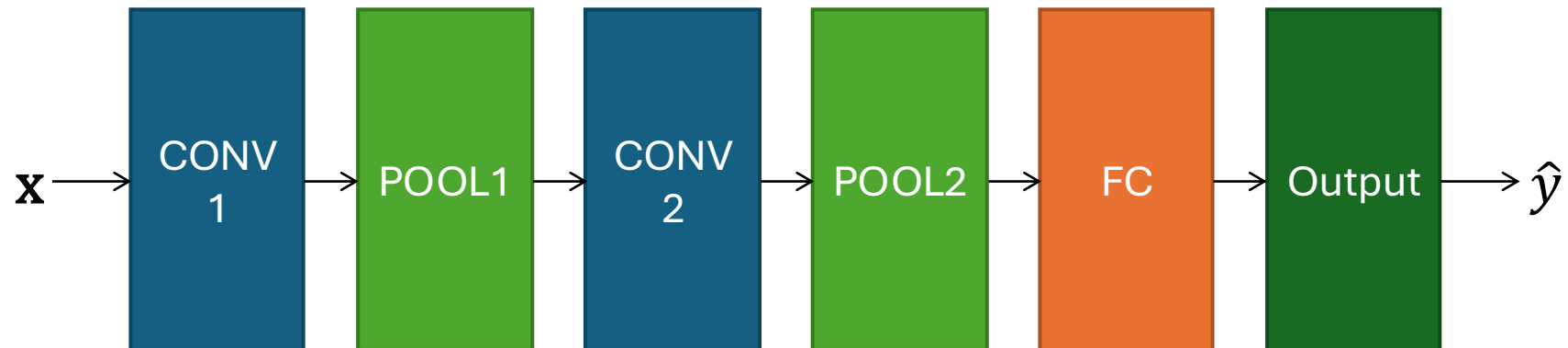


Convolutional Neural Network

Model Architecture

$k = 2$

- What are these layers?
- How does each layer work?



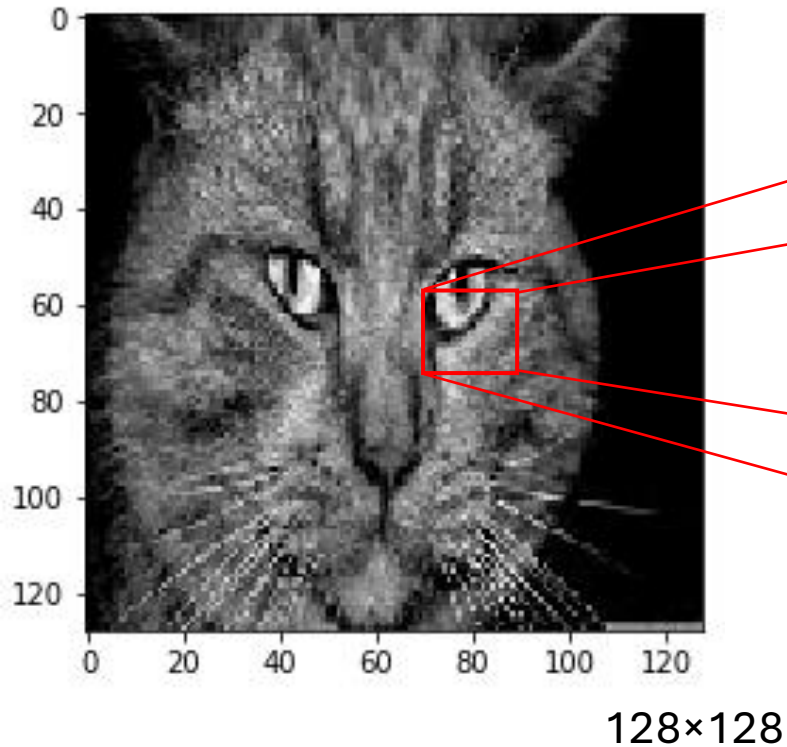
Input Layer



Input Layer

An image is a matrix

- A gray scale image has only 1 channel
 - Dimension is $(n_H \times n_W \times 1)$



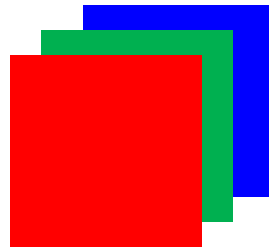
3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

n_H

n_W

Input Layer

Transforming RGB matrix to a training sample



10	20	90	80
140	30	40	130
110	100	50	60
120	160	150	70
30	90	140	20
110	50	60	150
160	80	70	120
10	130	100	40
50	110	160	70
90	10	30	130
140	40	20	100
80	120	150	60

flatten



10
20
...
150
70
30
90
...
100
40
50
110
...
150
60

Feature vector $\mathbf{x}$ has totally n_x features

*If we have a large image, e.g.
1024 × 960 pixels*

*The input features are 1024 × 960 × 3
2,949,120 features*

Not a good way to work

Convolutional Layer



Convolution of Matrices

Convolutional Operator

A grayscale image (I) of 6×6 pixels

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Convolutional operator returns the sum of pairwise multiplication between two matrices

Filter (F) (3×3 pixels)

1	0	-1
1	0	-1
1	0	-1

Convolution of Matrices

Convolutional Operator

Step 1

3	1	0	0	1	-1	2	7	4
1	1	5	0	8	-1	9	3	1
2	1	7	0	2	-1	5	1	3
0	1	3	1	7	8			
4	2	1	6	2	8			
2	4	5	2	3	9			

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 3 & 0 & 1 \\ 1 & 5 & 8 \\ 2 & 7 & 2 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (3 \times 1) + (0 \times 0) + (1 \times (-1)) + \dots + (2 \times (-1)) = -5$$

-5

Convolution of Matrices

Convolutional Operator

Step 2

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 0 & 1 & 2 \\ 5 & 8 & 9 \\ 7 & 2 & 5 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (0 \times 1) + (1 \times 0) + (2 \times (-1)) + \dots + (5 \times (-1)) = -4$$

-5	-4
----	----

Convolution of Matrices

Convolutional Operator

Step 3

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 1 & 2 & 7 \\ 8 & 9 & 3 \\ 2 & 5 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (1 \times 1) + (2 \times 0) + (7 \times (-1)) + \dots + (1 \times (-1)) = 0$$

-5	-4	0
----	----	---

Convolution of Matrices

Convolutional Operator

Step 4

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 2 & 7 & 4 \\ 9 & 3 & 1 \\ 5 & 1 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (2 \times 1) + (7 \times 0) + (4 \times (-1)) + \dots + (3 \times (-1)) = 8$$

-5	-4	0	8
----	----	---	---

Convolution of Matrices

Convolutional Operator

Step 5

3	0	1	2	7	4
1	1	5	0	8	-1
2	1	7	0	2	-1
0	1	1	0	3	-1
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 1 & 5 & 8 \\ 2 & 7 & 2 \\ 0 & 1 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (1 \times 1) + (5 \times 0) + (8 \times (-1)) + \dots + (3 \times (-1)) = -10$$

-5	-4	0	8
-10			

Convolution of Matrices

Convolutional Operator

Step 6

3	0	1	2	7	4
1	5	1	8	0	9
2	7	1	2	0	5
0	1	1	3	0	1
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 5 & 8 & 9 \\ 7 & 2 & 5 \\ 1 & 3 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (5 \times 1) + (8 \times 0) + (9 \times (-1)) + \dots + (1 \times (-1)) = -2$$

-5	-4	0	8
-10	-2		

Convolution of Matrices

Convolutional Operator

Step 7

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 8 & 9 & 3 \\ 2 & 5 & 1 \\ 3 & 1 & 7 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (8 \times 1) + (9 \times 0) + (3 \times (-1)) + \dots + (7 \times (-1)) = 2$$

-5	-4	0	8
-10	-2	2	

Convolution of Matrices

Convolutional Operator

Step 8

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 9 & 3 & 1 \\ 5 & 1 & 3 \\ 1 & 7 & 8 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (9 \times 1) + (3 \times 0) + (1 \times (-1)) + \dots + (8 \times (-1)) = 3$$

-5	-4	0	8
-10	-2	2	3

Convolution of Matrices

Convolutional Operator

Step 9

3	0	1	2	7	4
1	5	8	9	3	1
2	1	7	0	2	-1
0	1	1	0	3	-1
4	1	2	0	1	-1
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 2 & 7 & 2 \\ 0 & 1 & 3 \\ 4 & 2 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (2 \times 1) + (7 \times 0) + (2 \times (-1)) + \dots + (1 \times (-1)) = 0$$

-5	-4	0	8
-10	-2	2	3
0			

Convolution of Matrices

Convolutional Operator

Step 10

3	0	1	2	7	4
1	5	8	9	3	1
2	7	1	2	5	1
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 7 & 2 & 5 \\ 1 & 3 & 1 \\ 2 & 1 & 6 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (7 \times 1) + (2 \times 0) + (5 \times (-1)) + \dots + (6 \times (-1)) = -2$$

-5	-4	0	8
-10	-2	2	3
0	-2		

Convolution of Matrices

Convolutional Operator

Step 11

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 2 & 5 & 1 \\ 3 & 1 & 7 \\ 1 & 6 & 2 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (2 \times 1) + (5 \times 0) + (1 \times (-1)) + \dots + (2 \times (-1)) = -4$$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	

Convolution of Matrices

Convolutional Operator

Step 12

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 5 & 1 & 3 \\ 1 & 7 & 8 \\ 6 & 2 & 8 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (5 \times 1) + (1 \times 0) + (3 \times (-1)) + \dots + (8 \times (-1)) = -7$$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7

Convolution of Matrices

Convolutional Operator

Step 13

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	0	3	1	7
4	2	0	1	6	2
2	4	0	5	2	3

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 0 & 1 & 3 \\ 4 & 2 & 1 \\ 2 & 4 & 5 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (0 \times 1) + (1 \times 0) + (3 \times (-1)) + \dots + (5 \times (-1)) = -3$$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3			

Convolution of Matrices

Convolutional Operator

Step 14

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 1 & 6 \\ 4 & 5 & 2 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (1 \times 1) + (3 \times 0) + (1 \times (-1)) + \dots + (2 \times (-1)) = -2$$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2		

Convolution of Matrices

Convolutional Operator

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

Step 15

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

$$\begin{bmatrix} 3 & 1 & 7 \\ 1 & 6 & 2 \\ 5 & 2 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (3 \times 1) + (1 \times 0) + (7 \times (-1)) + \dots + (3 \times (-1)) = -3$$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	

Convolution of Matrices

Convolutional Operator

Step 16

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Sliding the filter from left to right and top to bottom
- Stride = 1 (slide by 1 pixel)

$$\begin{bmatrix} 1 & 7 & 8 \\ 6 & 2 & 8 \\ 2 & 3 & 9 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = (1 \times 1) + (7 \times 0) + (8 \times (-1)) + \dots + (9 \times (-1)) = -16$$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Convolution of Matrices

Image Shrink Problem

- Applying filter to the image directly has image shrink problem
- The deeper convolutional layer is the more shrink image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6×6

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4×4

Convolution of Matrices

Padding

0	0	0	0	0	0	0	0
0	3	0	1	2	7	4	0
0	1	5	8	9	3	1	0
0	2	7	2	5	1	3	0
0	0	1	3	1	7	8	0
0	4	2	1	6	2	8	0
0	2	4	5	2	3	9	0
0	0	0	0	0	0	0	0

Padding size = 1

- Adding pixel with value equal to zero as the image boundary

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	3	0	1	2	7	4	0	0
0	0	1	5	8	9	3	1	0	0
0	0	2	7	2	5	1	3	0	0
0	0	0	1	3	1	7	8	0	0
0	0	4	2	1	6	2	8	0	0
0	0	2	4	5	2	3	9	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Padding size = 2

Convolution of Matrices

Prevent image shrinking with padding

0	0	0	0	0	0	0	0
0	3	0	1	2	7	4	0
0	1	5	8	9	3	1	0
0	2	7	2	5	1	3	0
0	0	1	3	1	7	8	0
0	4	2	1	6	2	8	0
0	2	4	5	2	3	9	0
0	0	0	0	0	0	0	0

6 × 6 with pad size 1

*

Stride = 1

1	0	-1
1	0	-1
1	0	-1

3 × 3

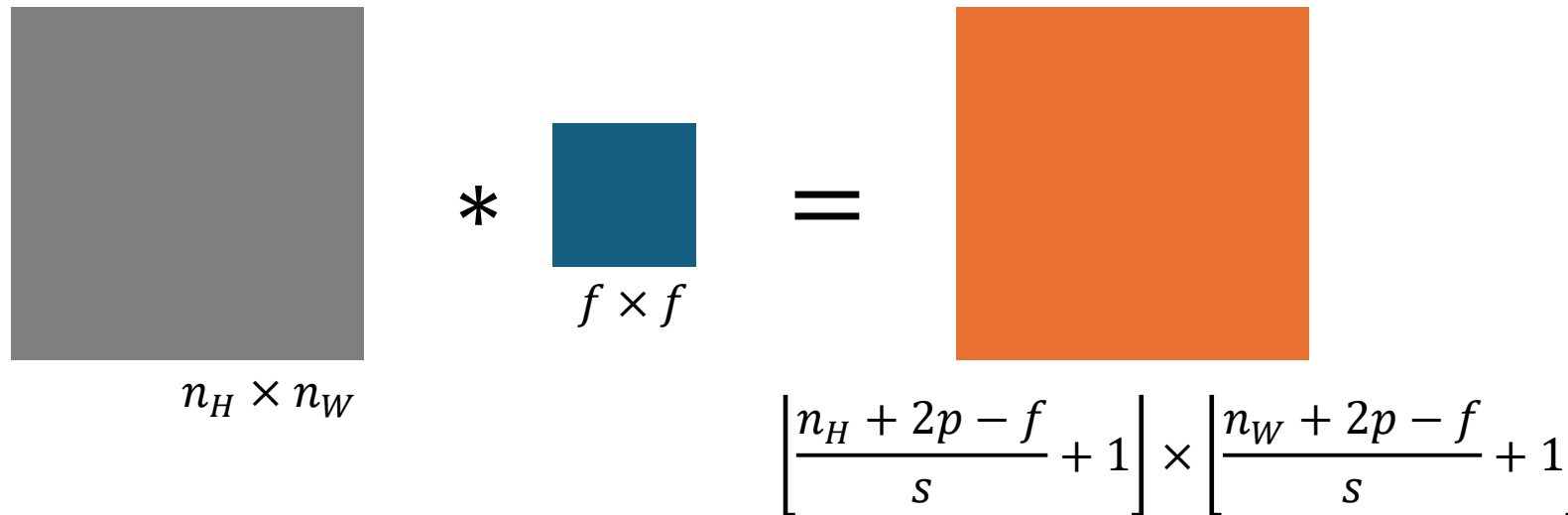
=

-5	-5	-6	-1	6	10
-12	-5	-4	0	8	11
-13	-10	-2	2	3	11
-10	0	-2	-4	-7	10
-7	-3	-2	-3	-16	12
-6	0	-2	1	-9	5

6 × 6

Convolution of Matrices

Summary of Filter, Padding, and Stride



where p be padding size
 s be stride
 f be filter size
 $\lfloor \cdot \rfloor$ be floor function

$$\begin{matrix} \text{Gray Matrix} & * & \text{Blue Matrix} & = & \text{Orange Matrix} \\ n_H \times n_W & & f \times f & & \end{matrix}$$
$$\left\lfloor \frac{n_H + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W + 2p - f}{s} + 1 \right\rfloor$$

Convolution of Volume Matrices

A color image as a volume matrix (3D)



$$n_H \times n_W \times n_C$$

*

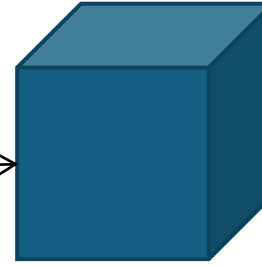


$$f \times f \times n_C$$

=



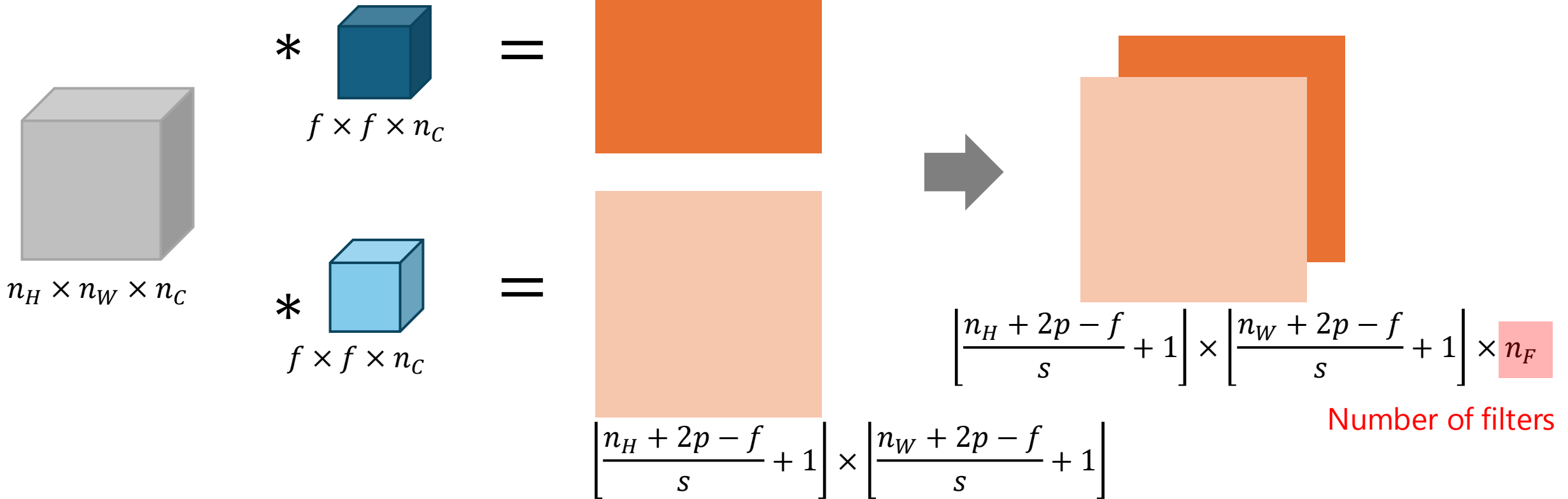
$$\left\lfloor \frac{n_H + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W + 2p - f}{s} + 1 \right\rfloor$$



Simplified Drawing of volume matrix

Convolution of Volume Matrices

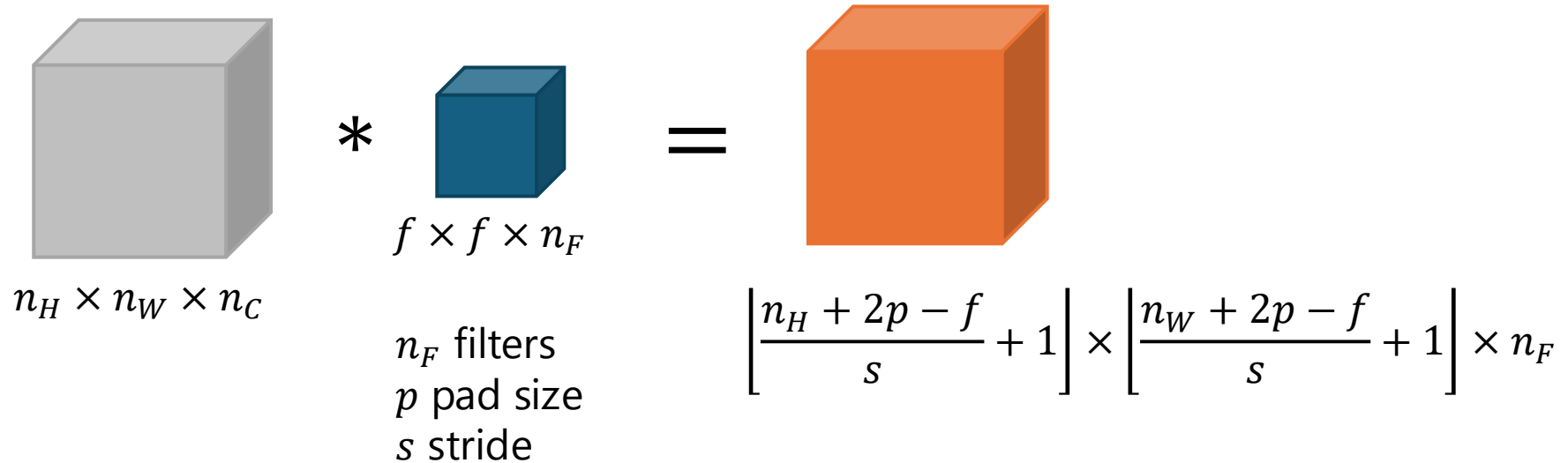
Multiple Filters



Convolution of Volume Matrices

Multiple Filters (Summary)

f is usually an odd number because we want the symmetric convolution of the central pixel



$n_H \times n_W \times n_C$

$*$

$f \times f \times n_F$

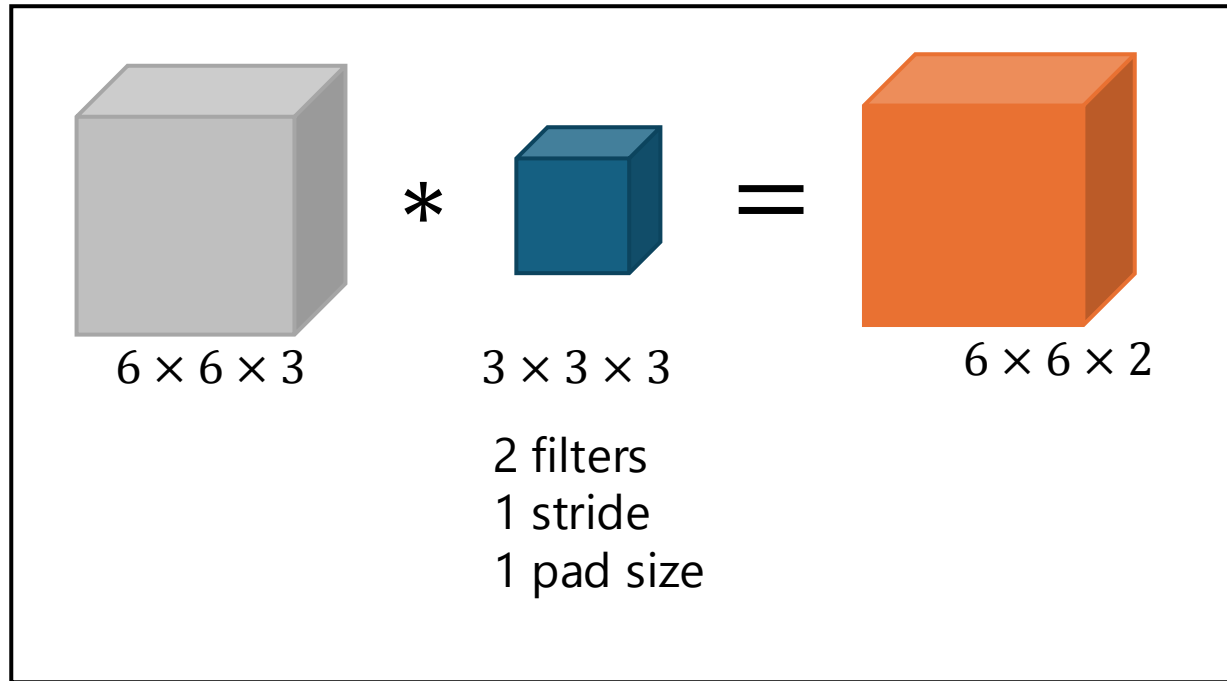
$=$

$\left\lfloor \frac{n_H + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W + 2p - f}{s} + 1 \right\rfloor \times n_F$

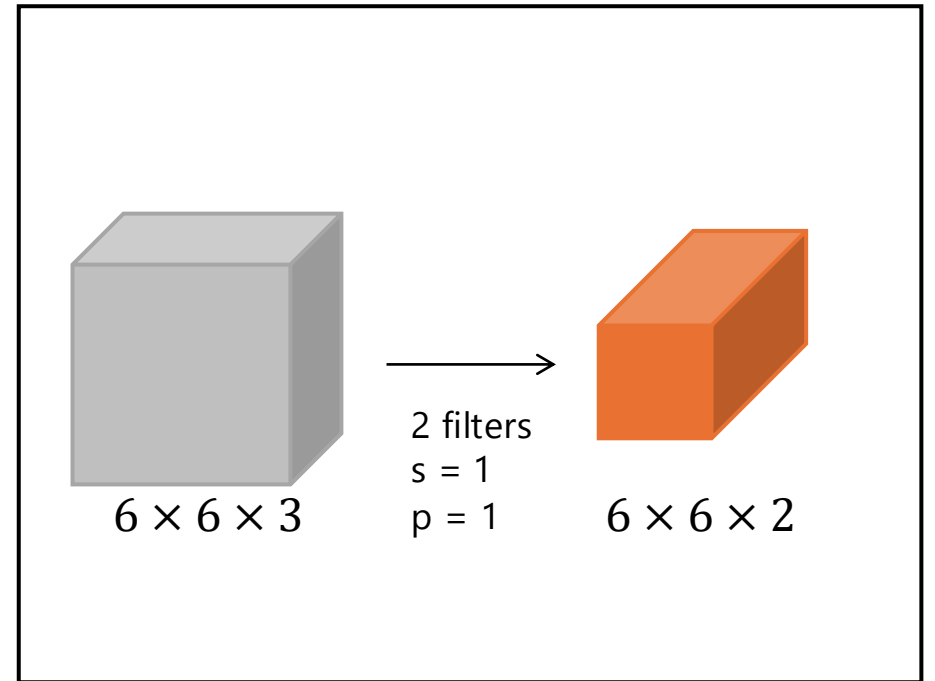
n_F filters
 p pad size
 s stride

Convolution of Volume Matrices

Multiple Filters (Example)



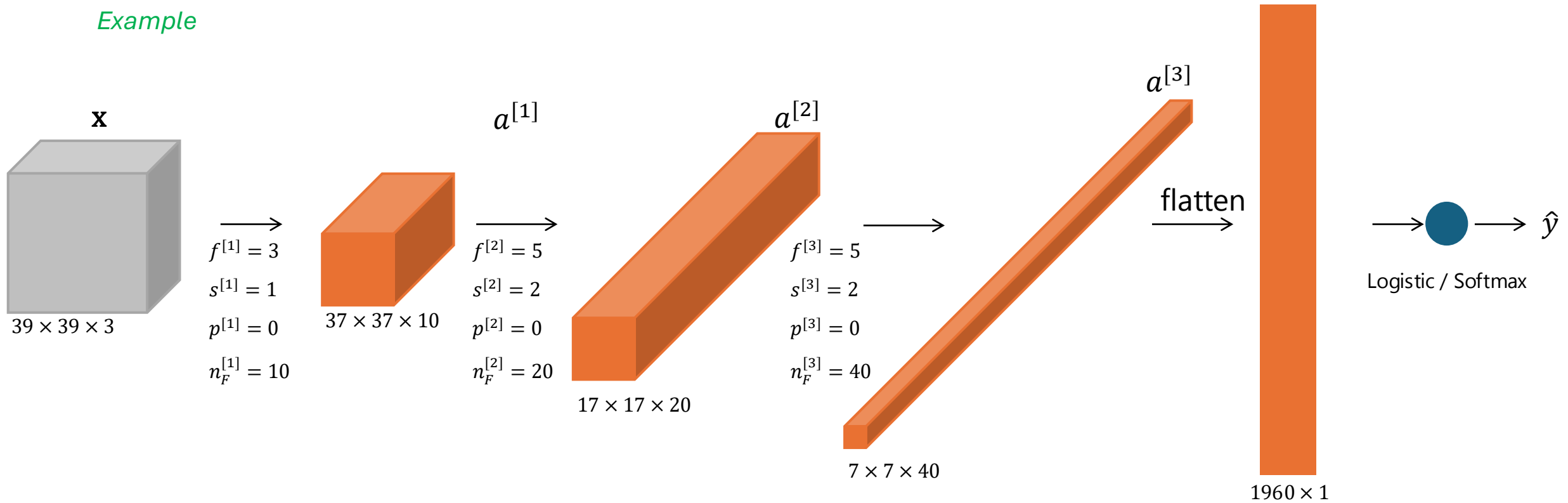
Simplified Drawing



A Simple CNN

Example

n_H and n_W are decreasing in the deeper convolutional layer
 n_C is increasing in the deeper convolutional layer



Summary of Notations

Let the layer l be a convolutional layer

$f^{[l]}$ denotes the filter size

$p^{[l]}$ denotes the pad size

$s^{[l]}$ denotes the stride

$n_F^{[l]}$ denotes the number of filters

Each filter has dimension $f^{[l]} \times f^{[l]} \times n_F^{[l-1]}$

Activation $a^{[l]}$ has dimension $n_H^{[l]} \times n_W^{[l]} \times n_F^{[l]}$

For m training samples, the activation

$A^{[l]}$ has dimension $m \times n_H^{[l]} \times n_W^{[l]} \times n_F^{[l]}$

Weights has dimension $f^{[l]} \times f^{[l]} \times n_F^{[l-1]} \times n_F^{[l]}$

Bias has dimension $1 \times 1 \times 1 \times n_F^{[l]}$

Input has dimension $n_H^{[l-1]} \times n_W^{[l-1]} \times n_F^{[l-1]}$

Output has dimension $n_H^{[l]} \times n_W^{[l]} \times n_F^{[l]}$

where

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$n_W^{[l]} = \left\lfloor \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

Pooling Layer



Max Pooling

Let $f = 2$ and $s = 2$

Step 1

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

9

- Max pooling takes the maximum value

Max Pooling

Let $f = 2$ and $s = 2$

Step 2

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

9	2
---	---

- Max pooling takes the maximum value

Max Pooling

Let $f = 2$ and $s = 2$

Step 3

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

9	2
6	

- Max pooling takes the maximum value

Max Pooling

Let $f = 2$ and $s = 2$

Step 4

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

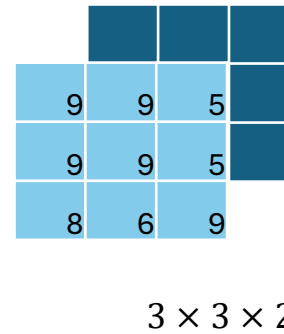
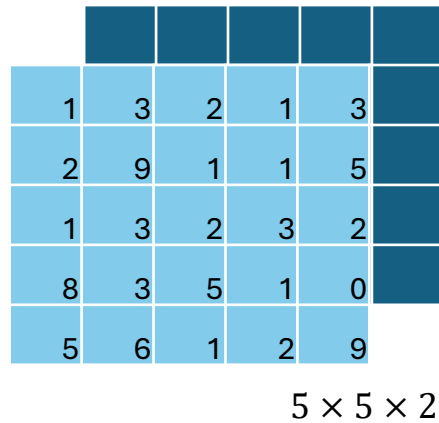
9	2
6	3

- Max pooling takes the maximum value

Max Pooling

Let $f = 3$ and $s = 1$

- Multiple Channels
 - Max Pooling is applied to each channel



Pooling Layer

Summary

Hyperparameters

f denotes filter size

s denotes stride

Common settings

- $f = 2, s = 2$
- $f = 3, s = 3$

Pooling

- Max (frequently used)
- Average (Rarely used)

I/O

Input has dimension $n_H \times n_W \times n_C$

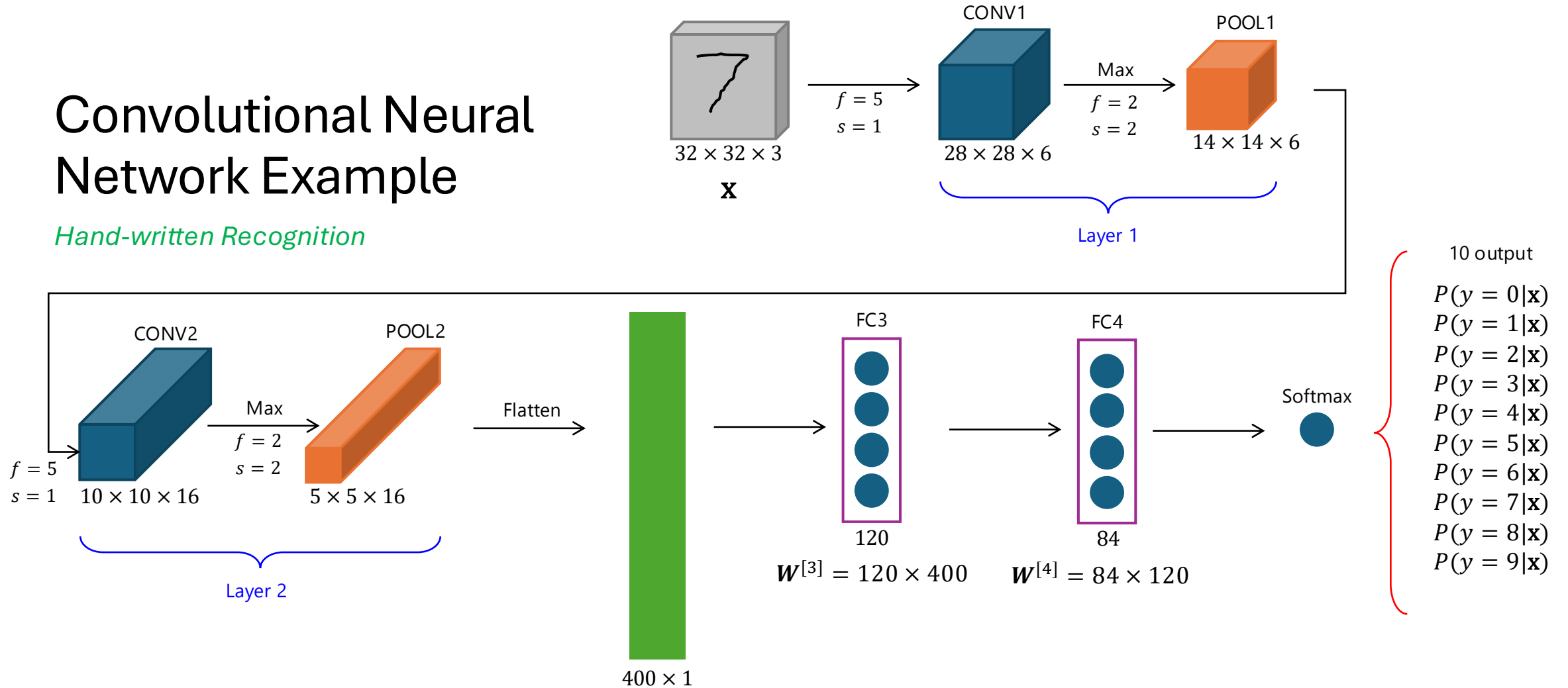
Output has dimension $\left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \times n_C$

Parameter learning

Not required

Convolutional Neural Network Example

Hand-written Recognition

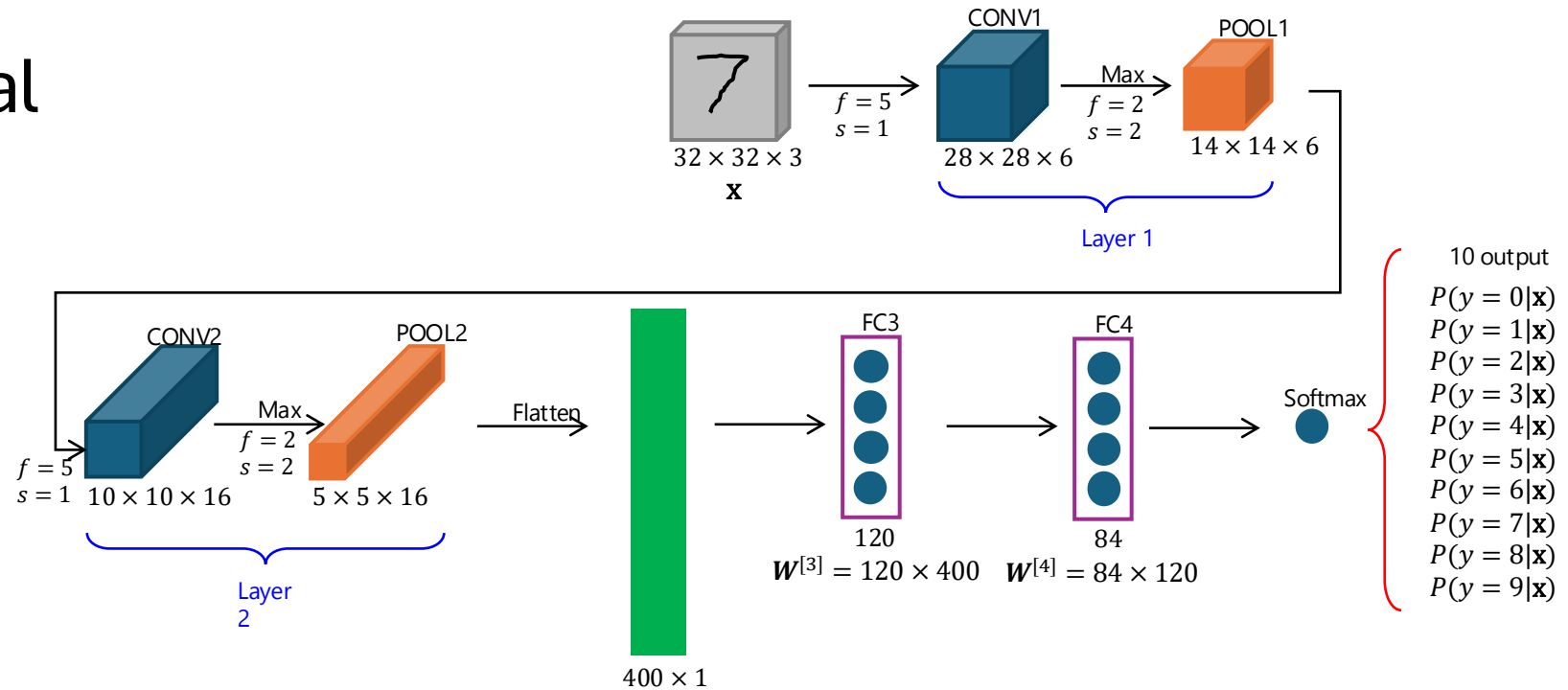


CONV $\rightarrow$ POOL $\rightarrow$ CONV $\rightarrow$ POOL $\rightarrow$ FC $\rightarrow$ FC $\rightarrow$ Softmax

Convolutional Neural Network Example

The change of dimension

Layers	Activation Shape	Activation Size
Input	$32 \times 32 \times 3$	3,072
CONV1	$28 \times 28 \times 6$	4,704
POOL1	$14 \times 14 \times 6$	1,176
CONV2	$10 \times 10 \times 16$	1,600
POOL2	$5 \times 5 \times 16$	400
FC3	120×1	120
FC4	84×1	84
Softmax	10×1	10



n_H and n_W are decreasing in the deeper convolutional layer

n_C is increasing in the deeper convolutional layer

Classic CNN

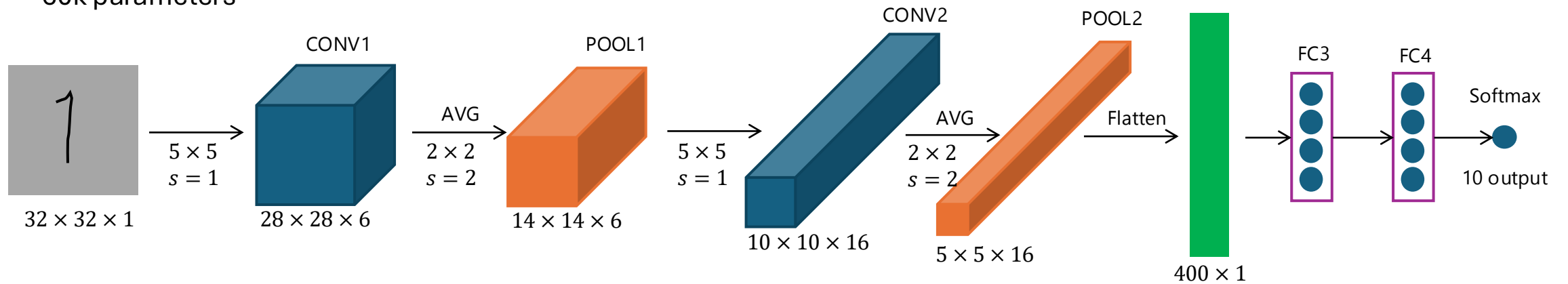


LeNet-5

Hand-written Recognition (Gray Image)

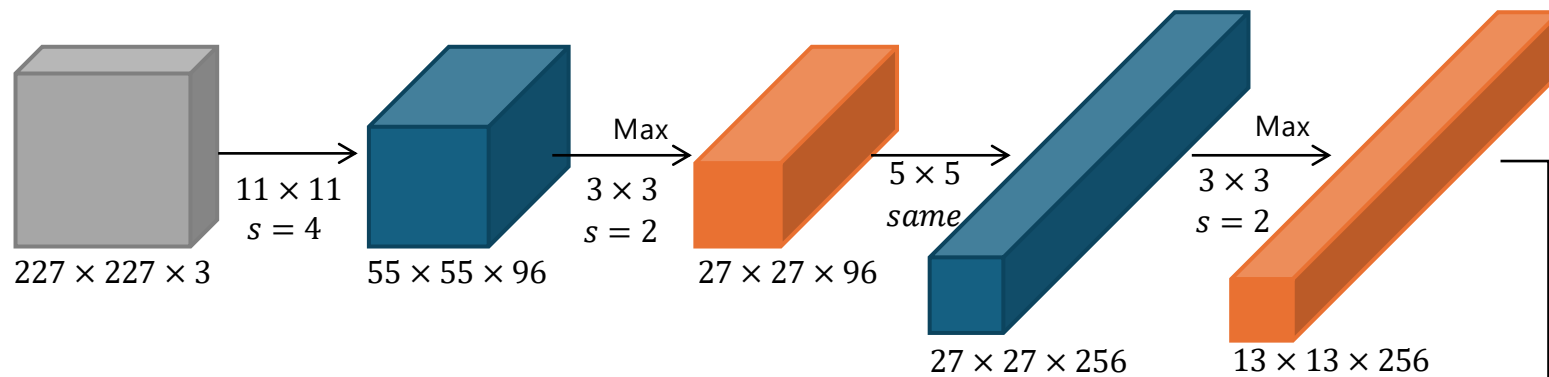
CONV → POOL → CONV → POOL → FC → FC → Softmax

~ 60k parameters

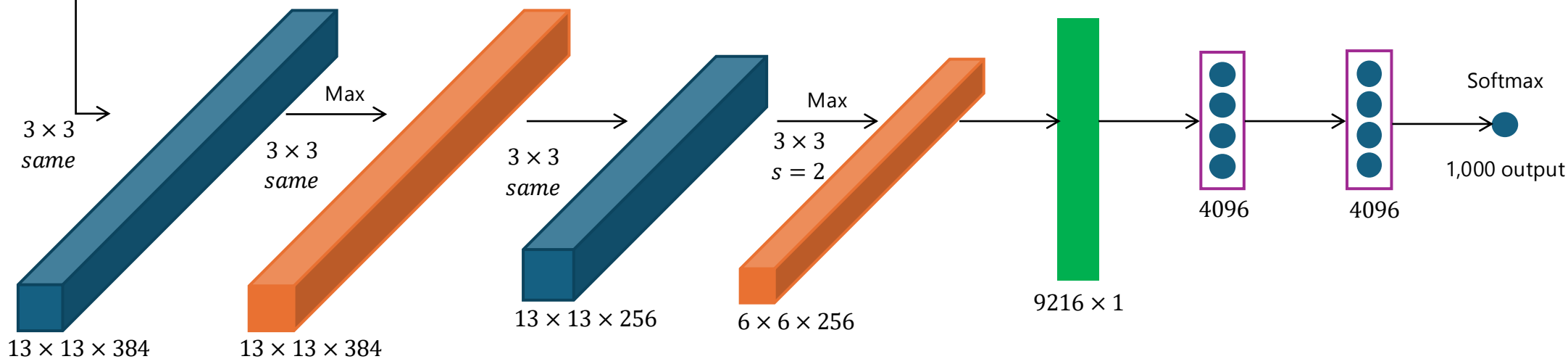


AlexNet

Similar to LeNet but much bigger



~ 60M parameters

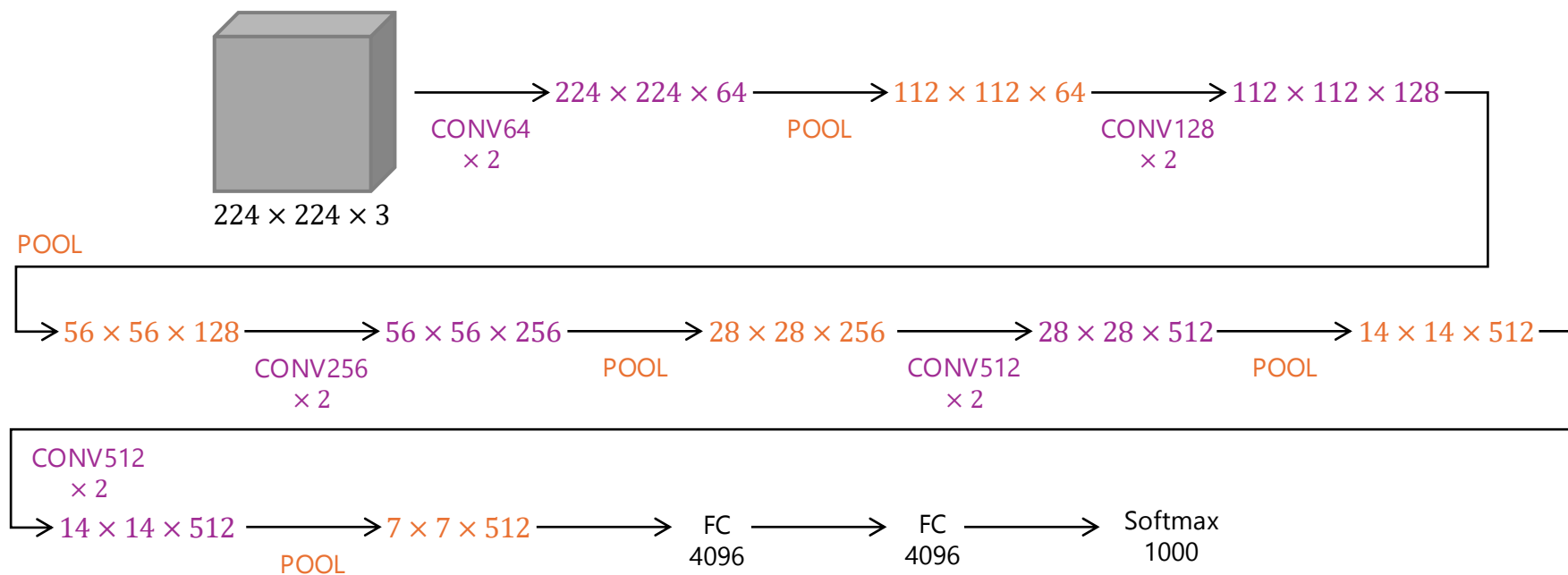


Training Data: ImageNet

VGG-16

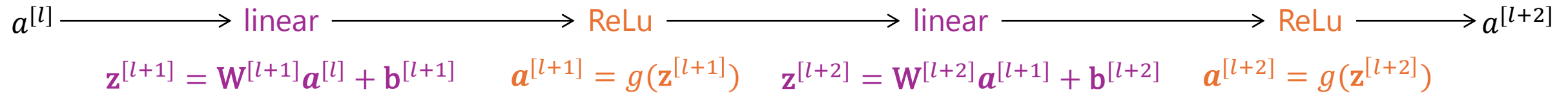
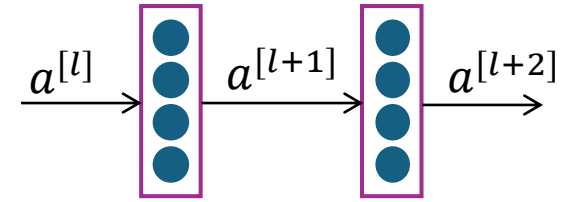
Simplified architecture

- 3×3 filter
- $s = 1$
- Same padding

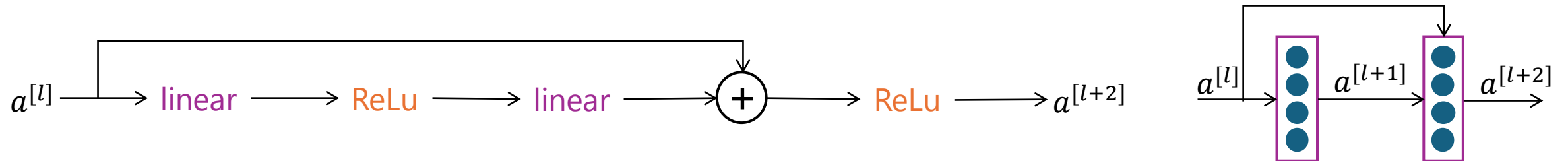


Residual Network

Normal Block

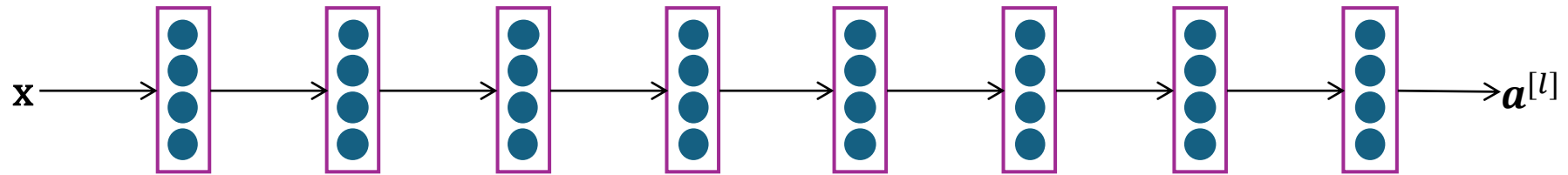


Residual Block

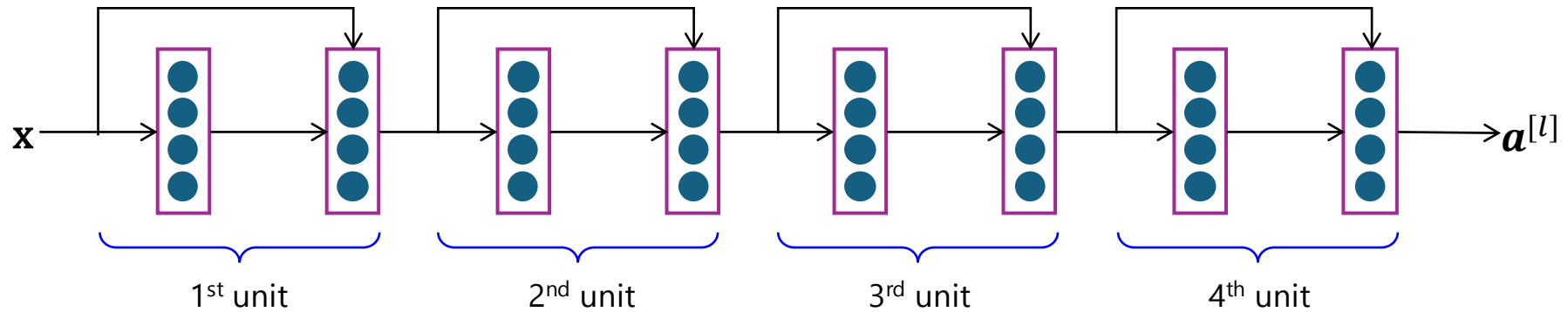


Residual Network

Plain Network

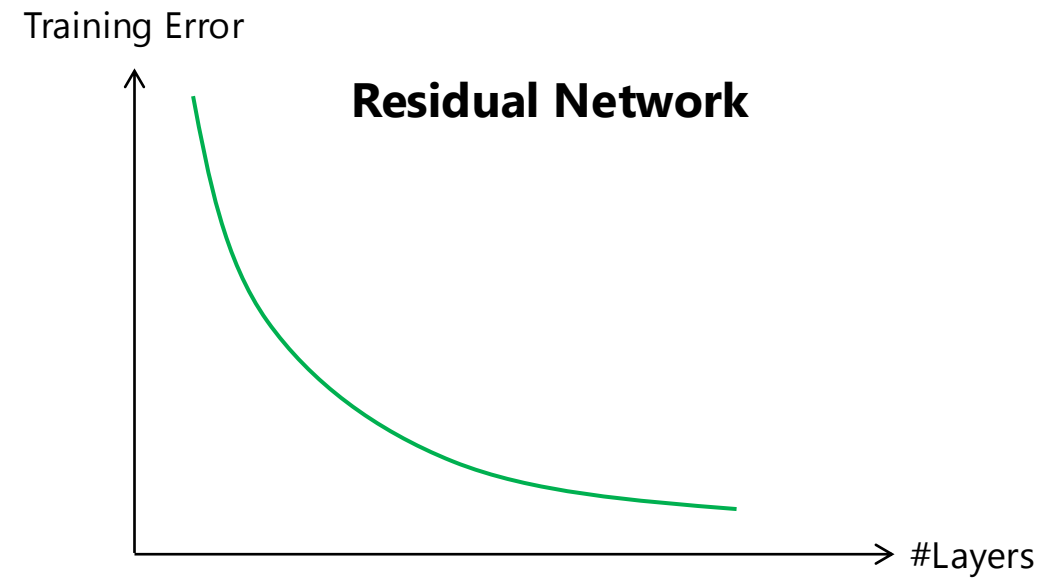
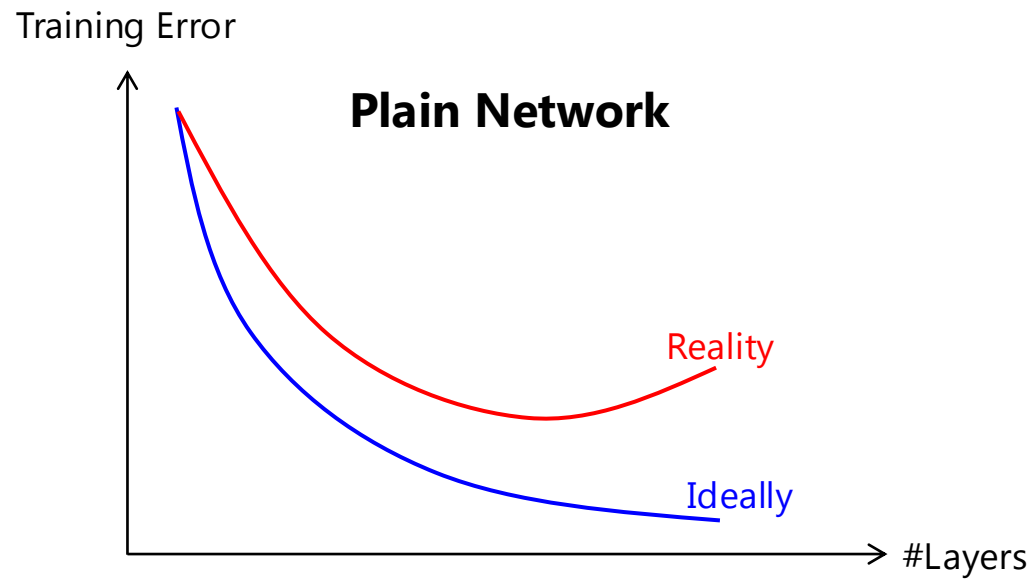


Residual Network



Residual Network

ResNet helps to handle gradient exploding



Data Augmentation

When you data is not enough

- Mirroring



Data Augmentation

When you data is not enough



- Random Cropping



Data Augmentation

When you data is not enough

- Rotation

